



南京航空航天大学
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

人工智能内容生成 (AIGC) 技术与应用



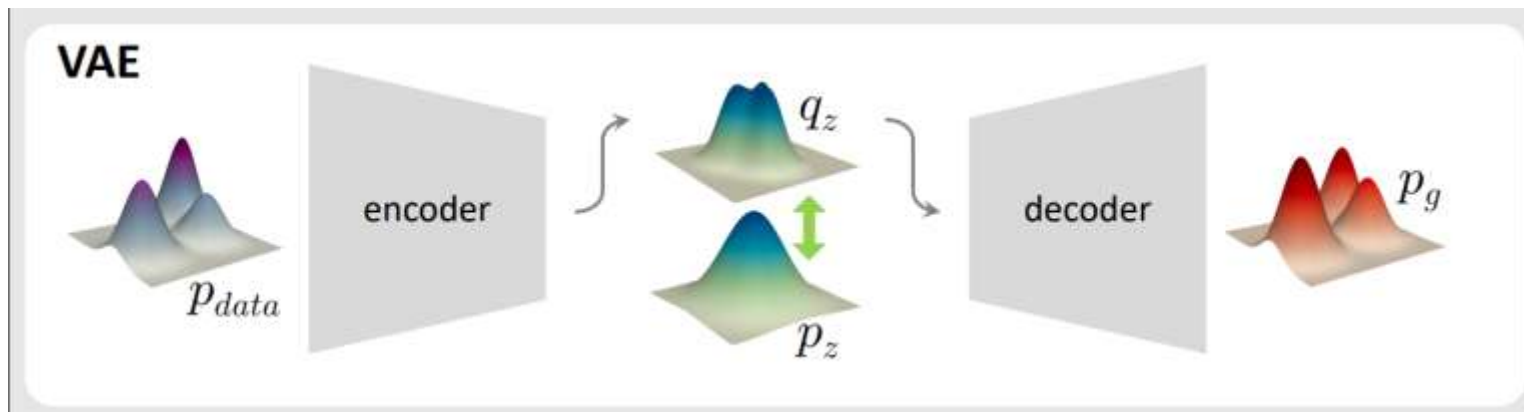
第五章

扩散去噪生成模型

深度生成式模型范式：简单分布采样 + 复杂映射

深度生成式模型范式：简单分布采样 + 复杂映射

- VAE: ①自编码器驱动z-x映射+②KL损失约束隐变量z服从标准高斯



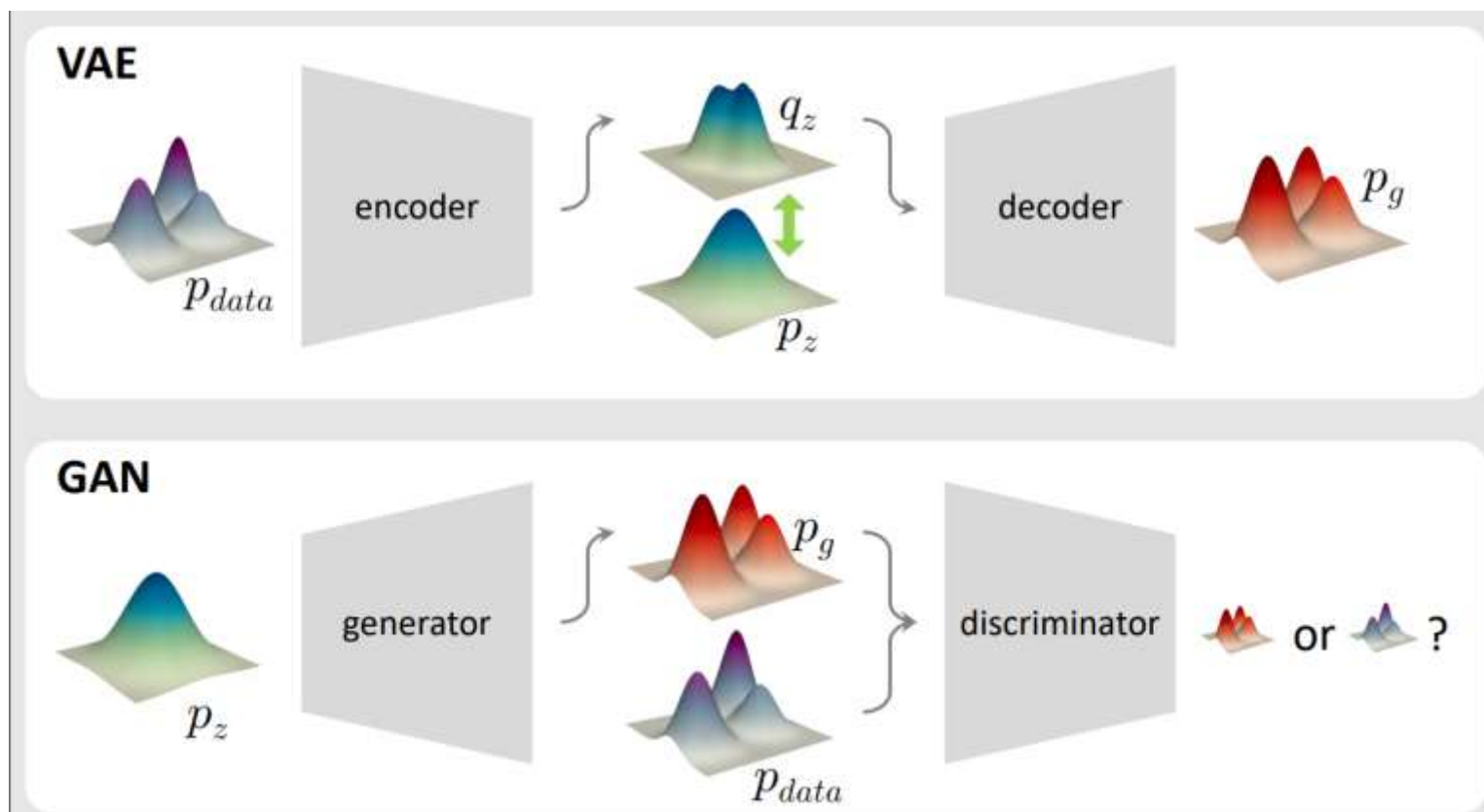
$$\mathcal{L} = \underbrace{\Sigma_x \{ |x - x'| \}}_{\text{重构损失}} + \underbrace{KL(N(\mu_x, \sigma_x^2) | N(0, 1))}_{\text{KL损失}}$$

重构损失

KL损失

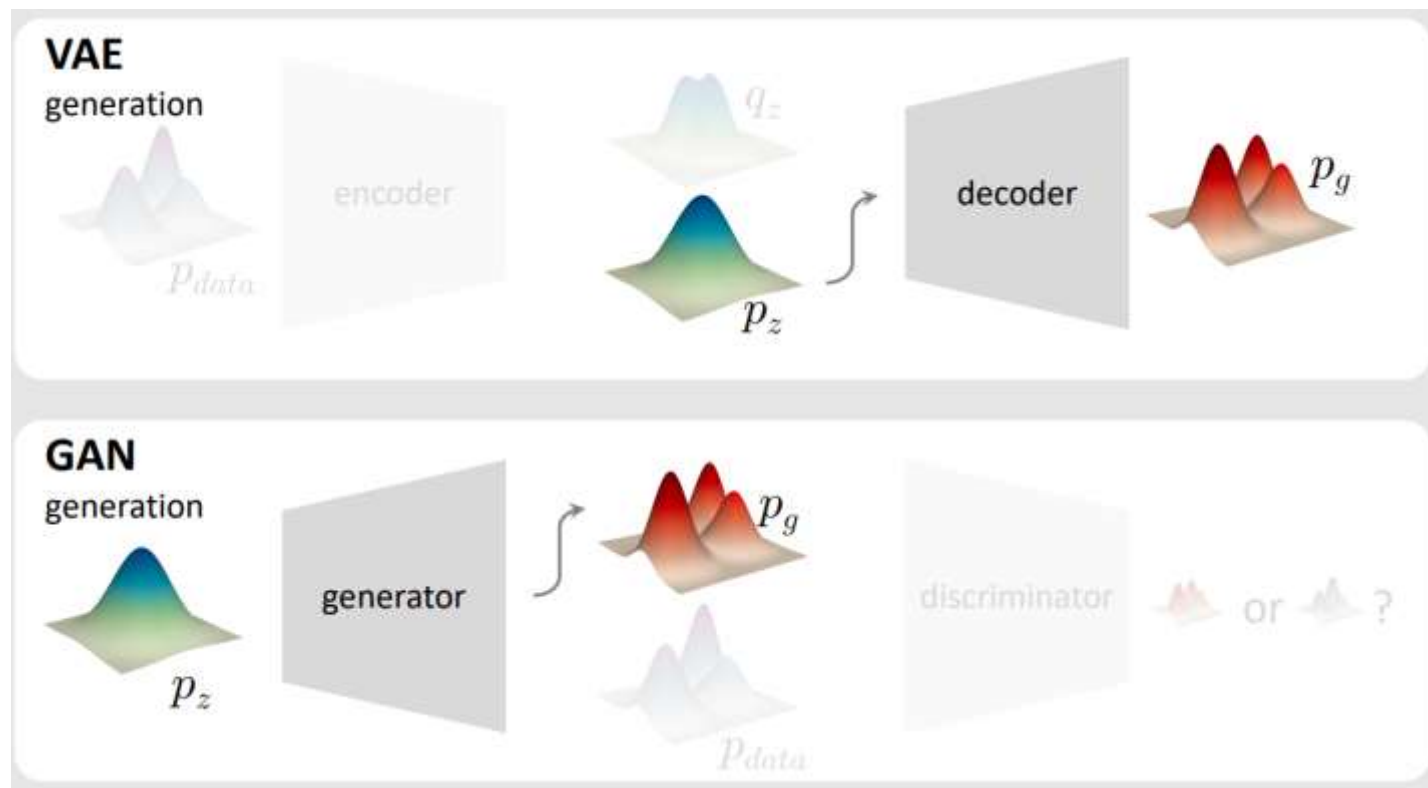
深度生成式模型范式：简单分布采样 + 复杂映射

- VAE: ①自编码器驱动x-z-x映射+②KL损失约束隐变量z服从标准高斯
- GAN: 噪声输入服从标准高斯+ 生成器驱动z-x映射

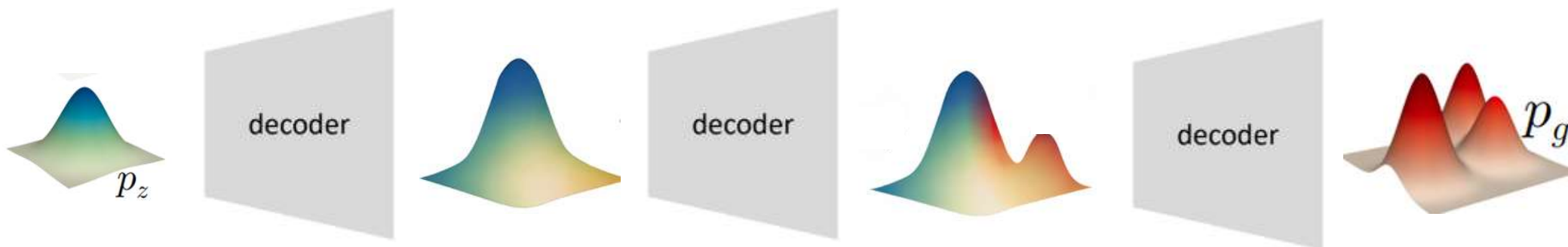


深度生成式模型范式：简单分布采样 + 复杂映射

- VAE 和GAN生成过程都是使用一次模型迭代构建 z - x 的映射，生成效果不佳

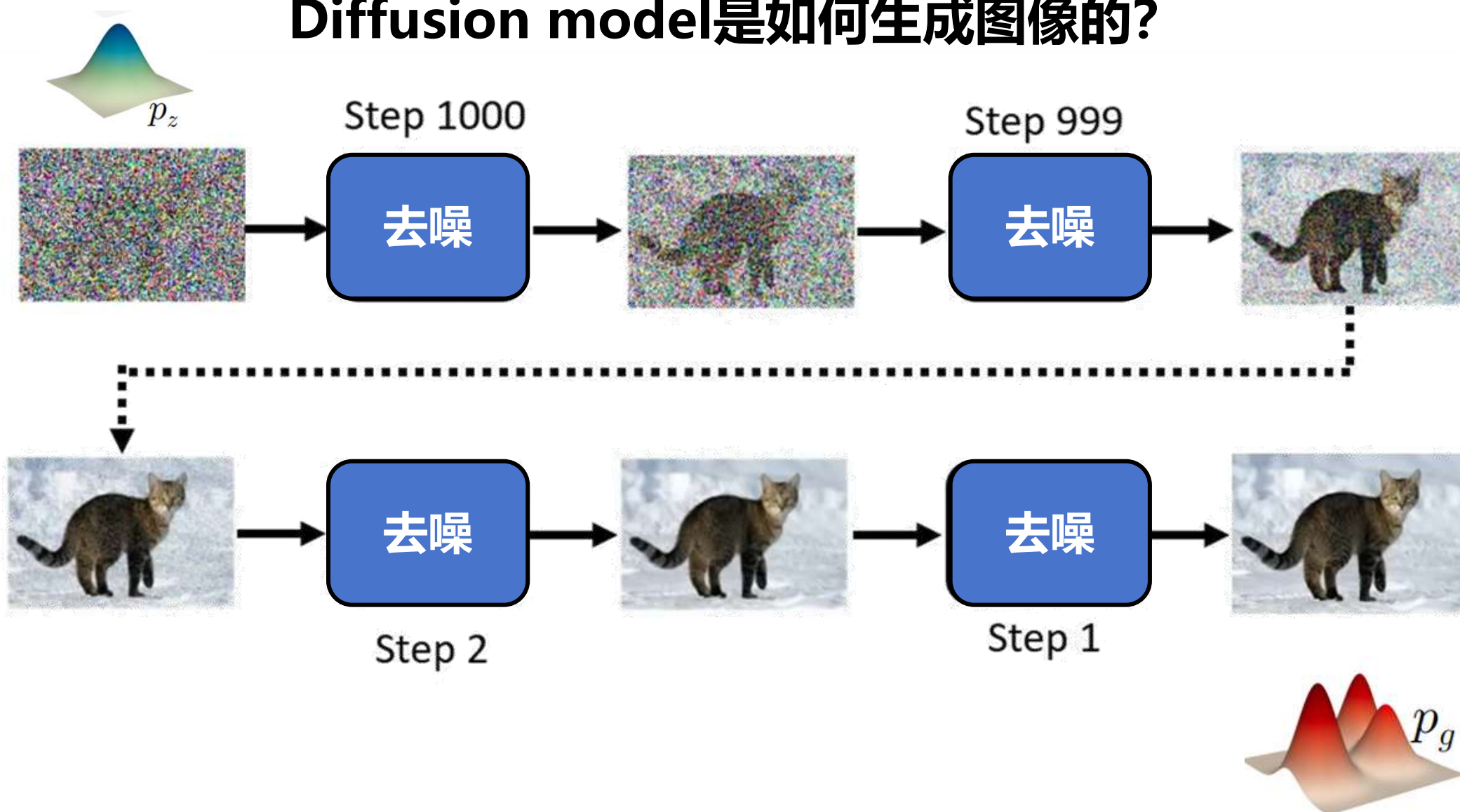


另一种思路：通过模型多次迭代预测，不断提高 z - x 复杂映射拟合精度



扩散去噪生成模型！

Diffusion model是如何生成图像的?



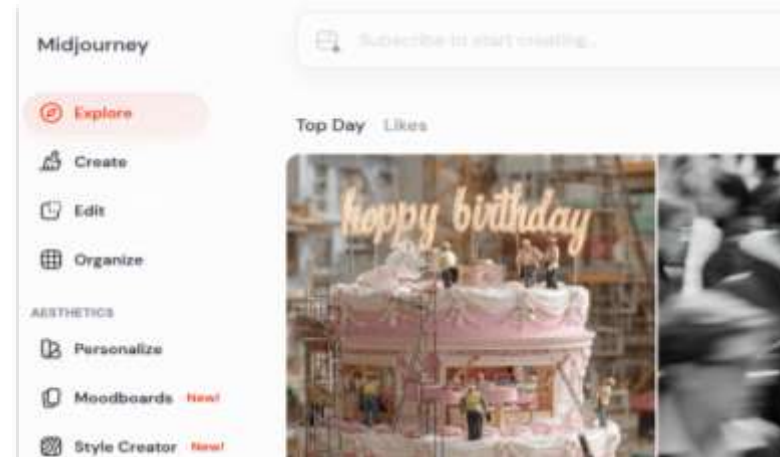
基于扩散模型的主流产品



Stable Diffusion



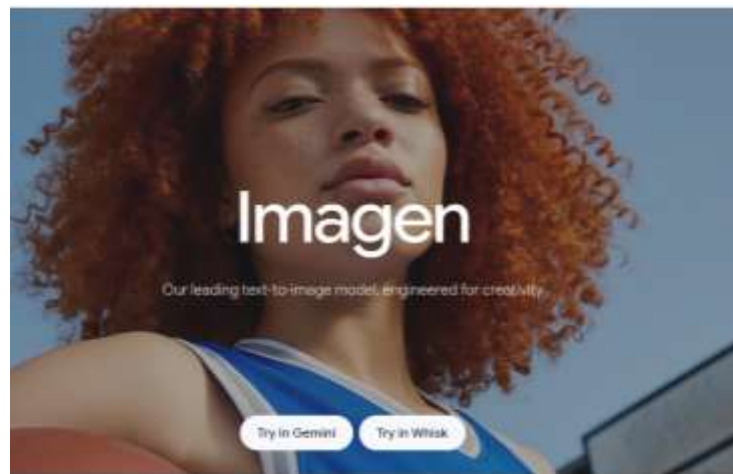
DALLE 3



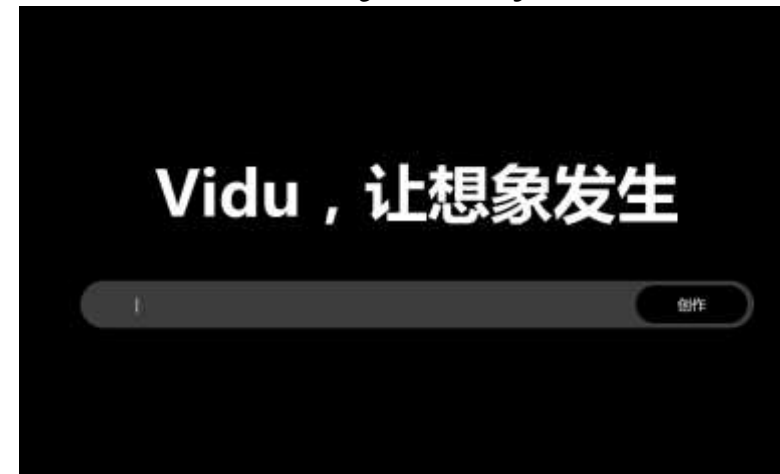
Midjourney



Sora



Imagen



Vidu

商汤秒画平台展示

<https://miaohua.sensetime.com>

Diffusion的过程类似雕塑

“雕像本来就在石头里，我只是把不要的部分去掉。”

——米开朗基罗



目录

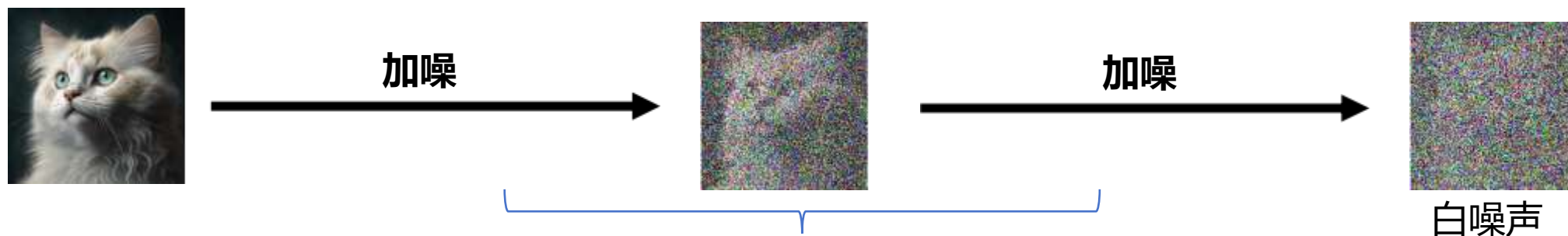
- **第一节** **扩散模型的基本原理**
- **第二节** **扩散模型的极大似然估计角度理解**
- **第三节** **扩散模型的代码实践**
- **第四节** **文生图扩散模型与潜在扩散模型**
- **第五节** **非视觉模态扩散模型**

扩散模型的基本原理

去噪扩散概率模型 (Denoising Diffusion Probabilistic Models, DDPM)

包括两个主要的阶段：**前向扩散阶段**和**反向去噪阶段**。

前向扩散阶段



$$x_{t+1} = \sqrt{1 - \beta_t} x_t + \sqrt{\beta_t} \epsilon, \epsilon \sim N(0, I) 0 \leq t \leq 999$$



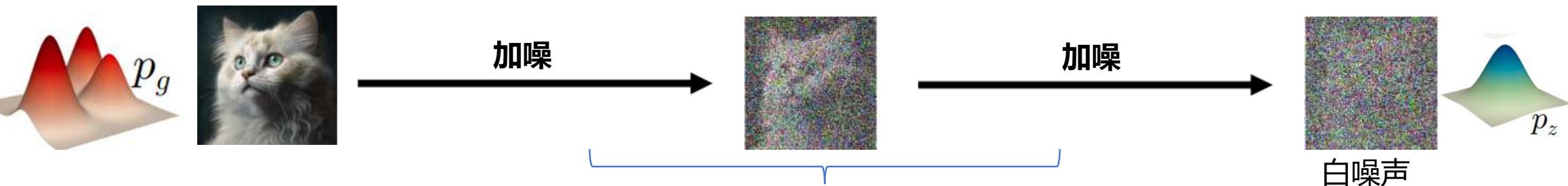
- $0 < \beta_t < 1$
- β_t 预先人为设定, 无需学习

扩散模型的基本原理

去噪扩散概率模型 (Denoising Diffusion Probabilistic Models, DDPM)

包括两个主要的阶段：**前向扩散阶段**和**反向去噪阶段**。

前向扩散阶段 (x-z的映射路径)



1000

$$x_{t+1} = \sqrt{1 - \beta_t}x_t + \sqrt{\beta_t}\epsilon, \epsilon \sim N(0, I) 0 \leq t \leq 999$$

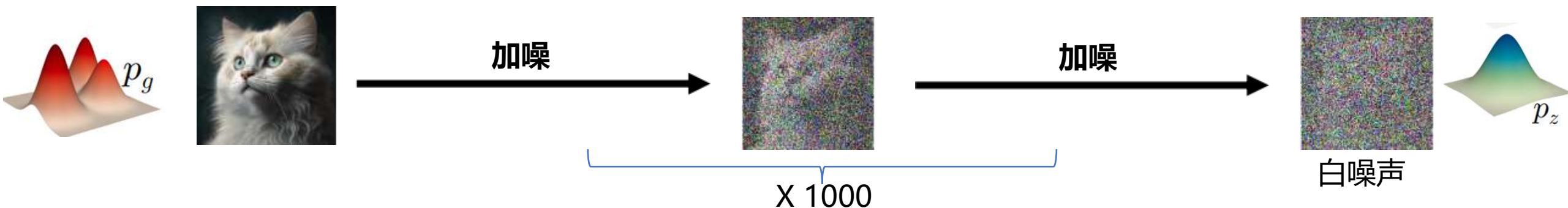
- 模型将逐步向原始数据添加噪声，直至退化为**完全噪声**
- 遵循**预先设定**的噪声增加路径。
- 前向扩散阶段构建数据x到简单隐变量z的映射，并且**无任何需要学习的参数**

扩散模型的基本原理

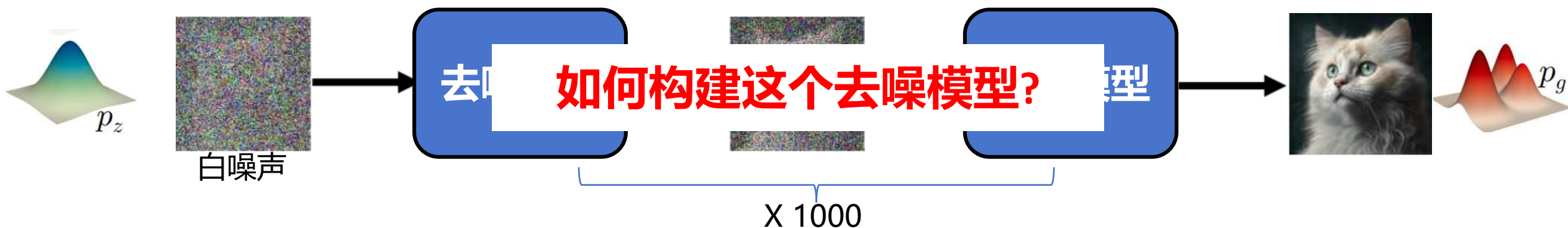
去噪扩散概率模型 (Denoising Diffusion Probabilistic Models, DDPM)

包括两个主要的阶段：**前向扩散阶段**和**反向去噪阶段**。

前向扩散阶段



反向去噪阶段：拟合前向扩散阶段的逆向过程



扩散模型的基本原理

去噪模型

目标：基于加噪图像 x_t 预测 x_{t-1} ，减少图像噪声程度

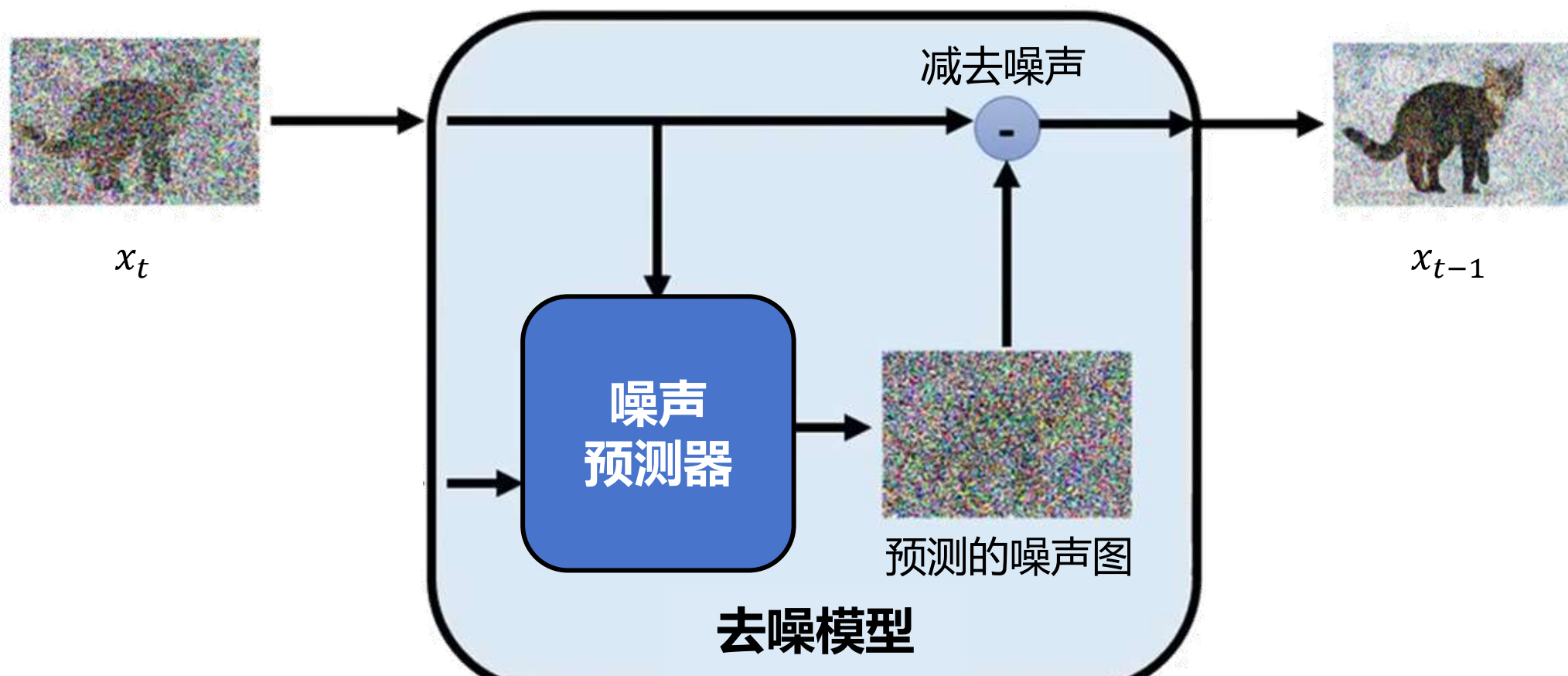


- 构建神经网络模型  输入 x_t , 输出 x_{t-1} ?

扩散模型的基本原理

去噪模型的结构

实践思路：预测输入图中的噪声比预测下一步图像简单

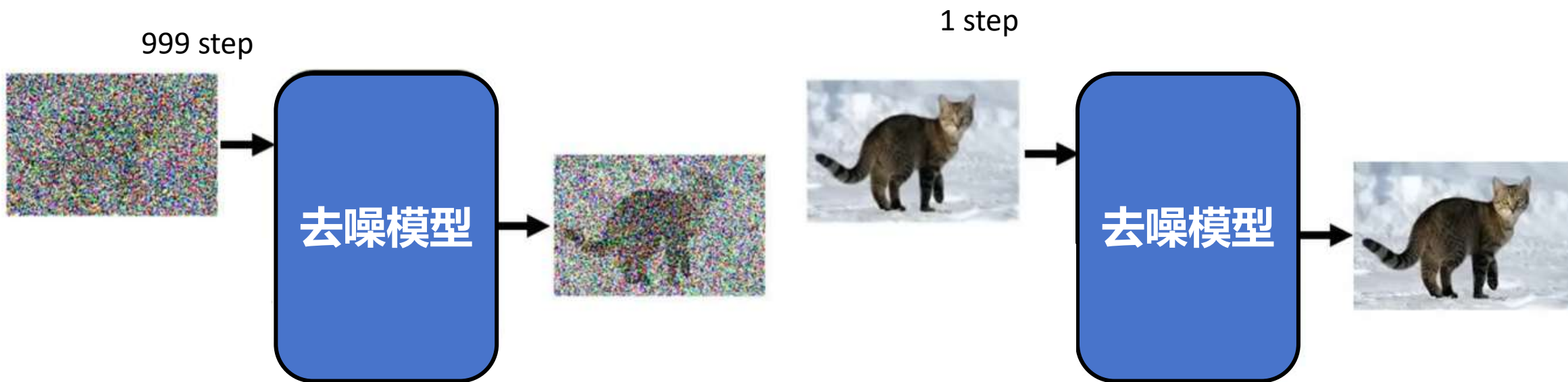


$$x_t = \sqrt{1 - \beta_{t-1}}x_{t-1} + \sqrt{\beta_{t-1}}\epsilon, \quad \epsilon \sim N(0, I)$$

扩散模型的基本原理

去噪模型结构

问题：不同Step下**噪声严重程度**不同，去噪模型面临的任务难度不一。

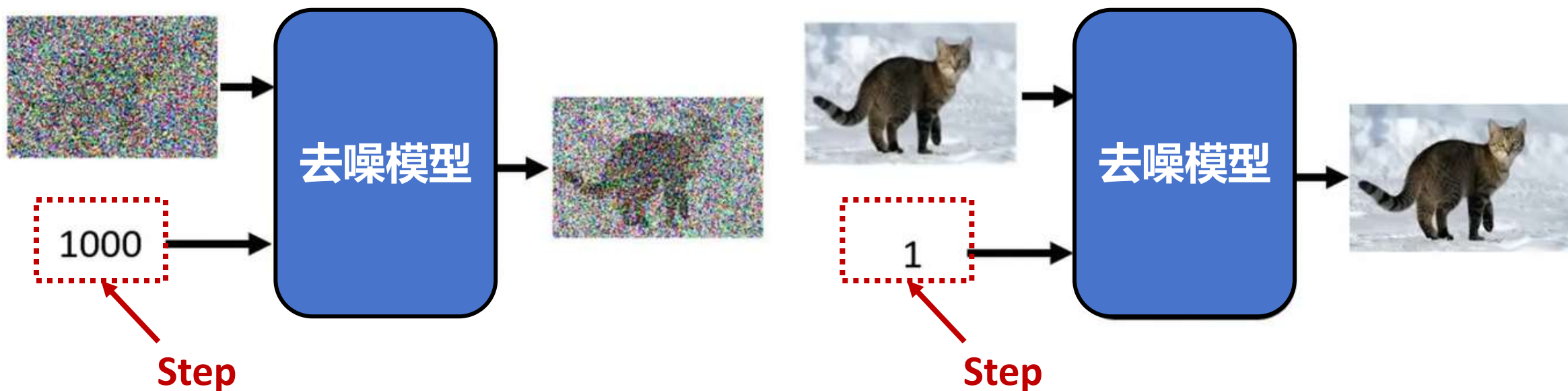


使用不同的去噪模型？
训练1000个模型，复杂度过高

扩散模型的基本原理

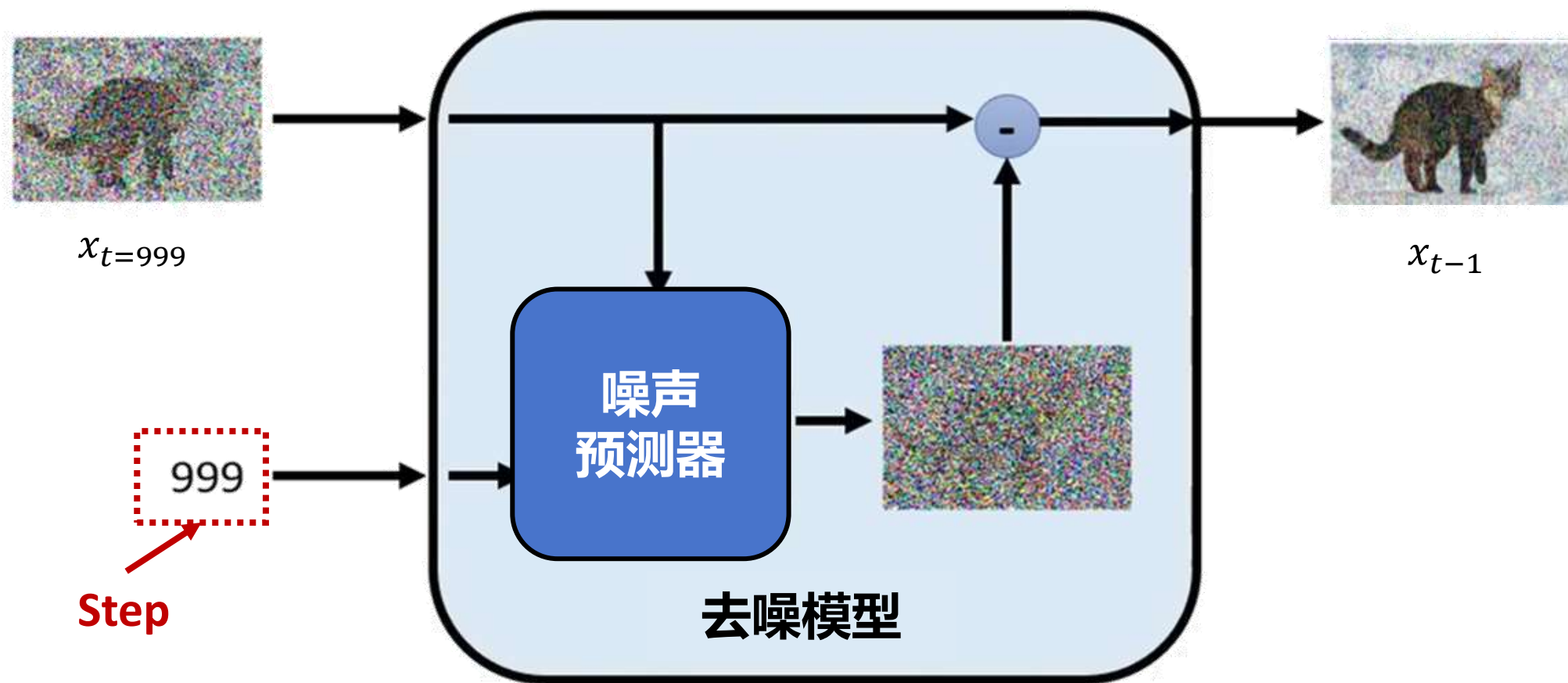
去噪模型结构

去噪部分使用同一个模型，通过额外输入区分噪声严重程度



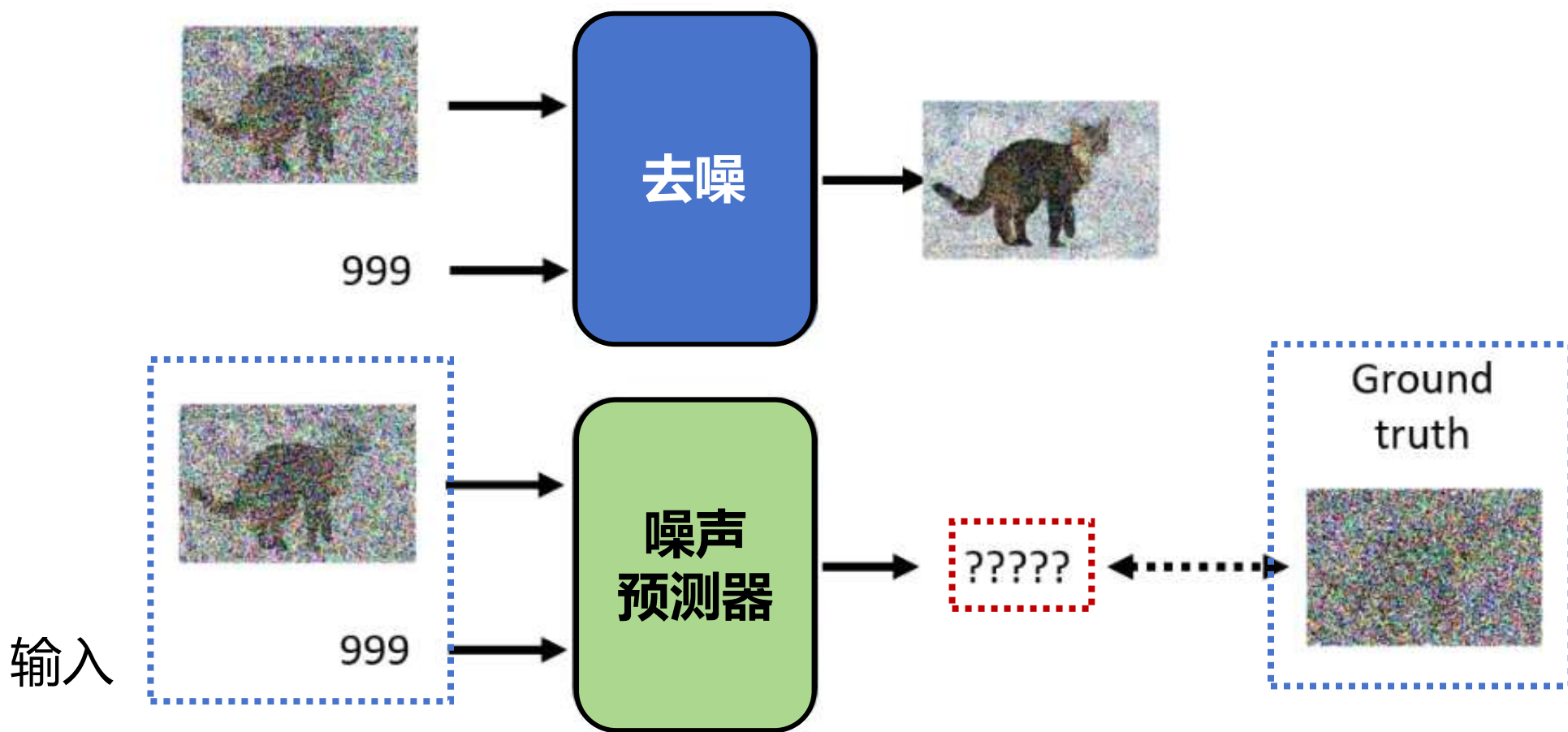
用Step(加噪步数)来刻画噪声严重程度：Step1000噪声很多，Step1噪声较少

去噪模型结构



扩散模型的基本原理

去噪模型训练

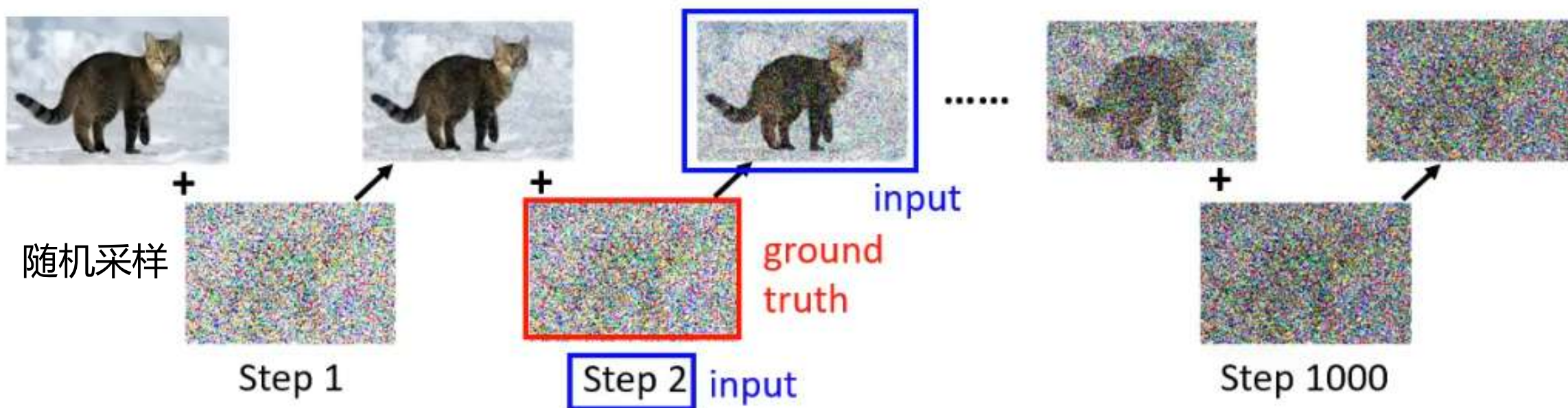


如何构造带噪声图片和Ground truth的数据对?
(加噪图像 x_t , 加噪步数 t , 噪声图 ε)

扩散模型的基本原理

去噪模型训练：构造训练数据对

通过前向扩散过程构造训练数据对：（加噪图像 x_t ，加噪步数 t ，噪声图 ε ）

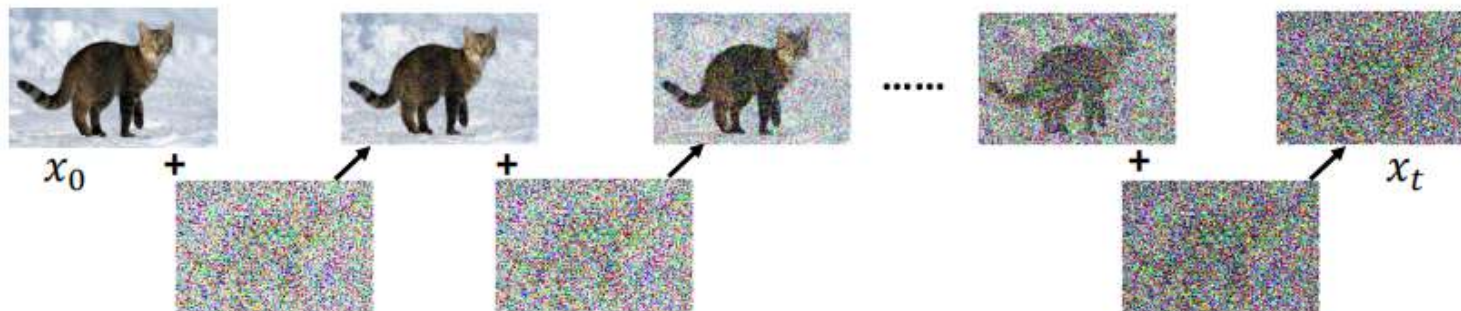


扩散模型的基本原理

去噪模型训练：构造训练数据对

通过前向扩散过程构造训练数据对：（加噪图像 x_t ，加噪步数 t ，噪声图 ε ）

$q(x_t|x_0)$



$q(x_t|x_{t-1})$

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon_t$$

$\varepsilon_t \sim \mathcal{N}(\mathbf{0}, I)$

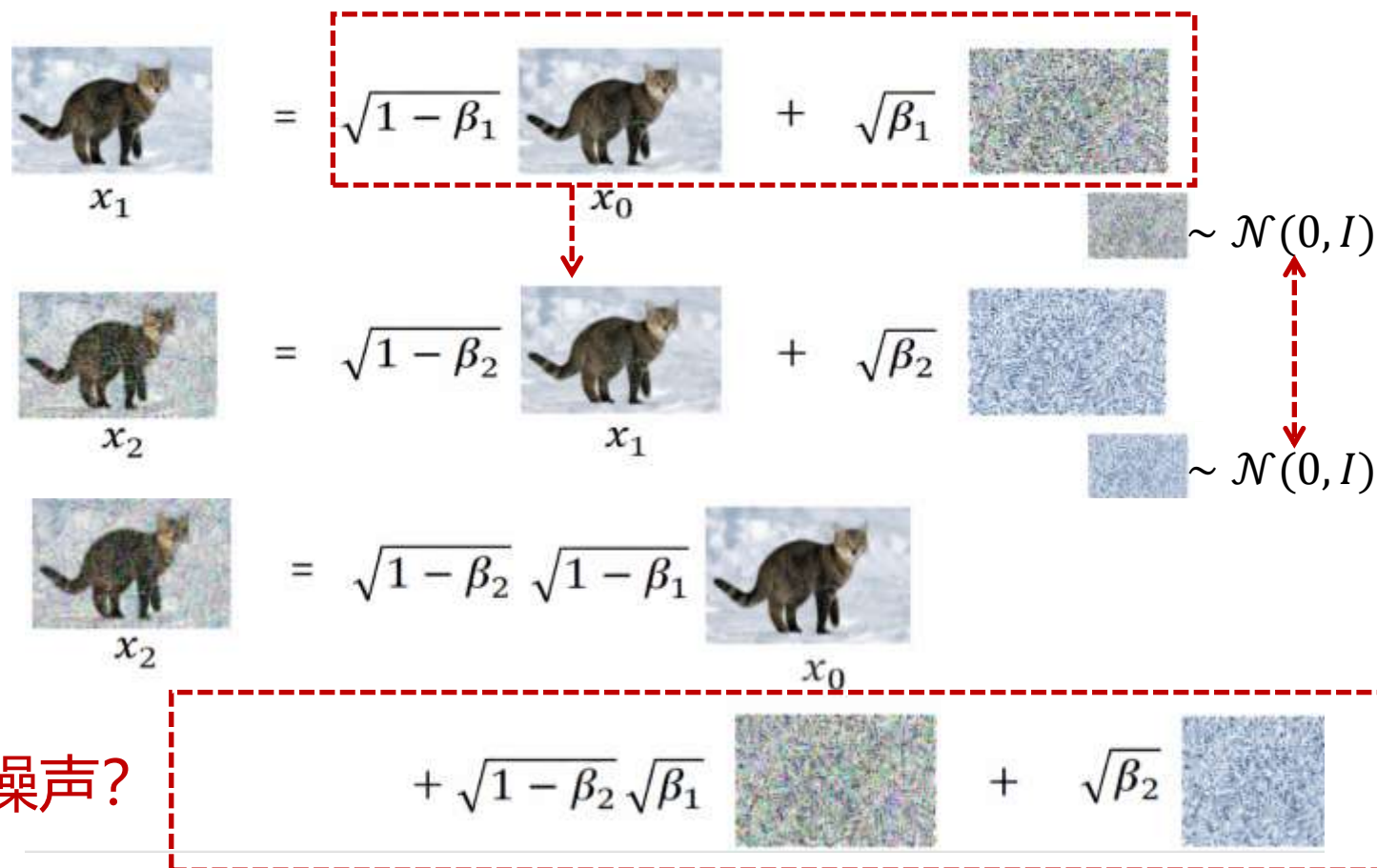
$\beta_1, \beta_2, \dots, \beta_T$

问题：构造数据 (x_t, t, ε) 需要采样 t 步，时间开销大

扩散模型的基本原理

去噪模型训练：构造训练数据对

重参数化技巧——转t步采样为一步采样



仍包含两步采样的噪声？

扩散模型的基本原理

去噪模型训练：构造训练数据对

- 重参数化技巧——转 t 步采样为一步采样
- 重要观察: 高斯随机变量的线性组合仍是高斯随机变量



x_2

$$= \sqrt{1 - \beta_2} \sqrt{1 - \beta_1}$$



x_0



$\sim \mathcal{N}(\mathbf{0}, I)$



$\sim \mathcal{N}(\mathbf{0}, I)$

1. 高斯分布乘一个系数 k

- 规则：均值变为 k 倍，方差变为 k^2 倍。

- 公式：

若 $X \sim \mathcal{N}(\mu, \sigma^2)$ ，则：

$$k \cdot X \sim \mathcal{N}(k\mu, k^2\sigma^2)$$

2. 两个高斯分布叠加（相加）

- 规则：均值直接相加，方差直接相加（前提是两个分布相互独立）。

- 公式：

若 $X \sim \mathcal{N}(\mu_1, \sigma_1^2)$ ， $Y \sim \mathcal{N}(\mu_2, \sigma_2^2)$ ，且 X 与 Y 独立，则：

$$X + Y \sim \mathcal{N}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$$

$$+ \sqrt{1 - \beta_2} \sqrt{\beta_1}$$



$$+ \sqrt{\beta_2}$$



$\sim \mathcal{N}(\mathbf{0}, I)$

$$+ \sqrt{1 - (1 - \beta_2)(1 - \beta_1)}$$



扩散模型的基本原理

去噪模型训练：构造训练数据对 重参数化技巧——转t步采样为一步采样

累加t次之后.....

$$\begin{aligned}
 & q(x_t | x_0) \\
 & \beta_1, \beta_2, \dots, \beta_T \\
 & \sim \mathcal{N}(\mathbf{0}, I) \\
 & \boxed{\alpha_t = 1 - \beta_t} \\
 & \boxed{\bar{\alpha}_t = \alpha_1 \alpha_2 \dots \alpha_t} \\
 & \text{III} \\
 & = \underbrace{\sqrt{1 - \beta_1} \dots \sqrt{1 - \beta_t}}_{\sqrt{\bar{\alpha}_t}} x_0 + \frac{\sqrt{1 - (1 - \beta_1) \dots (1 - \beta_t)}}{\sqrt{1 - \bar{\alpha}_t}} \epsilon
 \end{aligned}$$

扩散模型的基本原理

去噪模型训练：构造训练数据对

训练数据对：（加噪图像 x_t ，加噪步数 t ，噪声图 ε ）

$$\begin{aligned}
 x_t &= \underbrace{\sqrt{1 - \beta_1} \dots \sqrt{1 - \beta_t}}_{\sqrt{\bar{\alpha}_t}} \text{ (cat image)} + \frac{\sqrt{1 - (1 - \beta_1) \dots (1 - \beta_t)}}{\sqrt{1 - \bar{\alpha}_t}} \text{ (noise image)} \\
 \alpha_t &= 1 - \beta_t & \bar{\alpha}_T &= 0 \\
 \bar{\alpha}_t &= \alpha_1 \alpha_2 \dots \alpha_t & \bar{\alpha}_1 &= 1
 \end{aligned}$$

总结——通过重参数化技巧， x_t 的构造只需要1步采样



扩散模型的基本原理

去噪模型训练：构造训练数据对

训练数据对：（加噪图像 x_t ，加噪步数 t ，噪声图 ε ）

构建训练数据(x_t, t, ϵ):

- $t \in [0, T]$
- $\varepsilon \sim N(0, I)$
- $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{(1 - \bar{\alpha}_t)}\varepsilon$

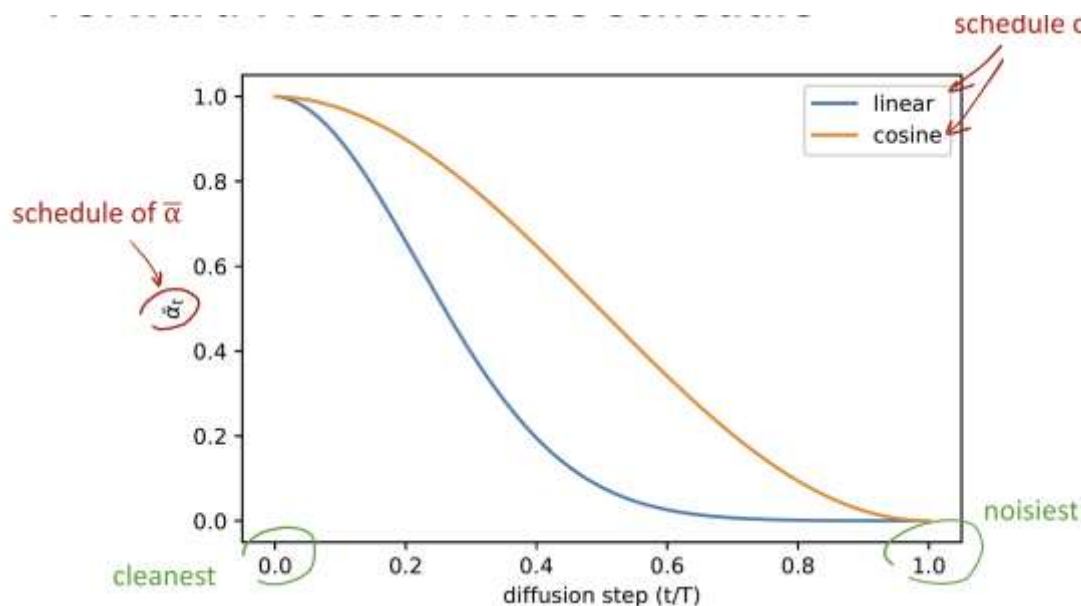
总结——通过重参数化技巧， x_t 的构造只需要1步采样

扩散模型的基本原理

去噪模型训练：构造训练数据对

训练数据对：（加噪图像 x_t ，加噪步数 t ，噪声图 ε ）

$$\text{Noisy Image} = \underbrace{\sqrt{1 - \beta_1} \dots \sqrt{1 - \beta_t}}_{\sqrt{\bar{\alpha}_t}} \times \text{Original Image} + \frac{\sqrt{1 - (1 - \beta_1) \dots (1 - \beta_t)}}{\sqrt{1 - \bar{\alpha}_t}} \times \text{Noise}$$



➤ α_t （不同time step的噪声程度）曲线

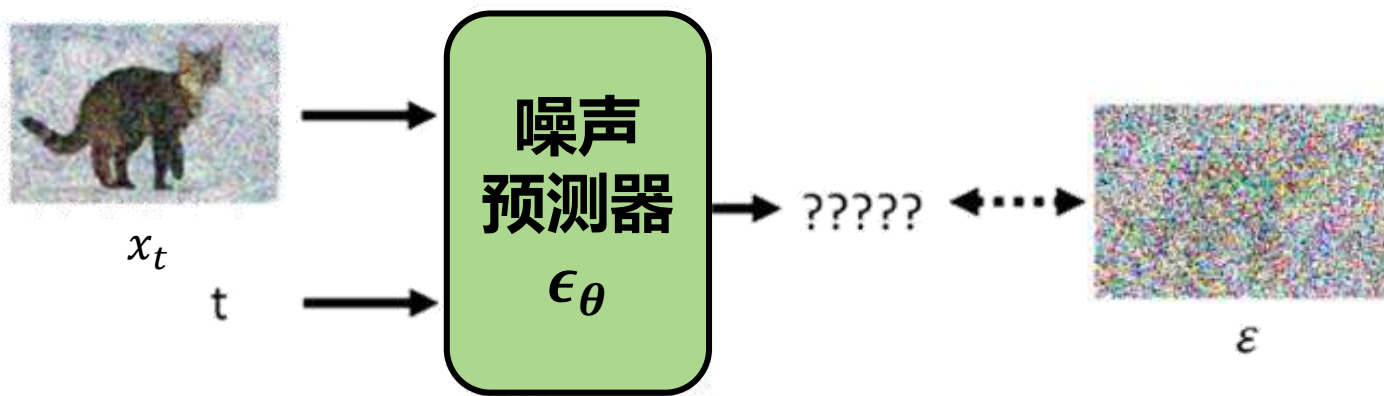
- $\alpha_0=1$ (原始图像)
- $\alpha_T=0$ (白噪声)
- 单调
- 非线性

扩散模型的基本原理

去噪模型训练

➤ 使用MSE loss优化噪声预测器

$$L = ||\epsilon - \epsilon_{\theta}(x_t, t)||$$



构建训练数据($x_t, t, epsilon$):

- 采样 $t \in [0, T]$
- $\epsilon \sim N(0, I)$
- $X_t = \sqrt{\alpha}x_0 + \sqrt{(1 - \alpha)}\epsilon$

扩散模型的基本原理

去噪模型训练

Algorithm 1 Training

- 1: **repeat**
- 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ ← 采样一张干净的图
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$ 采样一个数字 T 为 step 的最大值
- 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ← 采样一个噪声图像
- 5: Take gradient descent step on

$$\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$$

- 6: **until** converged

↑
ground truth

加噪图 x_t

噪声预测器



x_0 : clean image



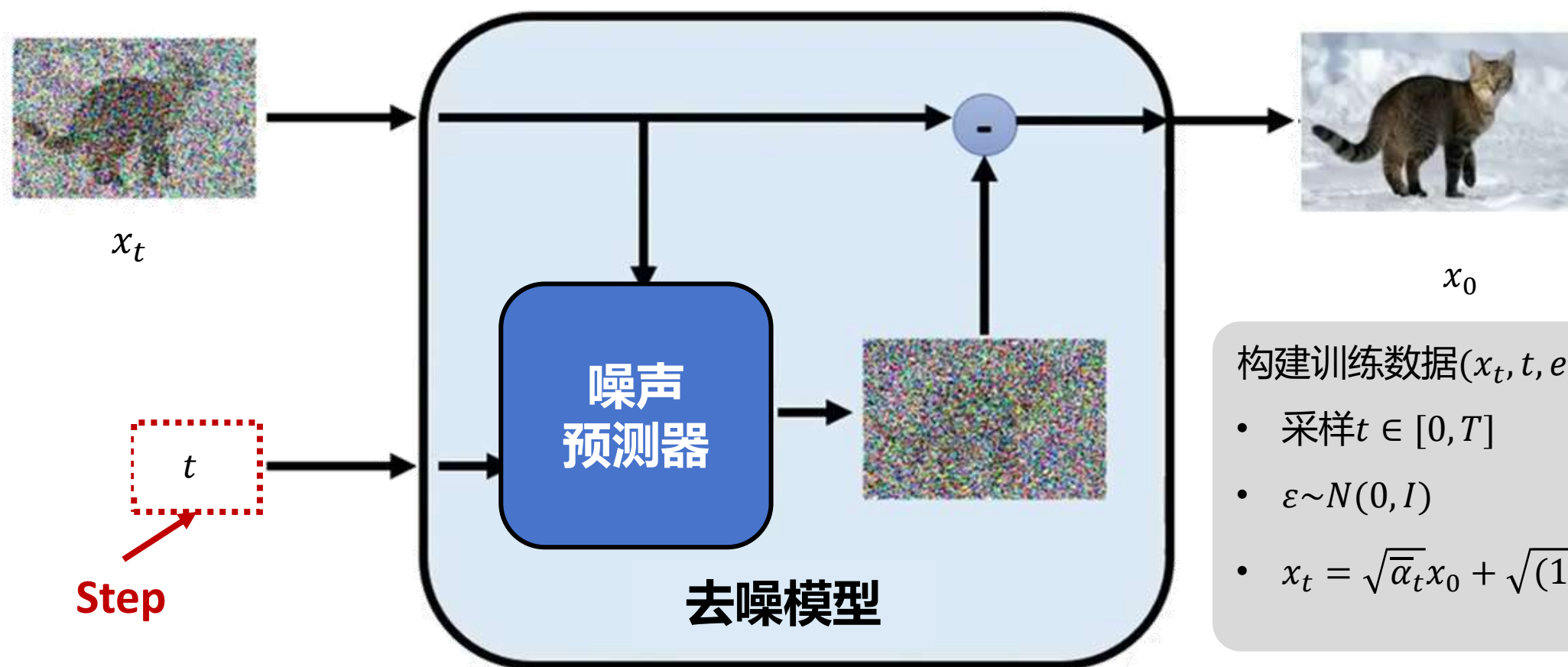
ϵ : noise

$\bar{\alpha}_1, \bar{\alpha}_2, \dots, \bar{\alpha}_T$ 事先定好的参数
→
由大到小

扩散模型的基本原理

去噪模型结构

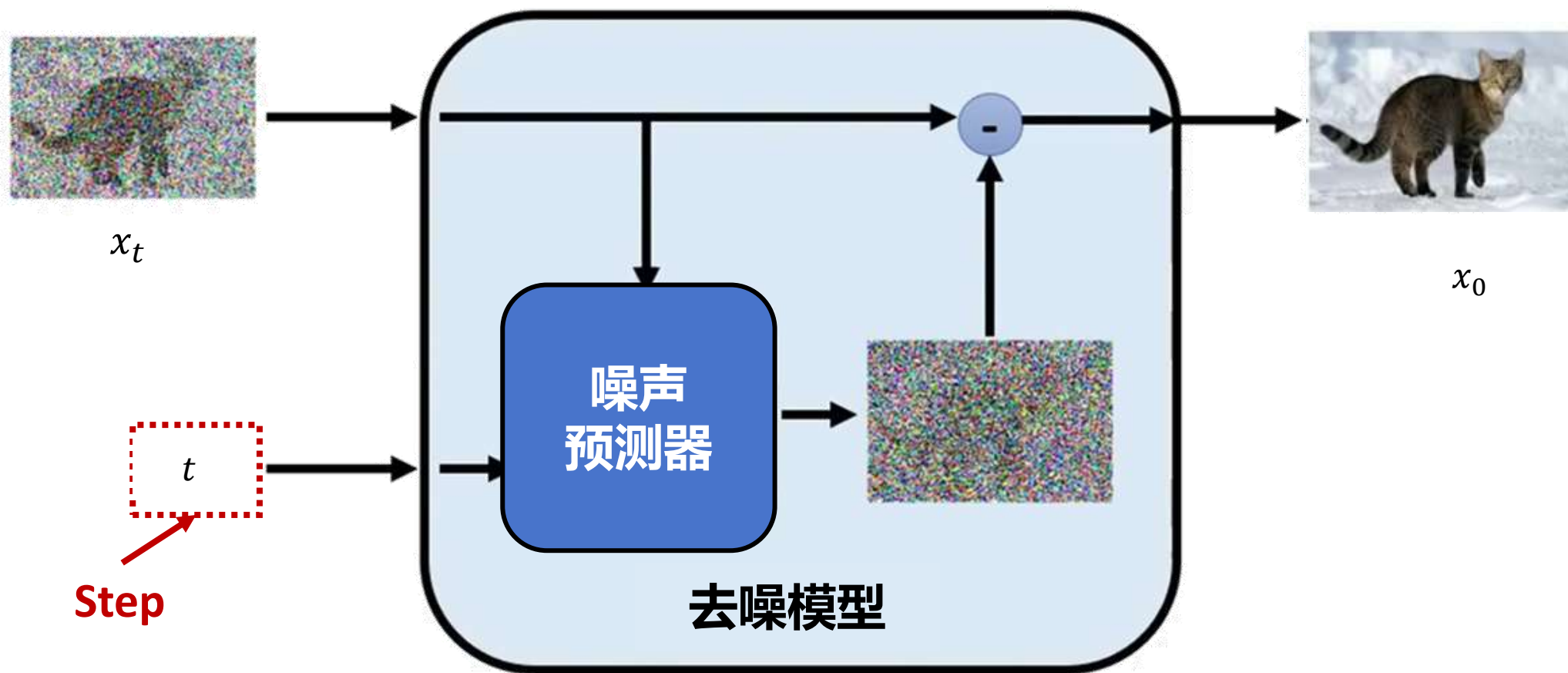
实践思路：预测输入图中的噪声



➤ 去噪模型：噪声预测器 ϵ_θ ，输入 x_t ，输出 x_0 到 x_t 的累计噪声

去噪模型结构

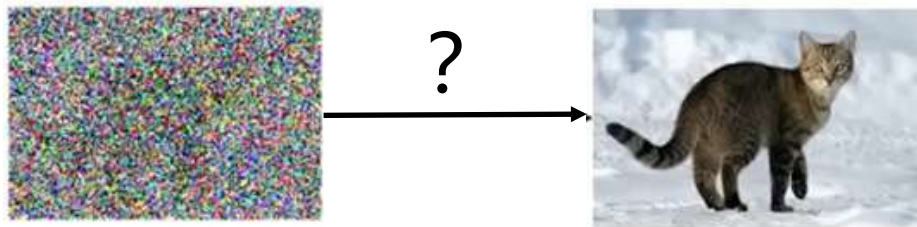
实践思路：预测输入图中的噪声



扩散模型的基本原理

扩散模型推理过程

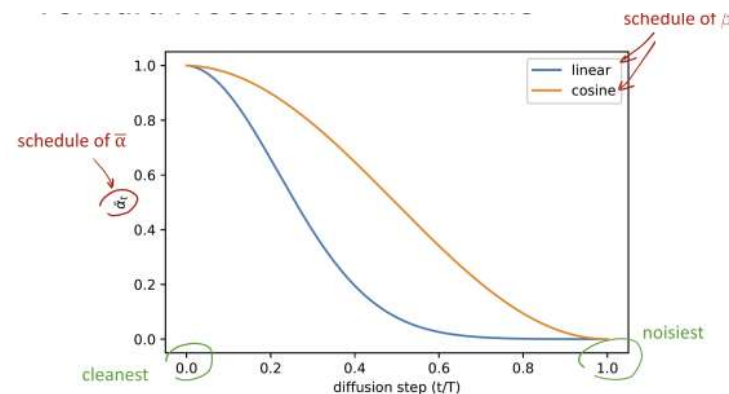
- **目标生成/推理过程：** 基于随机噪声生成原始图像



- **去噪模型：** 噪声预测器 ϵ_θ ，输入 x_t ，输出 x_0 到 x_t 的累计噪声
- **理想方案：** 直接预测 x_0 到 x_T 的累加噪声

$$x_0(x_T, T) = \frac{x_T - \sqrt{1 - \bar{\alpha}_T} \epsilon_\theta(x_T, T)}{\sqrt{\bar{\alpha}_T}},$$

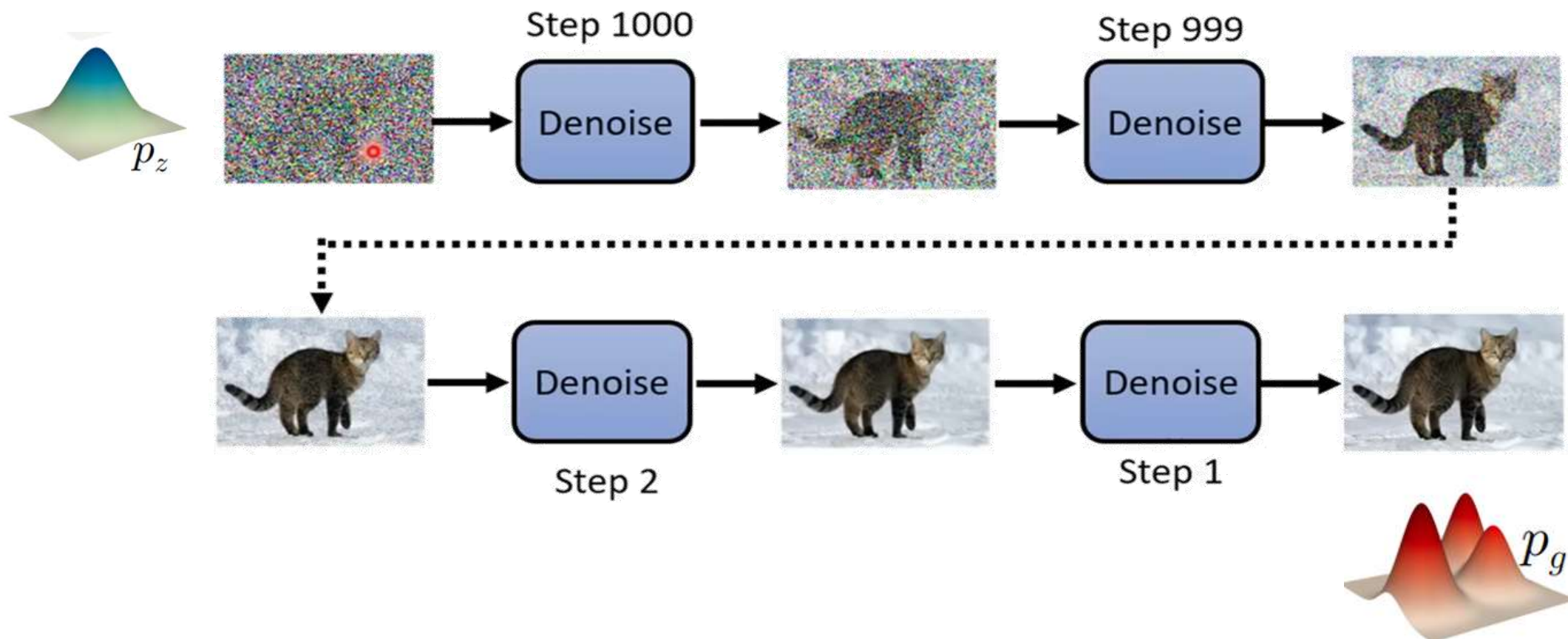
$\alpha_T = 0$



扩散模型的基本原理

扩散模型推理过程

- 实际解决方案：用噪声预测器 ϵ_θ 推测 x_t 到 x_{t-1} 的映射



扩散模型的基本原理

扩散模型推理过程($x_t \rightarrow x_{t-1}$)

观察1: 前向条件分布 $q(x_t|x_{t-1})$ 已知, 但是反向条件分布 $q(x_{t-1}|x_t)$ 形式未知

$$q(x_t|x_{t-1})$$


 x_t

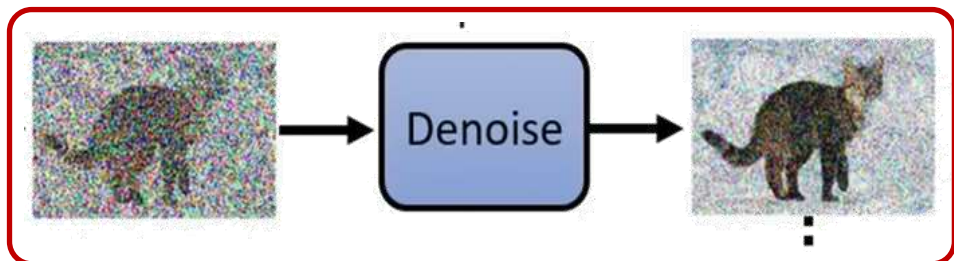
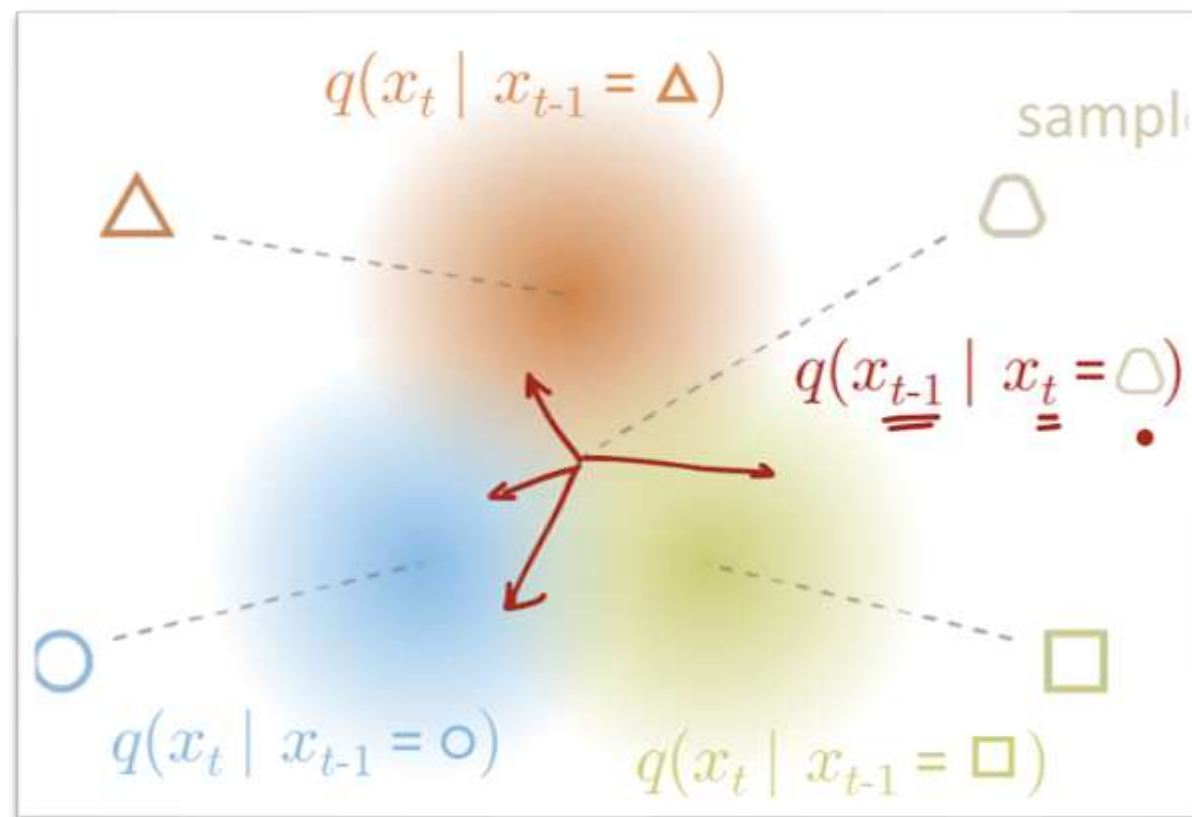
$$= \sqrt{1-\beta_t}$$


 x_{t-1}

$$+ \sqrt{\beta_t}$$



$q(x_{t-1}|x_t)$?

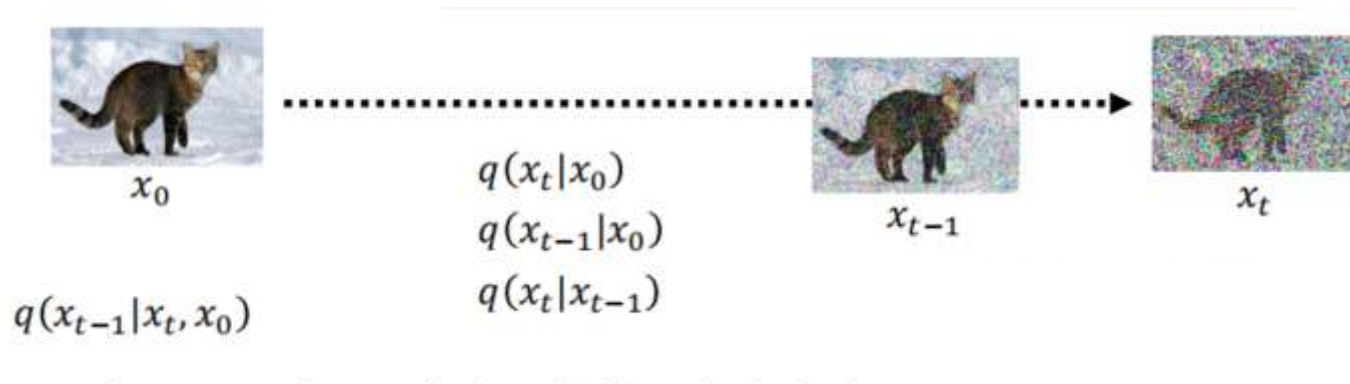

 x_t
 x_{t-1}


扩散模型的基本原理

扩散模型推理过程($x_t \rightarrow x_{t-1}$)

观察1: 前向条件分布 $q(x_t|x_{t-1})$ 已知, 但是反向条件分布 $q(x_{t-1}|x_t)$ 形式未知

观察2: 联合反向条件分布 $q(x_{t-1}|x_t, x_0)$ 是也是已知的



$q(x_{t-1}|x_t, x_0)$ 分布的表示

➤ $q(x_{t-1}|x_t, x_0) \sim \mathcal{N}(\mu_t, \sigma_t)$

➤
$$\mu_t = \frac{\sqrt{\bar{\alpha}_t}\beta_t x_0 + \bar{\alpha}_t(1-\bar{\alpha}_{t-1})x_t}{1-\bar{\alpha}_t}$$

$$= \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon \right)$$

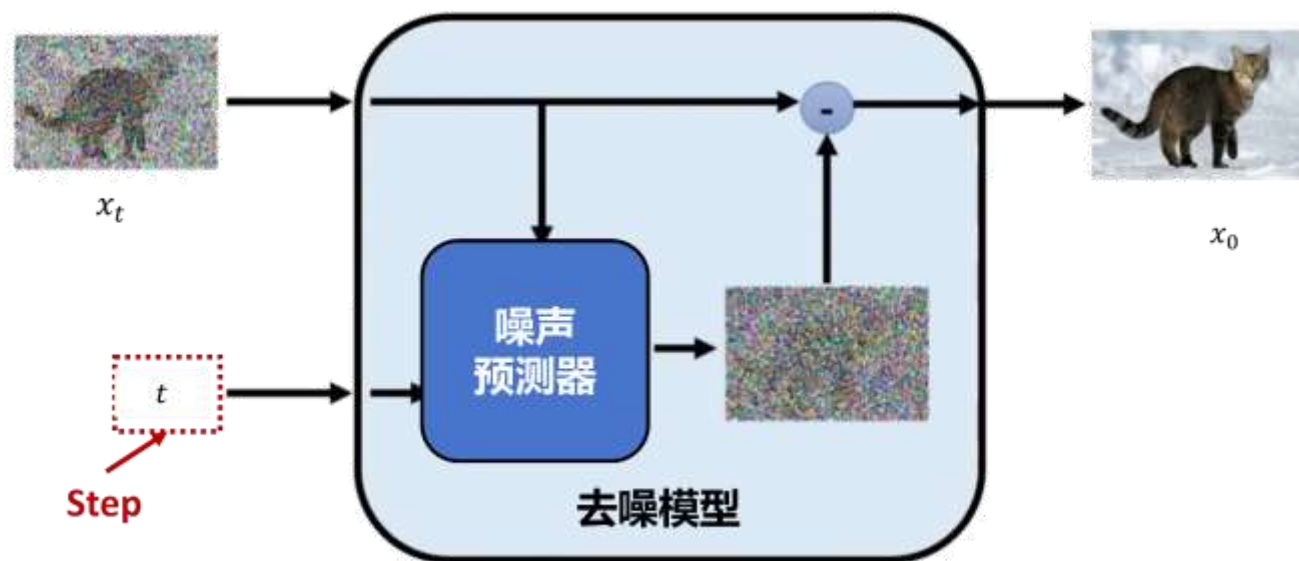
➤
$$\sigma_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \beta_t I$$

扩散模型的基本原理

扩散模型推理过程($x_t \rightarrow x_{t-1}$)

观察1: 前向条件分布 $q(x_t|x_{t-1})$ 已知, 但是反向条件分布 $q(x_{t-1}|x_t)$ 形式未知

观察2: 联合反向条件分布 $q(x_{t-1}|x_t, x_0)$ 是也是已知的



解决思路: 基于 $\epsilon_\theta(x_t, t)$ 先近似推断 x_0 , 然后代入 $q(x_{t-1}|x_t, x_0)$ 采样 x_{t-1}



扩散模型的基本原理

扩散模型推理过程($x_t \rightarrow x_{t-1}$)

解决思路：基于 $\epsilon_\theta(x_t, t)$ 先近似推断 x_0 ，然后代入 $q(x_{t-1}|x_t, x_0)$ 采样 x_{t-1}

$$q(x_{t-1}|x_t, x_0) \sim \mathcal{N}\left(\frac{1}{\sqrt{\bar{\alpha}_t}}\left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon\right), \sigma_t^2 I\right)$$

$$P_\theta(x_{t-1}|x_t) = q(x_{t-1}|x_t, x_0(x_t, t))$$

$$x_0(x_t, t) = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

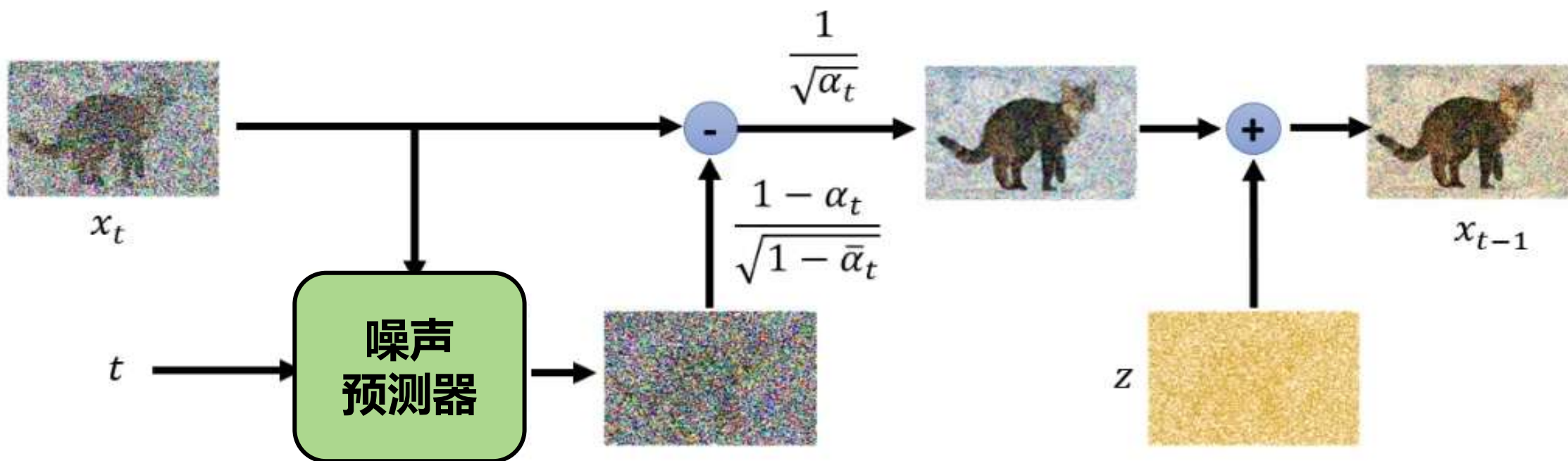
$$p_\theta(x_{t-1}|x_t) = \mathcal{N}\left(\frac{1}{\sqrt{\bar{\alpha}_t}}\left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right), \sigma_t^2 I\right)$$

分母没有0 & 实际并不需要计算 x_0

扩散模型的基本原理

扩散模型推理过程($x_t \rightarrow x_{t-1}$)

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, 1)$$



扩散模型的基本原理

扩散模型推理过程($x_t \rightarrow x_{t-1}$)

Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ← 完全是噪声的图
- 2: **for** $t = T, \dots, 1$ **do**
- 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$ 采样一个噪声图像
- 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
- 5: **end for**
- 6: **return** $\mathbf{x}_0$



$\mathbf{x}_T$

$\bar{\alpha}_1, \bar{\alpha}_2, \dots, \bar{\alpha}_T$

$\alpha_1, \alpha_2, \dots, \alpha_T$

事先定好的权重

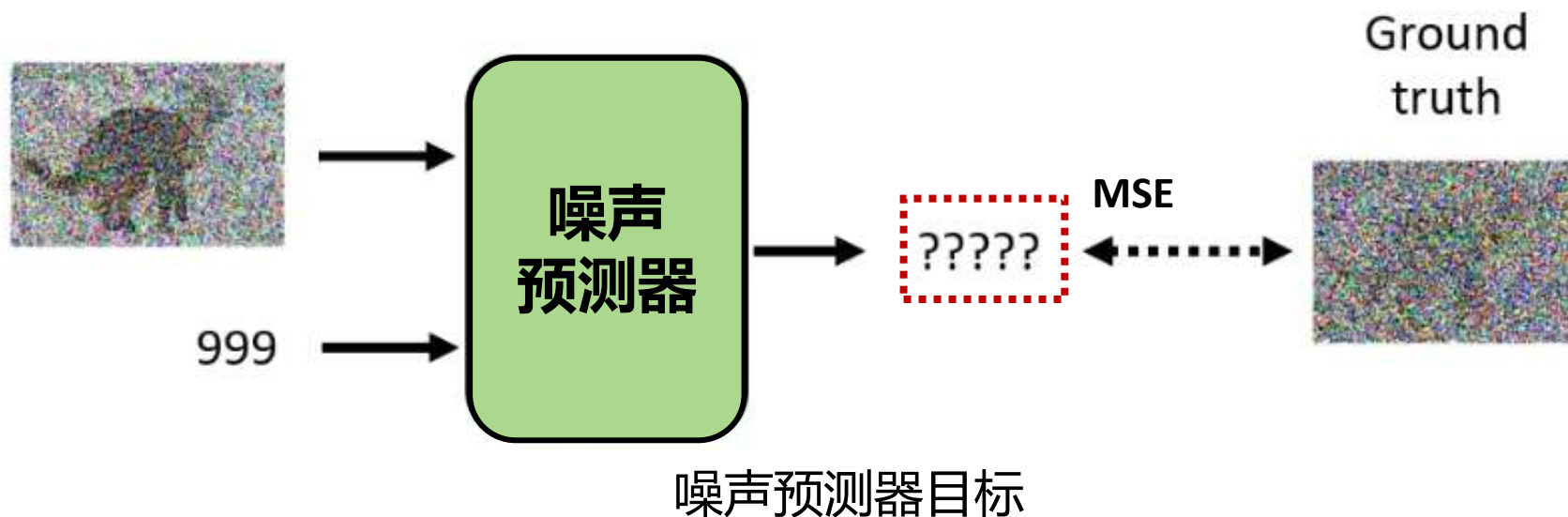
目录

- 第一节 扩散模型的基本原理
- 第二节 扩散模型的极大似然估计角度理解
- 第三节 扩散模型的代码实践
- 第四节 文生图扩散模型与潜在扩散模型
- 第五节 是视觉模态扩散模型

扩散模型的极大似然估计角度理解

从极大似然估计角度理解理解扩散模型

- 生成式模型优化目标：最大化可观测样本似然
- 噪声预测器优化目标：推断输入加噪图像中的包含的噪声图像

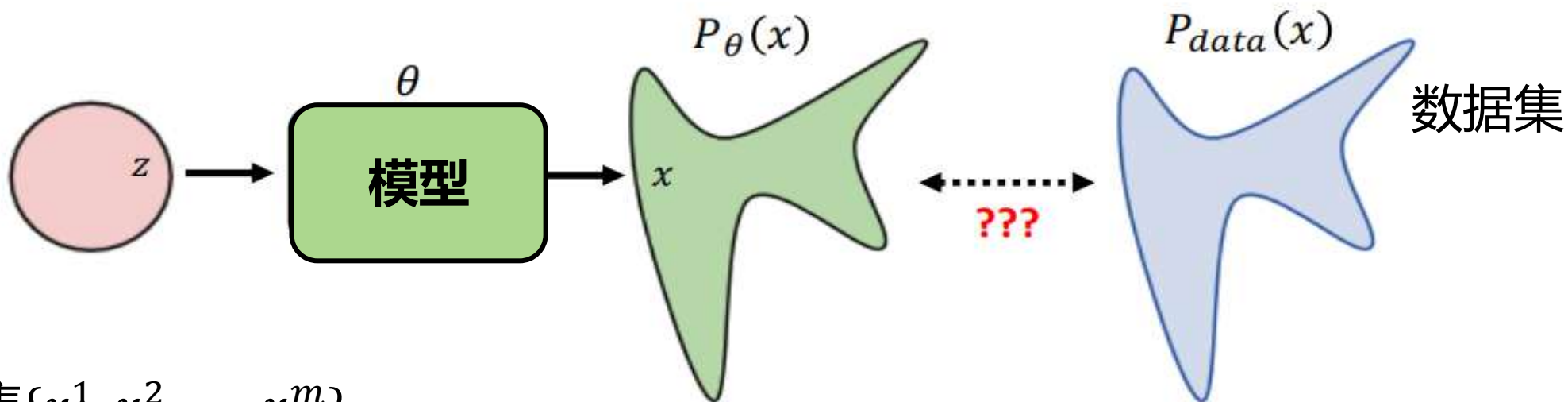


噪声预测器优化目标与似然估计到底是什么关系？

扩散模型的极大似然估计角度理解

回顾：最大似然估计

核心思想：使模型生成分布和真实分布类似



给定数据集 $\{x^1, x^2, \dots, x^m\}$

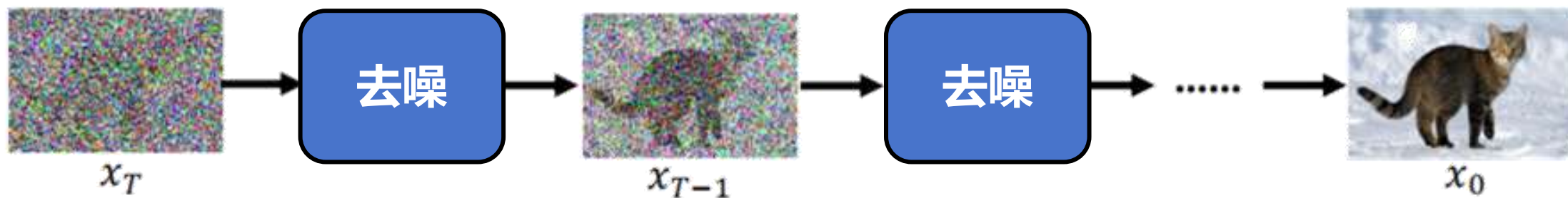
似然函数 p_θ ：概率模型生成给定样本 x 的概率

最大似然估计：寻找模型参数，最大化**样本集**的似然

$$\theta^* = \operatorname{argmax}_\theta \prod_{i=1}^m p_\theta(x^i)$$

扩散模型的极大似然估计角度理解

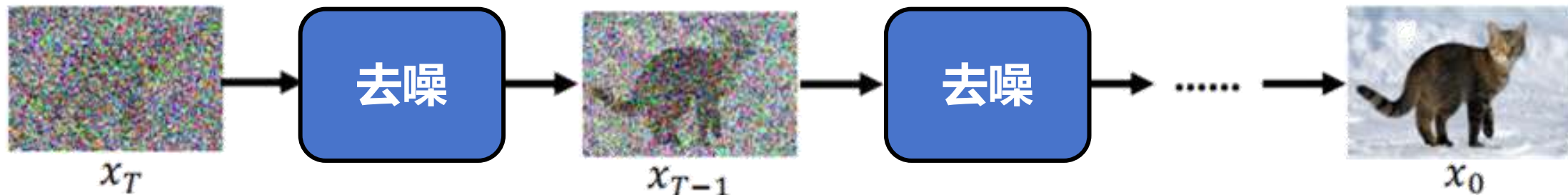
似然估计推导



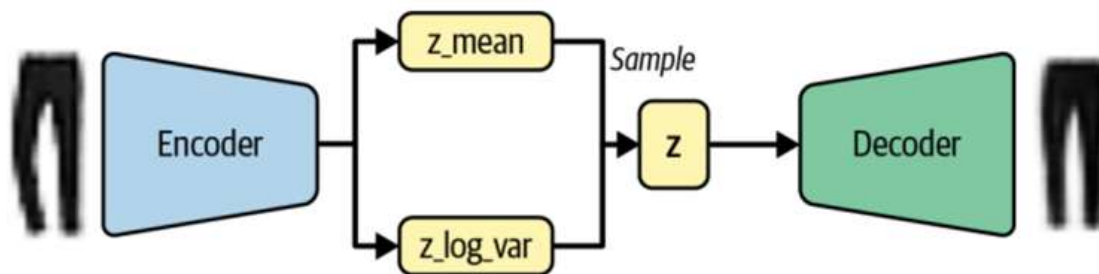
$$p_{\theta}(x_0) = \int p(x_0, x_{1:T}) dx_{1:T} = \int p(x_T) p_{\theta}(x_{T-1}|x_T) \dots p_{\theta}(x_0|x_1) dx_{1:T}$$

扩散模型的极大似然估计角度理解

似然估计对比： DDPM vs VAE



$$p_{\theta}(x_0) = \int p(x_0, x_{1:T}) dx_{1:T} = \int p(x_T) p_{\theta}(x_{T-1}|x_T) \dots p_{\theta}(x_0|x_1) dx_{1:T}$$

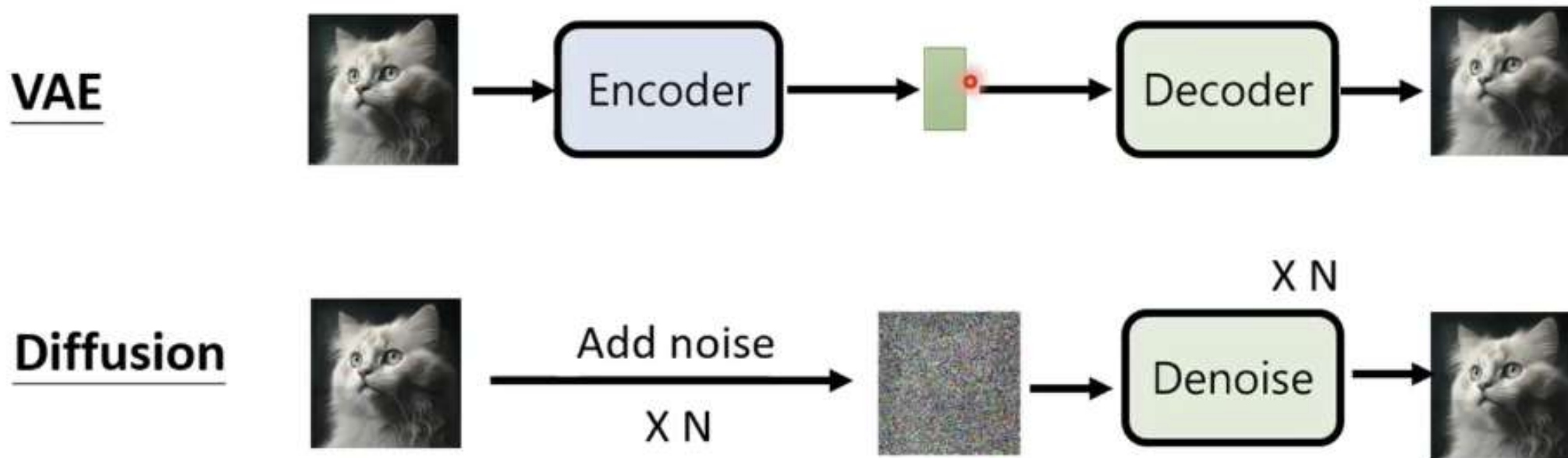


$$p_{\theta}(x_0) = \int p(x_0, z) dz = \int p(z) p_{\theta}(x_0|z) dz$$

DDPM 可以看作由T个隐变量构成的多隐层VAE，即 $\mathbf{z} = \mathbf{x}_{1:T}$

扩散模型的极大似然估计角度理解

似然估计对比： DDPM vs VAE



DDPM 可以看作由T个隐变量构成的多隐层VAE

与VAE类似，DDPM中最大化似然估计等价于最大化置信下界

VAE 最大化 $\log p_{\theta}(x)$ $\longrightarrow$ 最大化 $E_{q(z|x)}[\log(\frac{p(x,z)}{q(z|x)})]$
编码器

DDPM 最大化 $\log p_{\theta}(x)$ $\longrightarrow$ 最大化 $E_{q(x_1:x_T|x_0)}[\log(\frac{p(x_0:x_T)}{q(x_1:x_T|x_0)})]$
前向过程
(扩散过程)



扩散模型的极大似然估计角度理解

回归：VAE中置信下界推导过程

最大似然 $L = \sum_x \log p(x)$

引入 $q(z|x)$, $\int q(z|x) dz = 1$

$$\log p(x) = \int q(z|x) \log p(x) dz$$

↓ 贝叶斯定理

$$\log p(x) = \int q(z|x) \log \frac{p(z, x)}{p(z|x)} dz$$

↓ 分解

$$\log p(x) = \int q(z|x) \log \left(\frac{p(z, x)}{q(z|x)} \cdot \frac{q(z|x)}{p(z|x)} \right) dz$$

拆分

$$\log p(x) = \int q(z|x) \log \left(\frac{p(z, x)}{q(z|x)} \right) dz + \underbrace{\int q(z|x) \log \left(\frac{q(z|x)}{p(z|x)} \right) dz}_{\text{KL散度} \geq 0}$$

KL散度 ≥ 0

$$\text{所以 } \log p(x) \geq \underbrace{\int q(z|x) \log \left(\frac{p(z, x)}{q(z|x)} \right) dz}_{L_b, \text{ 下界}}$$

L_b , 下界

DDPM的置信下界推导，只需进行符号替换 $\mathbf{z} = \mathbf{x}_{1:T}$

扩散模型的极大似然估计角度理解

最大化置信下界

$$\log p(x) \geq E_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_{0:T})}{q(x_{1:T}|x_0)} \right]$$

$$\text{最大化 } E_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_{0:T})}{q(x_{1:T}|x_0)} \right]$$

条件概率链式法则

$$= E_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t)}{\prod_{t=1}^T q(x_t|x_{t-1})} \right]$$

拆出连乘第一项

$$= E_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_T) p_\theta(x_0|x_1) \prod_{t=2}^T p_\theta(x_{t-1}|x_t)}{q(x_1|x_0) \prod_{t=2}^T q(x_t|x_{t-1})} \right]$$

引入额外条件
不影响分布

$$= E_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_T) p_\theta(x_0|x_1) \prod_{t=2}^T p_\theta(x_{t-1}|x_t)}{q(x_1|x_0) \prod_{t=2}^T q(x_t|x_{t-1}, x_0)} \right]$$

拆分对数运算

$$= E_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_T) p_\theta(x_0|x_1)}{q(x_1|x_0)} + \log \prod_{t=2}^T \frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1}, x_0)} \right]$$

贝叶斯公式
置换 x_{t-1} 和 x_t

$$= E_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_T) p_\theta(x_0|x_1)}{q(x_1|x_0)} + \log \prod_{t=2}^T \frac{p_\theta(x_{t-1}|x_t)}{\frac{q(x_{t-1}|x_t, x_0) q(x_t|x_0)}{q(x_{t-1}|x_0)}} \right]$$

扩散模型的极大似然估计角度理解

最大化置信下界

$$= E_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_T)p_\theta(x_0|x_1)}{q(x_1|x_0)} + \log \prod_{t=2}^T \frac{p_\theta(x_{t-1}|x_t)}{\frac{q(x_{t-1}|x_t, x_0)q(x_t|x_0)}{q(x_{t-1}|x_0)}} \right]$$

$$= E_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_T)p_\theta(x_0|x_1)}{\cancel{q(x_1|x_0)}} + \log \frac{\cancel{q(x_1|x_0)}}{q(x_T|x_0)} + \log \prod_{t=2}^T \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} \right]$$

$$= E_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_T)p_\theta(x_0|x_1)}{q(x_T|x_0)} + \sum_{t=2}^T \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} \right]$$

$$= E_{q(x_{1:T}|x_0)} [\log p_\theta(x_0|x_1)] + E_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_T)}{q(x_T|x_0)} \right] + \sum_{t=2}^T \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} \right]$$

$$= E_{q(x_1|x_0)} [\log p_\theta(x_0|x_1)] + E_{q(x_T|x_0)} \left[\log \frac{p(x_T)}{q(x_T|x_0)} \right] + \sum_{t=2}^T \mathbb{E}_{q(x_t, x_{t-1}|x_0)} \left[\log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} \right]$$

$$= \underbrace{E_{q(x_1|x_0)} [\log p_\theta(x_0|x_1)]}_{\text{重构项}} - \underbrace{KL(q(x_T|x_0) || p_\theta(x_T))}_{\text{先验匹配项}} - \sum_{t=2}^T \underbrace{E_{q(x_t|x_0)} [D_{KL}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t))]}_{\text{去噪匹配项}}$$

重构项

先验匹配项

去噪匹配项



扩散模型的极大似然估计角度理解

最大化置信下界

$$\begin{aligned} \operatorname{argmax}_{\theta} & \underbrace{E_{q(x_1|x_0)}[\log p_{\theta}(x_0|x_1)]}_{\text{重构项}} - \underbrace{KL(q(x_T|x_0)||p_{\theta}(x_T))}_{\text{先验匹配项}} - \sum_{t=2}^T \underbrace{E_{q(x_t|x_0)}[D_{KL}(q(x_{t-1}|x_t, x_0)||p_{\theta}(x_{t-1}|x_t))]}_{\text{去噪匹配项}} \\ &= -E_{q(x_1|x_0)}[\mathcal{L}_0] - \mathcal{L}_T(\text{常数}) - \sum_{t=2}^T E_{q(x_t|x_0)}[\mathcal{L}_{t-1}] \\ &= -\sum_{t=1}^T E_{q(x_t|x_0)}[\mathcal{L}_{t-1}] - \mathcal{L}_T(\text{常数}) \end{aligned}$$

$$\mathcal{L}_{t-1} = D_{KL}(q(x_{t-1}|x_t, x_0)||p_{\theta}(x_{t-1}|x_t))$$

$$\mathcal{L}_T = D_{KL}(q(x_T|x_0)||p_{\theta}(x_T)) = D_{kl}(q(x_T)||p(x_T)) \text{ 常数}$$

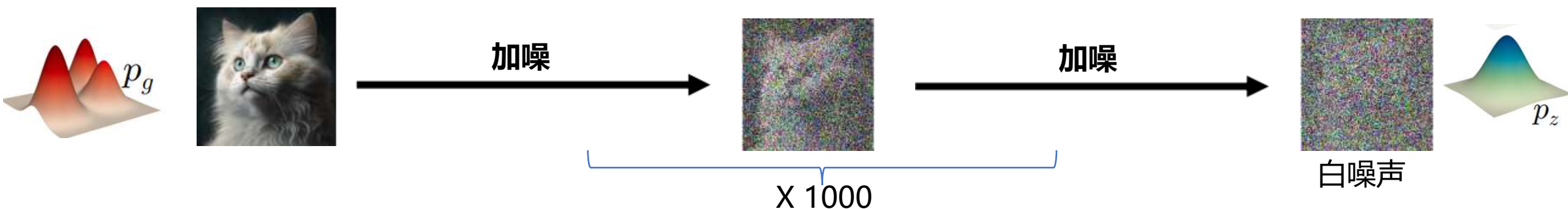
$$\mathcal{L}_0 = -\log p_{\theta}(x_0|x_1) = D_{KL}(q(x_0|x_1, x_0)||p_{\theta}(x_0|x_1)) \quad \mathcal{L}_{t-1} \text{在} t=1 \text{时的值}$$

回顾

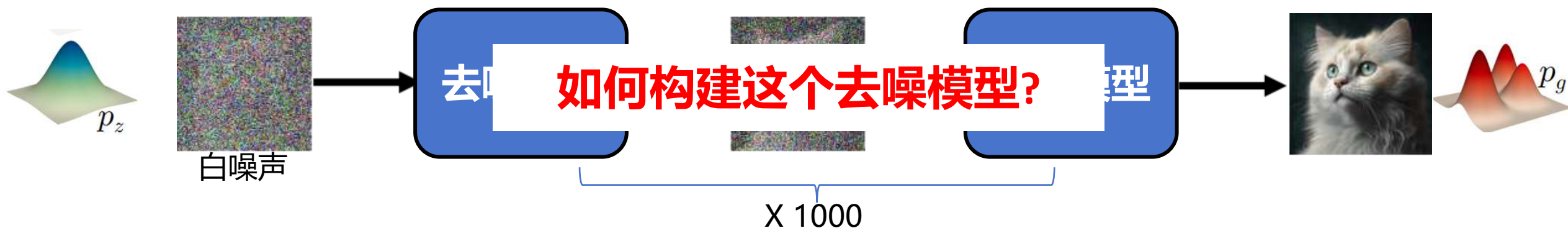
去噪扩散概率模型 (Denoising Diffusion Probabilistic Models, DDPM)

包括两个主要的阶段：**前向扩散阶段**和**反向去噪阶段**。

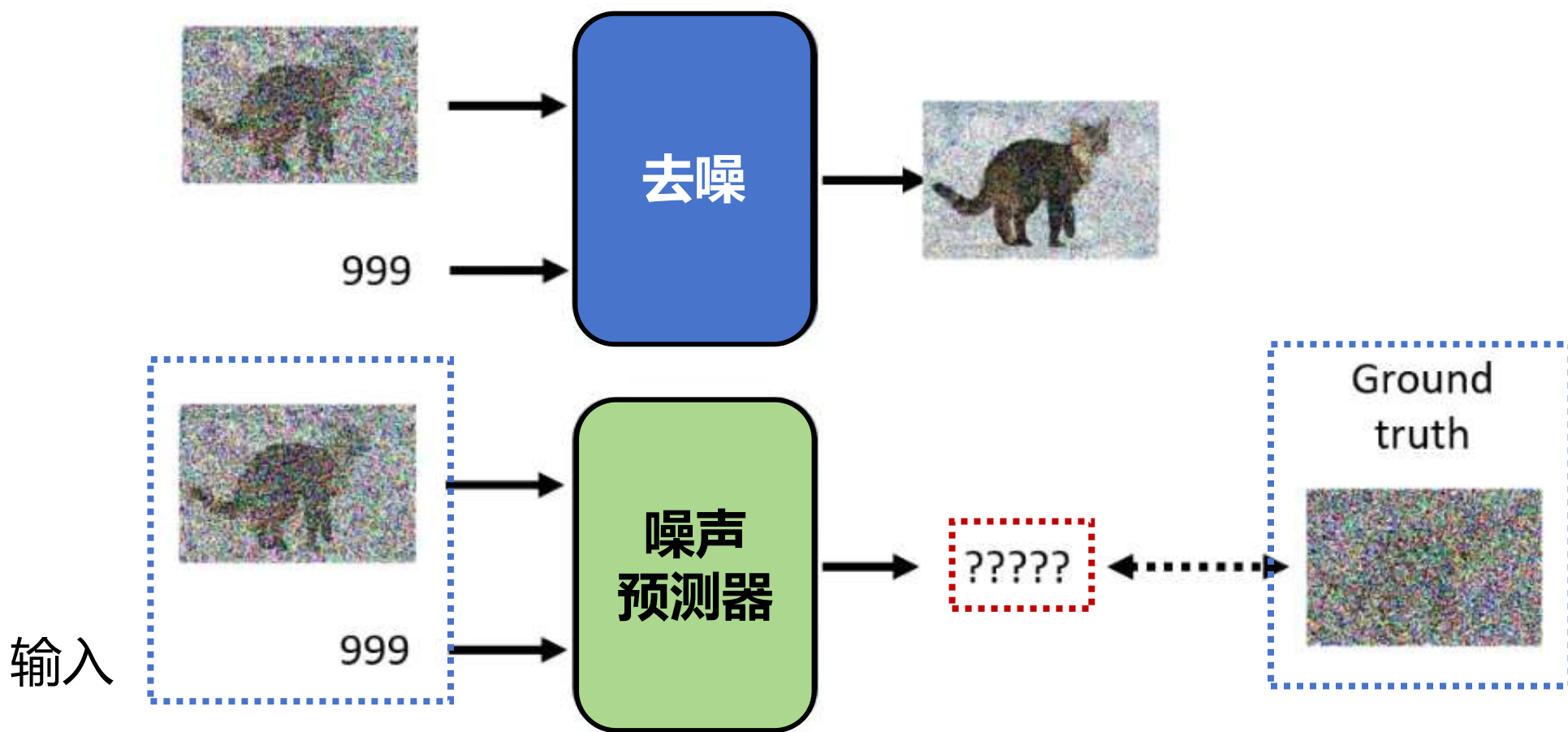
前向扩散阶段



反向去噪阶段：拟合前向扩散阶段的逆向过程



去噪模型训练

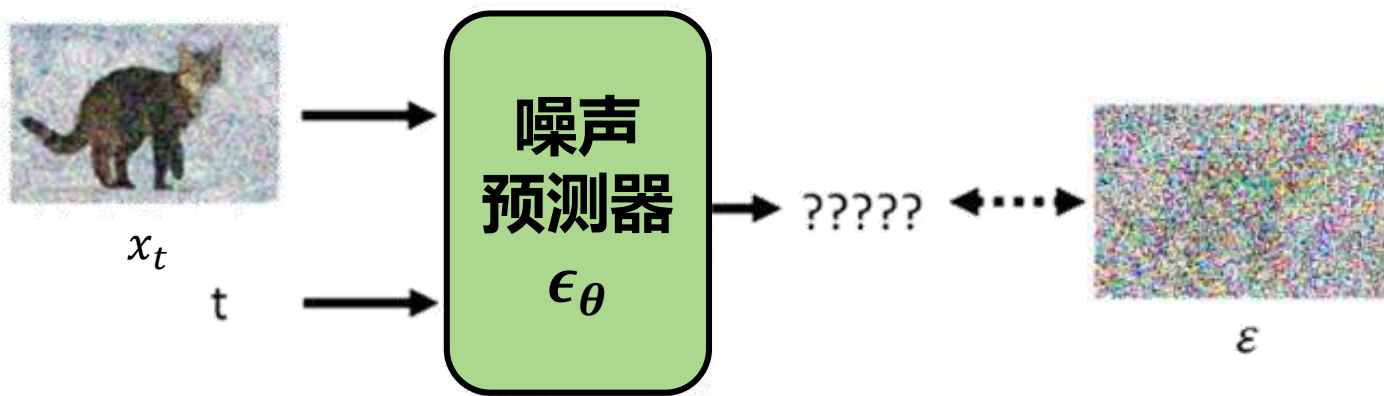


如何构造带噪声图片和Ground truth的数据对?
(加噪图像 x_t , 加噪步数 t , 噪声图 ε)

去噪模型训练

➤ 使用MSE loss优化噪声预测器

$$L = ||\epsilon - \epsilon_{\theta}(x_t, t)||$$



构建训练数据($x_t, t, epsilon$):

- 采样 $t \in [0, T]$
- $\epsilon \sim N(0, I)$
- $X_t = \sqrt{\alpha}x_0 + \sqrt{(1 - \alpha)}\epsilon$

最大化似然估计

DDPM中最大化似然估计等价于最大化置信下界

DDPM 最大化 $\log p_{\theta}(x)$ $\longrightarrow$ 最大化 $E_{q(x_1:x_T|x_0)}[\log(\frac{p(x_0:x_T)}{q(x_1:x_T|x_0)})]$



$$= \underbrace{E_{q(x_1|x_0)}[\log p_{\theta}(x_0|x_1)]}_{\text{重构项}} - \underbrace{KL(q(x_T|x_0)||p_{\theta}(x_T))}_{\text{先验匹配项}} - \sum_{t=2}^T \underbrace{E_{q(x_t|x_0)}[D_{KL}(q(x_{t-1}|x_t, x_0)||p_{\theta}(x_{t-1}|x_t))]}_{\text{去噪匹配项}}$$

重构项

先验匹配项

去噪匹配项

扩散模型的极大似然估计角度理解

最大化置信下界

$$\begin{aligned}
 & \operatorname{argmax}_{\theta} E_{q(x_1|x_0)} [\log p_{\theta}(x_0|x_1)] - KL(q(x_T|x_0)||p_{\theta}(x_T)) - \sum_{t=2}^T E_{q(x_t|x_0)} [D_{KL}(q(x_{t-1}|x_t, x_0)||p_{\theta}(x_{t-1}|x_t))] \\
 & \quad \underbrace{\hspace{10em}}_{\text{重构项}} \quad \underbrace{\hspace{10em}}_{\text{先验匹配项}} \quad \underbrace{\hspace{10em}}_{\text{去噪匹配项}} \\
 & = -E_{q(x_1|x_0)}[\mathcal{L}_0] - \mathcal{L}_T(\text{常数}) - \sum_{t=2}^T E_{q(x_t|x_0)}[\mathcal{L}_{t-1}]
 \end{aligned}$$

等价于 $\rightarrow \operatorname{argmin}_{\theta} \sum_{t=1}^T E_{q(x_t|x_0)}[\mathcal{L}_{t-1}] = \sum_{t=1}^T E_{q(x_t|x_0)}[D_{KL}(q(x_{t-1}|x_t, x_0)||p_{\theta}(x_{t-1}|x_t))]$



扩散模型的极大似然估计角度理解

最大化置信下界

$$\begin{aligned}\operatorname{argmax}_{\theta} \sum_{t=1}^T E_{q(x_t|x_0)}[\mathcal{L}_{t-1}] &= \sum_{t=1}^T E_{q(x_t|x_0)}[D_{KL}(\underbrace{q(x_{t-1}|x_t, x_0)}_{\text{扩散加噪}} \parallel \underbrace{p_{\theta}(x_{t-1}|x_t)}_{\text{反向去噪}})] \\ &= \sum_{t=1}^T E_{q(x_t|x_0)}[w_t \|\epsilon - \epsilon_{\theta}(x_t, t)\|^2] \quad w_t \text{由} \alpha_t \text{与} \beta_t \text{确定, 实践中设置为1}\end{aligned}$$

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}\left(\frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon\right), \sigma_t I\right)$$

$$p_{\theta}(x_{t-1}|x_t) = \mathcal{N}\left(\frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon_{\theta}(x_t, t)\right), \sigma_t I\right)$$

$$D_{KL}(\mathcal{N}_1 \parallel \mathcal{N}_2) = \log\left(\frac{\sigma_2}{\sigma_1}\right) + \frac{\sigma_2^2 + \|\mu_1 - \mu_2\|^2}{2\sigma_1^2} - \frac{1}{2}$$



扩散模型的极大似然估计角度理解

最大化置信下界

$$\begin{aligned}\operatorname{argmax}_{\theta} \sum_{t=1}^T E_{q(x_t|x_0)}[\mathcal{L}_{t-1}] &= \sum_{t=1}^T E_{q(x_t|x_0)}[D_{KL}(\underbrace{q(x_{t-1}|x_t, x_0)}_{\text{真实噪声}} \parallel \underbrace{p_{\theta}(x_{t-1}|x_t)}_{\text{预测噪声}})] \\ &= \sum_{t=1}^T E_{q(x_t|x_0)}[w_t \|\epsilon - \epsilon_{\theta}(x_t, t)\|^2] \quad w_t \text{由} \alpha_t \text{与} \beta_t \text{确定, 实践中设置为1} \\ &= \sum_{t=1}^T E_{q(\sqrt{\alpha_t}x_0 + \sqrt{1-\alpha_t}\epsilon|x_0)}[w_t \|\epsilon - \epsilon_{\theta}(\sqrt{\alpha_t}x_0 + \sqrt{1-\alpha_t}\epsilon, t)\|^2] \\ &\quad \text{实际变量为} \epsilon \\ &= \sum_{t=1}^T E_{\epsilon} \left[w_t \|\epsilon - \epsilon_{\theta}(\sqrt{\alpha_t}x_0 + \sqrt{1-\alpha_t}\epsilon, t)\|^2 \right] \\ \text{遍历} t \text{求和等价于对} t \text{随机采样} &= TE_{\epsilon, t} \left[w_t \|\epsilon - \epsilon_{\theta}(\sqrt{\alpha_t}x_0 + \sqrt{1-\alpha_t}\epsilon, t)\|^2 \right]\end{aligned}$$

重看训练过程

Algorithm 1 Training

- 1: **repeat**
- 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ ← 采样一张干净的图
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$ 采样一个数字 T 为 step 的最大值
- 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ← 采样一个噪声图像
- 5: Take gradient descent step on
$$\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2$$
- 6: **until** converged

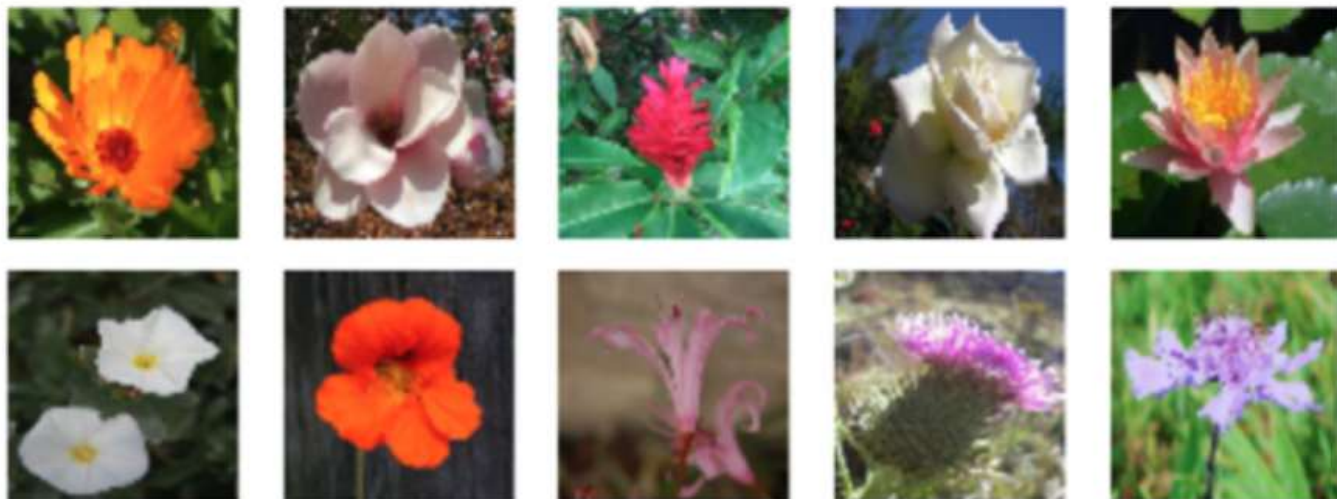
$$\operatorname{argmax}_{\theta} E_{\epsilon, t, \mathbf{x}_0} [\|\epsilon - \epsilon_{\theta}(x_t, t)\|]$$

目录

- 第一节 扩散模型的基本原理
- 第二节 扩散模型的极大似然估计角度理解
- 第三节 扩散模型的代码实践
- 第四节 文生图扩散模型与潜在扩散模型
- 第五节 是视觉模态扩散模型

扩散模型的代码实践

以Flowers数据集为例构建扩散模型



Flowers数据集

- 102种不同的花
- 8000个图片
- 64*64分辨率

```
train_data = utils.image_dataset_from_directory(  
    "/app/data/pytorch-challenge-flower-dataset/dataset",  
    labels=None,  
    image_size=(64, 64),  
    batch_size=None,  
    shuffle=True,  
    seed=42,  
    interpolation="bilinear",  
) ❶
```

```
def preprocess(img):  
    img = tf.cast(img, "float32") / 255.0  
    return img
```

```
train = train_data.map(lambda x: preprocess(x)) ❷  
train = train.repeat(5) ❸  
train = train.batch(64, drop_remainder=True) ❹
```

扩散模型的代码实践

前向扩散过程

$$x_t = \sqrt{1 - \beta_1} \dots \sqrt{1 - \beta_t} x_0 + \sqrt{1 - (1 - \beta_1) \dots (1 - \beta_t)} \epsilon$$


```
def linear_diffusion_schedule(diffusion_times):
    min_rate = 0.0001
    max_rate = 0.02
    betas = min_rate + tf.convert_to_tensor(diffusion_times) * (max_rate - min_rate)
    alphas = 1 - betas
    alphaBars = tf.math.cumprod(alphas)
    signal_rates = alphaBars
    noise_rates = 1 - alphaBars
    return noise_rates, signal_rates
```

T = 1000

diffusion_times = [x/T for x in range(T)] ❶

linear_noise_rates, linear_signal_rates = linear_diffusion_schedule(
 diffusion_times

) ❷

➤ 定义扩散参数: β_t 和 $\bar{\alpha}_t$

- 线性参数形式
- β_t 随t线性变化

扩散模型的代码实践

前向扩散过程

➤ 定义扩散参数: β_t 和 $\bar{\alpha}_t$

• 余弦参数形式

```
def offset_cosine_diffusion_schedule(diffusion_times): ②
    min_signal_rate = 0.02
    max_signal_rate = 0.95
    start_angle = tf.acos(max_signal_rate)
    end_angle = tf.acos(min_signal_rate)

    diffusion_angles = start_angle + diffusion_times * (end_angle - start_angle)

    signal_rates = tf.cos(diffusion_angles)
    noise_rates = tf.sin(diffusion_angles)
```

• 偏移余弦参数形式

```
def offset_cosine_diffusion_schedule(diffusion_times): ②
    min_signal_rate = 0.02
    max_signal_rate = 0.95
    start_angle = tf.acos(max_signal_rate)
    end_angle = tf.acos(min_signal_rate)

    diffusion_angles = start_angle + diffusion_times * (end_angle - start_angle)

    signal_rates = tf.cos(diffusion_angles)
    noise_rates = tf.sin(diffusion_angles)
    return noise_rates, signal_rates
```

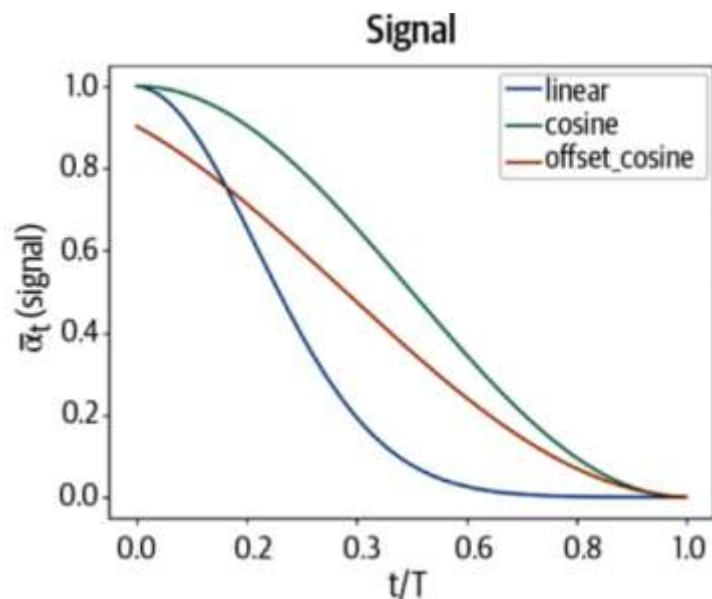
$$x_t = \frac{\sqrt{1 - \beta_1} \dots \sqrt{1 - \beta_t}}{\sqrt{\bar{\alpha}_t}} \text{ (cat image)} + \frac{\sqrt{1 - (1 - \beta_1) \dots (1 - \beta_t)}}{\sqrt{1 - \bar{\alpha}_t}} \text{ (noise image)}$$

➤ 只定义 $\bar{\alpha}_t$ 不定义 β_t

$$\bar{\alpha}_t = \cos^2\left(\frac{t}{T} \cdot \frac{\pi}{2}\right)$$

➤ 添加偏移量, 避免加噪初期的步幅过小。

前向扩散过程



- 不同参数设定下的 $\bar{\alpha}_t$ 变化曲线不同
- 余弦参数 (cosine) 下添加噪声的过程更为渐进
- 实践中偏移余弦 (offset_cosine) 的设置效果最好



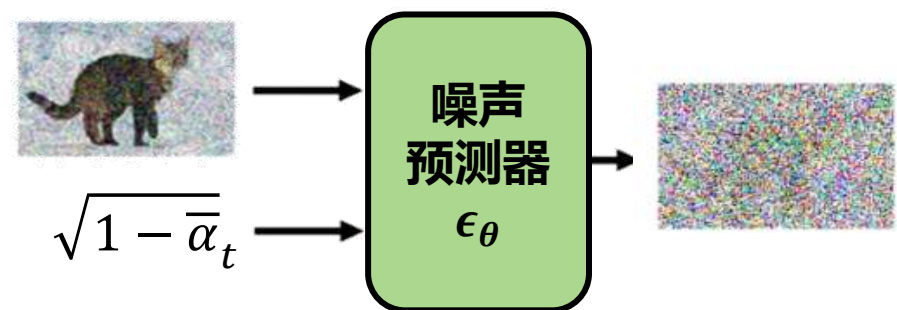
不同 t 下图片被破坏的程度

扩散模型的代码实践

反向去噪过程

```
class DiffusionModel(models.Model):
    def __init__(self):
        super().__init__()
        self.normalizer = layers.Normalization()
        self.network = unet
        self.ema_network = models.clone_model(self.network)
        self.diffusion_schedule = cosine_diffusion_schedule
        ...

    def denoise(self, noisy_images, noise_rates, signal_rates, training):
        if training:
            network = self.network
        else:
            network = self.ema_network
        pred_noises = network(
            [noisy_images, noise_rates**2], training=training
        )
        pred_images = (noisy_images - noise_rates * pred_noises) / signal_rates
        return pred_noises, pred_images
```



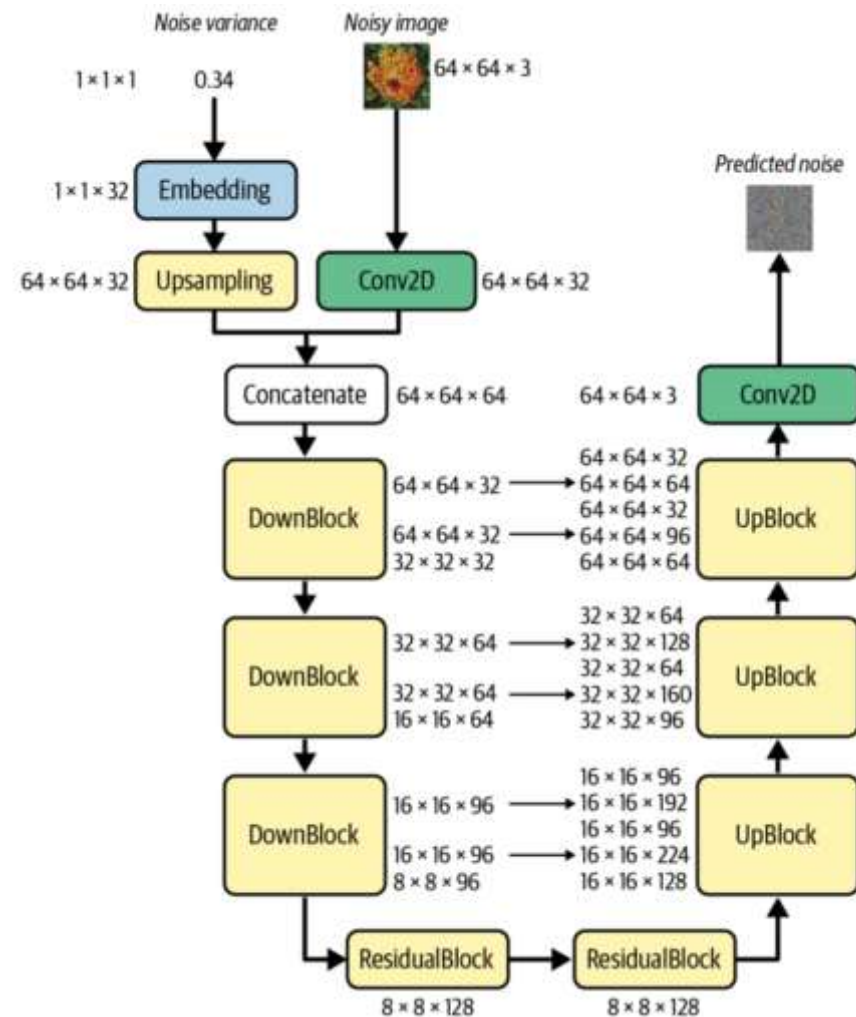
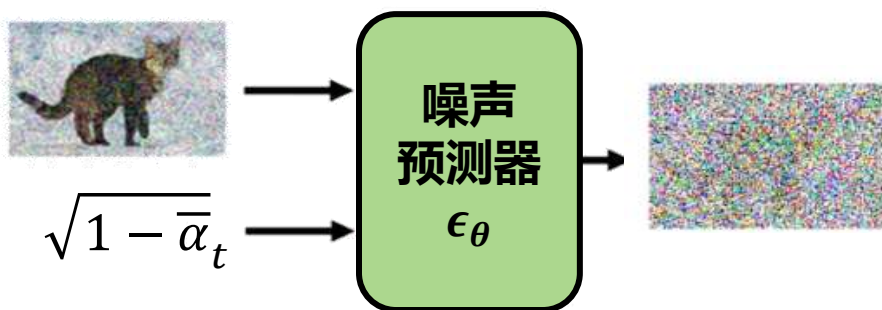
$$x_0(x_t, t) = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

如何设置噪声预测器网络结构？

扩散模型的代码实践

噪声预测器网络架构：U-Net

- 核心任务：接收加噪图像与噪声强度，输出噪声图
- 任务特性：输入输出形状一致
- U-Net：呈 ‘U’字形结构的卷积神经网络

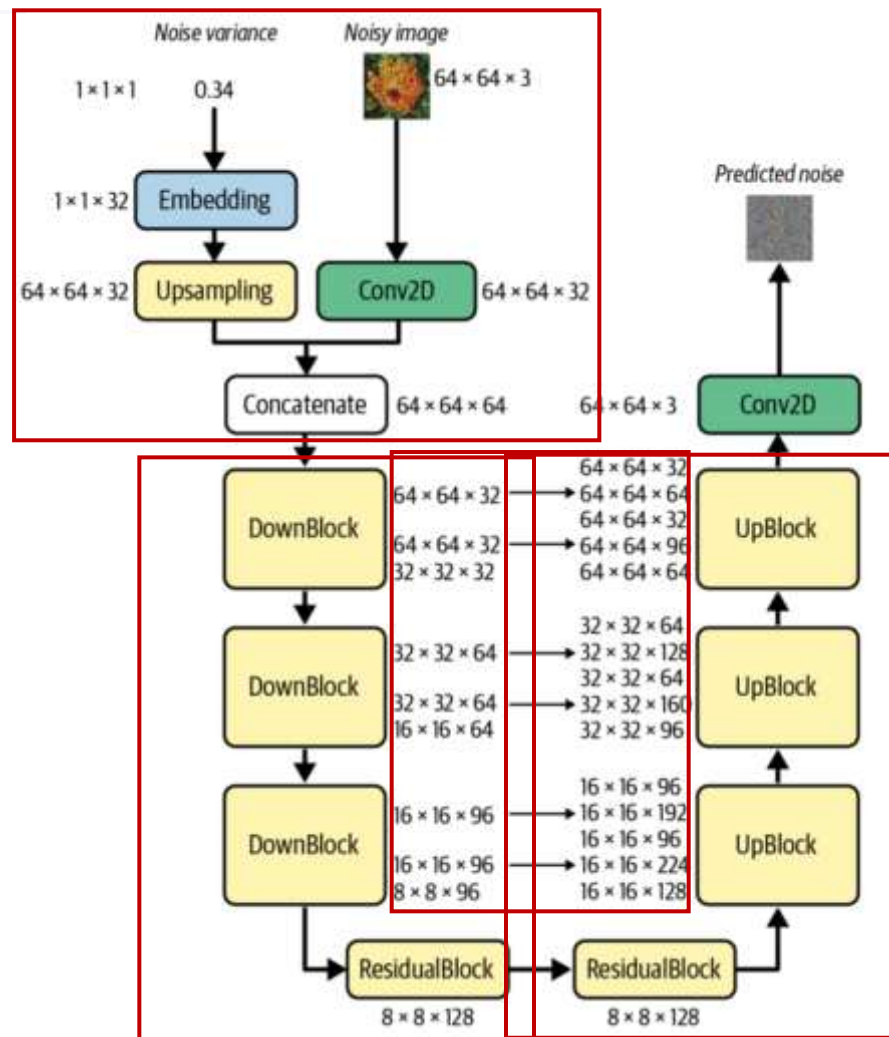


扩散模型的代码实践

噪声预测器网络架构：U-Net

2. 关键组件流程

- 输入融合
 - 双路输入：图像 (64×64) + 噪声方差 (标量)。
 - **Embedding 层**：将噪声方差（时间步）映射为特征向量，告诉网络当前的去噪阶段。
- 下采样路径
 - 模块：DownBlock
 - 作用：逐层压缩图像尺寸 ($64 \rightarrow 32 \rightarrow 16$)，提取高层抽象特征。
- 上采样路径
 - 模块：UpBlock
 - 作用：逐层恢复图像尺寸，还原空间信息。
- 跳跃连接
 - 图示：中间的横向箭头。
 - 关键作用：将左侧丢失的**高频细节**（纹理、边缘）直接拼接到右侧，保证生成的噪声细节清晰。



扩散模型的代码实践

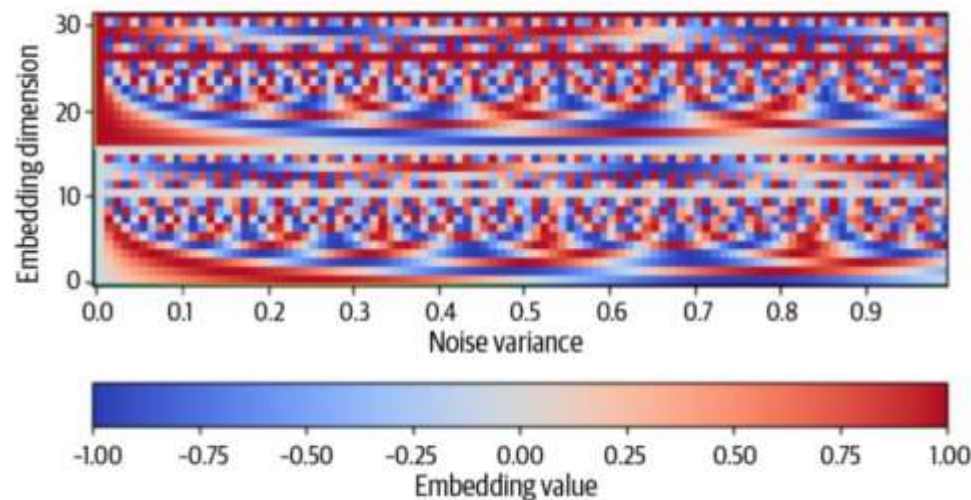
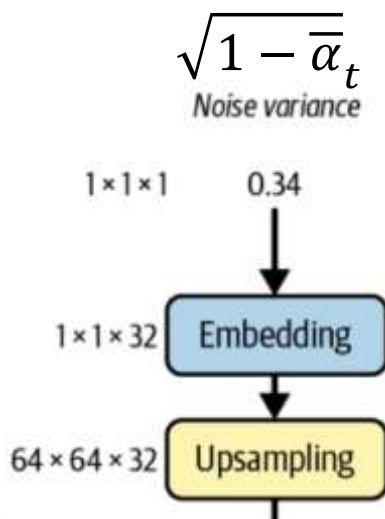
噪声预测器网络架构：U-Net

噪声嵌入层: 输入为1维数值（噪声强度），输出为32维向量

$$\gamma(x) = (\sin(2\pi e^{0f} x), \dots, \sin(2\pi e^{(L-1)f} x), \cos(2\pi e^{0f} x), \dots, \cos(2\pi e^{(L-1)f} x))$$

$$f = \frac{\ln(1000)}{L-1}$$

```
def sinusoidal_embedding(x):
    frequencies = tf.exp(
        tf.linspace(
            tf.math.log(1.0),
            tf.math.log(1000.0),
            16,
        )
    )
    angular_speeds = 2.0 * math.pi * frequencies
    embeddings = tf.concat(
        [tf.sin(angular_speeds * x), tf.cos(angular_speeds * x)], axis=3
    )
    return embeddings
```

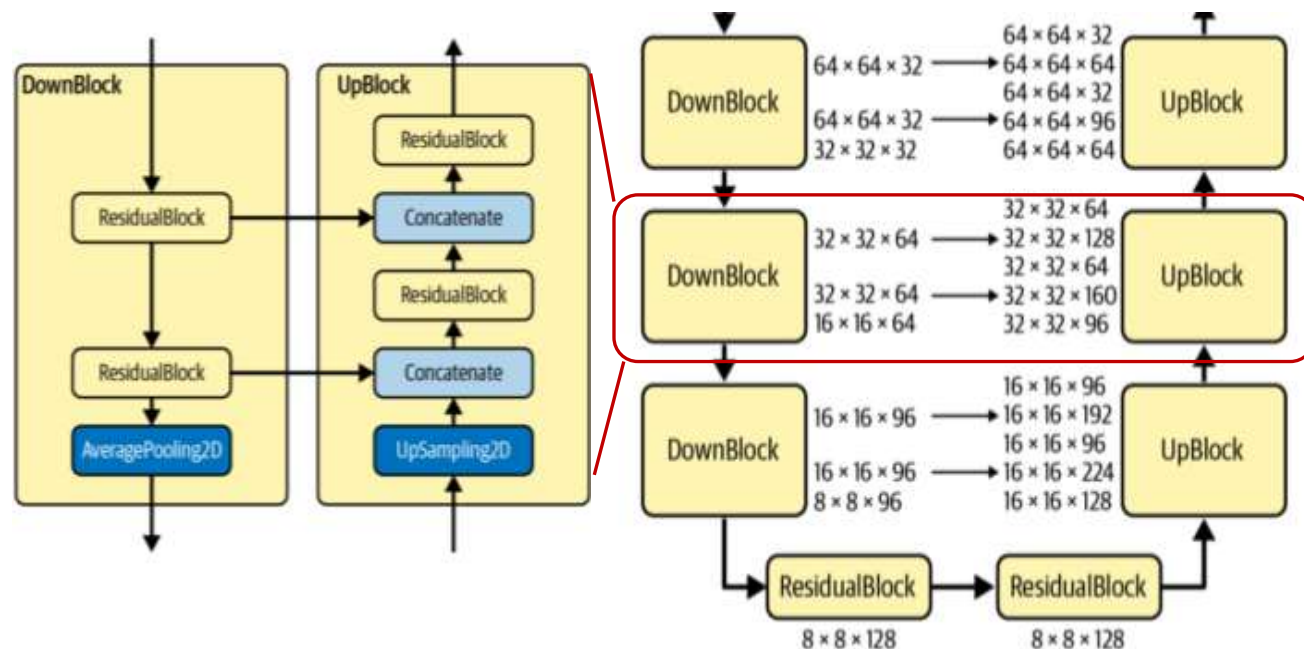


- 不同噪声强度对应嵌入向量不同
- 相邻噪声强度有一定的相似性

噪声预测器网络架构：U-Net

➤ 下采样模块 (DownBlock)

- 功能：压缩图像尺寸，提取高层特征。

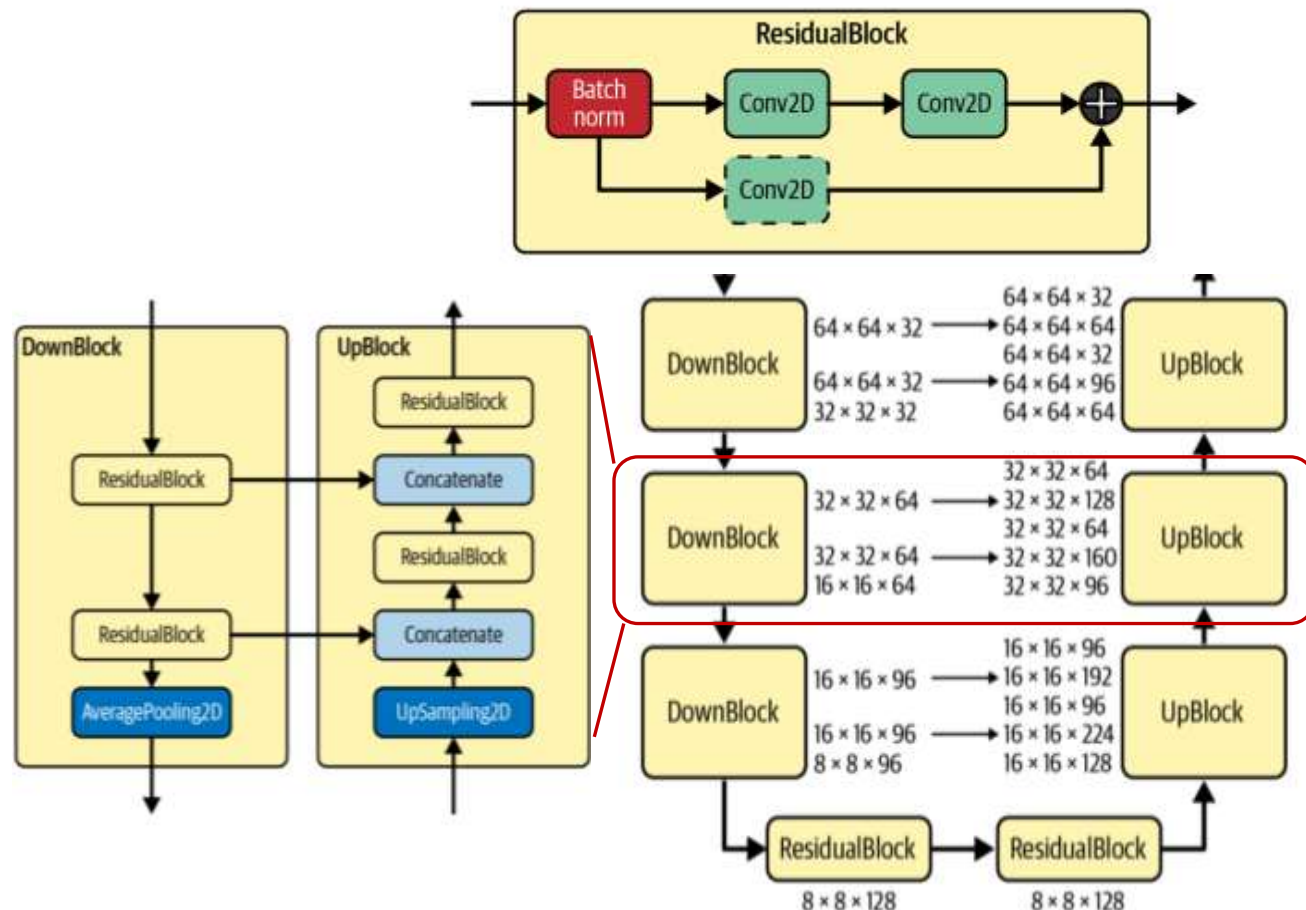


扩散模型的代码实践

噪声预测器网络架构：U-Net

➤ 流程细节：

- **特征提取：** 通过残差网络块 (ResidualBlock) 增加特征通道数，增强表示能力、保留局部空间信息
- **尺寸减半：** 经过AveragePooling2D 层，图像长宽减半，扩大感受野，提取更高层特征。
- **关键动作：** 将每个 ResidualBlock 的输出存入列表，作为跳跃连接备用。

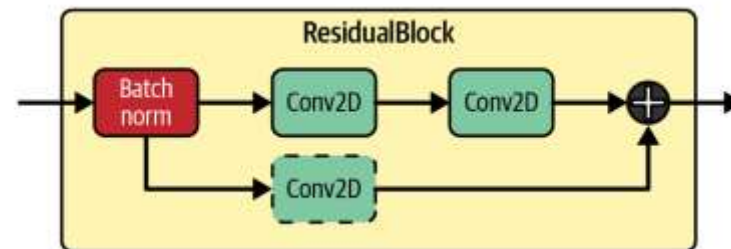


扩散模型的代码实践

噪声预测器网络架构：U-Net

```
def ResidualBlock(width):
    def apply(x):
        input_width = x.shape[3]
        if input_width == width: ❶
            residual = x
        else:
            residual = layers.Conv2D(width, kernel_size=1)(x)
        x = layers.BatchNormalization(center=False, scale=False)(x) ❷
        x = layers.Conv2D(
            width, kernel_s
        )(x) ❸
        x = layers.Conv2D(w
        x = layers.Add()([x
        return x

    return apply
```



残差网络块
ResidualBlock

特点：通过跳跃连接将输入直接加到输出上

Deep residual learning for image recognition

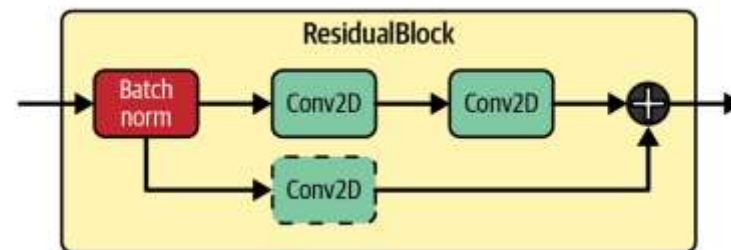
[K He, X Zhang, S Ren, J Sun](#) - ... and [pattern recognition](#), 2016 - openaccess.thecvf.com

... **Deeper** neural **networks** are more difficult to train. We present a **residual learning** framework to ease the training of **networks** that are substantially **deeper** than those used previously. ...

☆ Save 剪 Cite Cited by 311886 Related articles All 51 versions 剪

扩散模型的代码实践

噪声预测器网络架构：U-Net

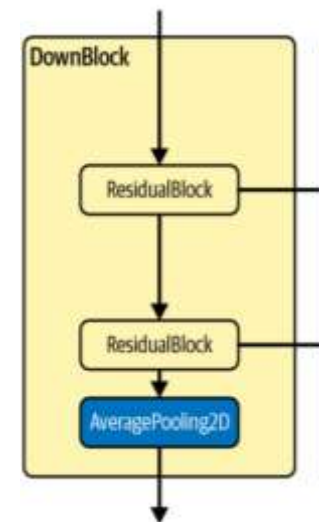


残差链接网络
ResidualBlock

- 解决两大难题：
- **梯度消失**：网络越深，深层的梯度越微小
 - 残差链接为梯度回传提供“快捷通道”。
 - **退化问题**：网络越深，准确率反而下降。
 - 残差链接保证深层网络至少不比浅层差。

噪声预测器网络架构：U-Net

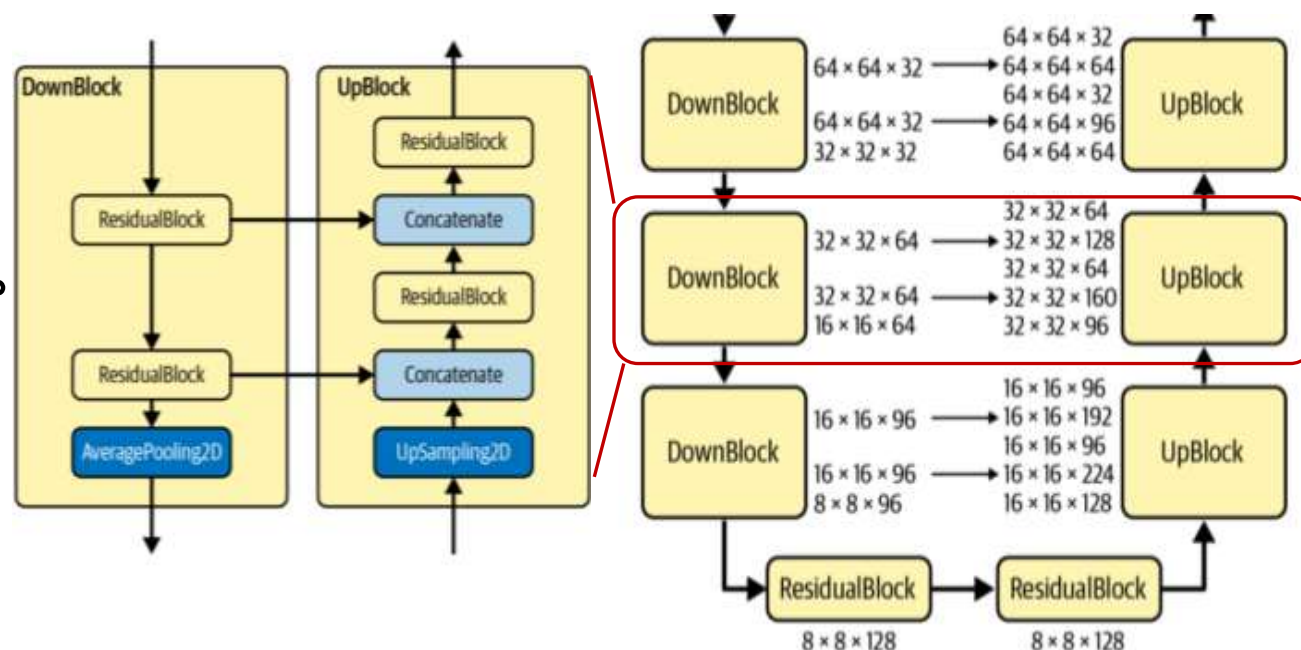
```
def DownBlock(width, block_depth):  
    def apply(x):  
        x, skips = x  
        for _ in range(block_depth):  
            x = ResidualBlock(width)(x) ❶  
            skips.append(x) ❷  
        x = layers.AveragePooling2D(pool_size=2)(x) ❸  
        return x  
    return apply
```



噪声预测器网络架构：U-Net

➤ 上采样模块 (UpBlock)

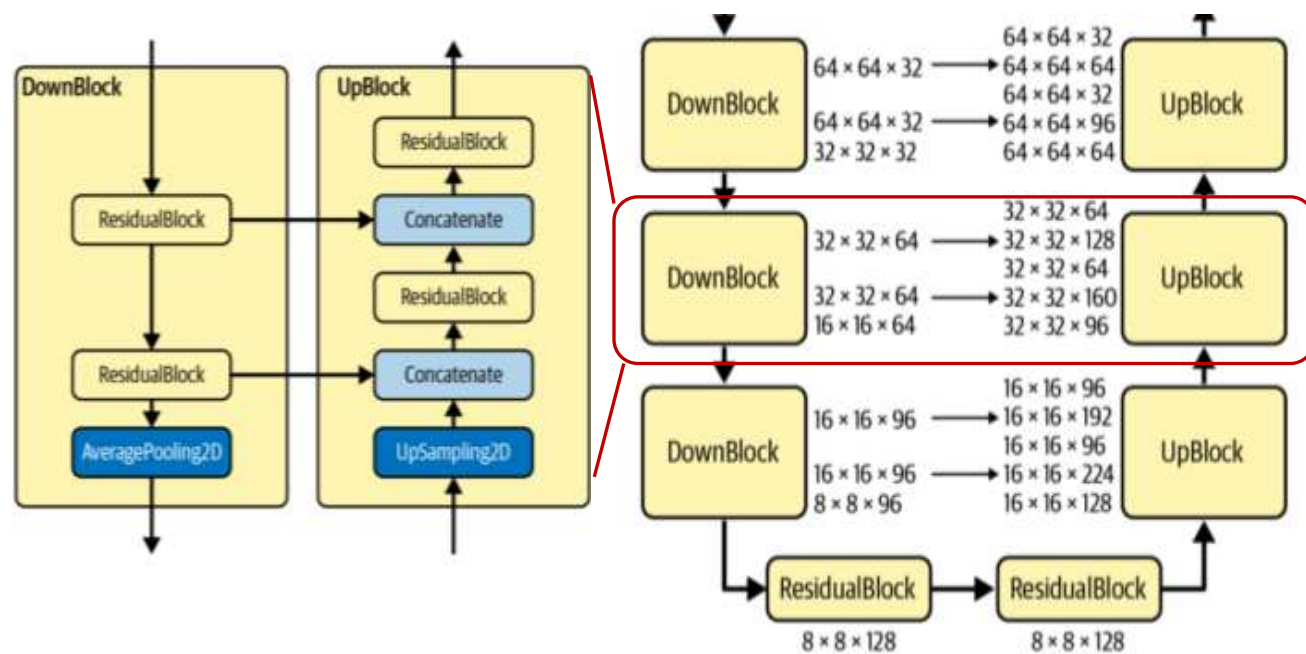
- 功能：恢复图像尺寸，融合细节信息。



噪声预测器网络架构：U-Net

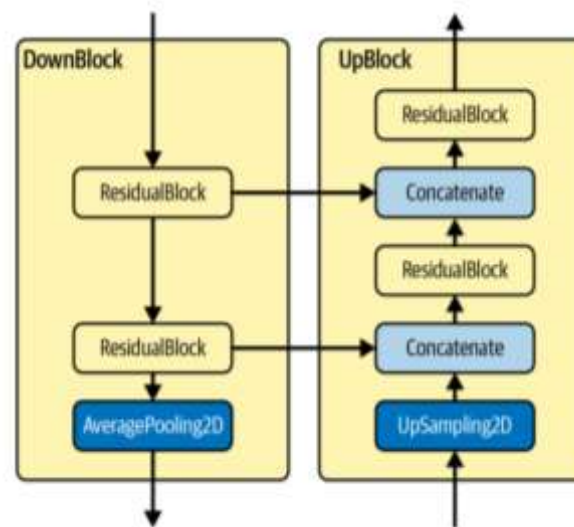
➤ 流程细节：

- 1. 尺寸翻倍：** 使用 UpSampling2D 层放大图像。
- 2. 特征融合：** 将 DownBlock 存下的特征与当前特征进行**拼接**。
- 3. 特征降维：** 通过 ResidualBlock 减少通道数，逐步还原图像细节。



噪声预测器网络架构：U-Net

```
def UpBlock(width, block_depth):
    def apply(x):
        x, skips = x
        x = layers.UpSampling2D(size=2, interpolation="bilinear")(x) ④
        for _ in range(block_depth):
            x = layers.Concatenate()([x, skips.pop()]) ⑤
            x = ResidualBlock(width)(x) ⑥
        return x
    return apply
```



噪声预测器网络架构：U-Net

```
noisy_images = layers.Input(shape=(64, 64, 3)) ❶
x = layers.Conv2D(32, kernel_size=1)(noisy_images) ❷
noise_variances = layers.Input(shape=(1, 1, 1)) ❸
noise_embedding = layers.Lambda(sinusoidal_embedding)(noise_variances) ❹
noise_embedding = layers.UpSampling2D(size=64, interpolation="nearest")(
    noise_embedding
) ❺
x = layers.Concatenate()([x, noise_embedding]) ❻

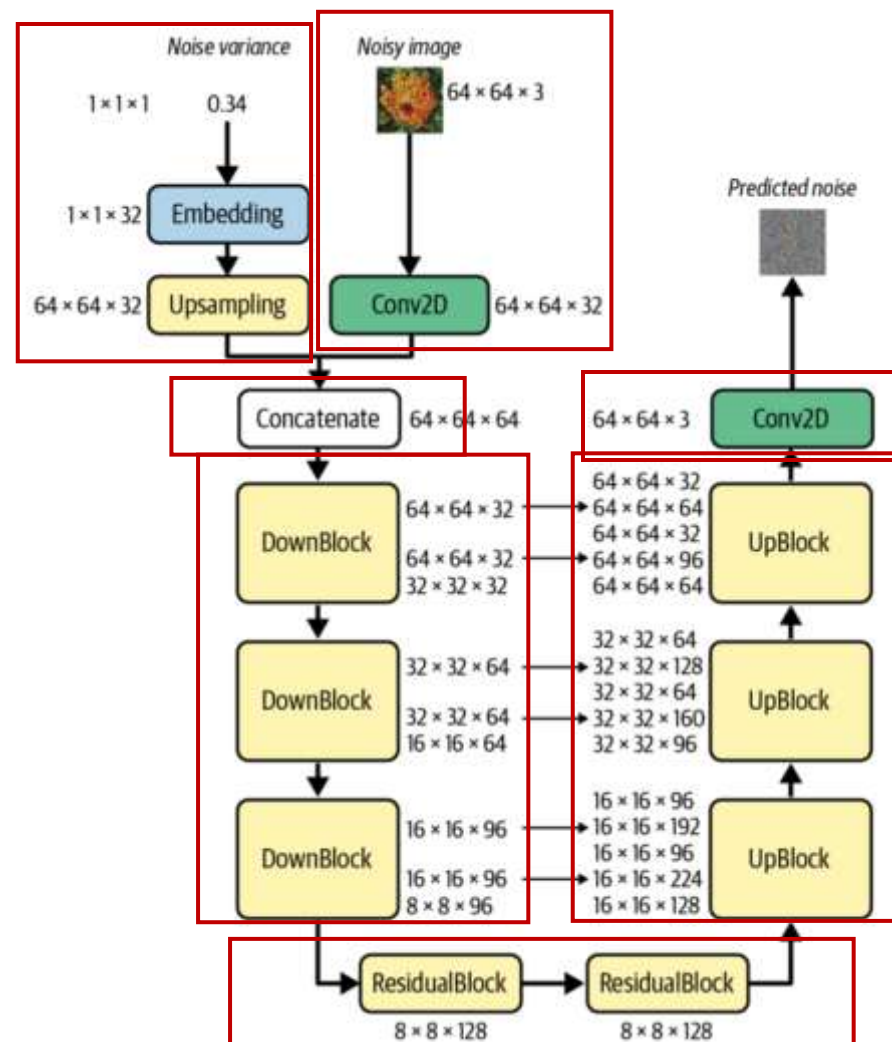
skips = [] ❼

x = DownBlock(32, block_depth = 2)([x, skips]) ❽
x = DownBlock(64, block_depth = 2)([x, skips])
x = DownBlock(96, block_depth = 2)([x, skips])
x = ResidualBlock(128)(x) ❾
x = ResidualBlock(128)(x)

x = UpBlock(96, block_depth = 2)([x, skips]) ❿
x = UpBlock(64, block_depth = 2)([x, skips])
x = UpBlock(32, block_depth = 2)([x, skips])

x = layers.Conv2D(3, kernel_size=1, kernel_initializer="zeros")(x) 11

UNET = models.Model([noisy_images, noise_variances], x, name="UNET") 12
```





扩散模型的代码实践

扩散模型训练

➤ 训练流程伪代码

Algorithm 1 Training

- 1: **repeat**
- 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$ ③
- 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ②
- 5: Take gradient descent step on
$$\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2$$
 ⑦ ⑧
- 6: **until** converged ⑤

$$\alpha_1, \alpha_2, \dots, \alpha_T$$
 ④

```
def train_step(self, images):
    images = self.normalizer(images, training=True) ①
    noises = tf.random.normal(shape=tf.shape(images)) ②
    batch_size = tf.shape(images)[0]
    diffusion_times = tf.random.uniform(
        shape=(batch_size, 1, 1, 1), minval=0.0, maxval=1.0
    ) ③
    noise_rates, signal_rates = self.cosine_diffusion_schedule(
        diffusion_times
    ) ④
    noisy_images = signal_rates * images + noise_rates * noises ⑤
    with tf.GradientTape() as tape:
        pred_noises, pred_images = self.denoise(
            noisy_images, noise_rates, signal_rates, training=True
        ) ⑥
        noise_loss = self.loss(noises, pred_noises) ⑦
        gradients = tape.gradient(noise_loss, self.network.trainable_weights)
        self.optimizer.apply_gradients(
            zip(gradients, self.network.trainable_weights)
        ) ⑧
        self.noise_loss_tracker.update_state(noise_loss)

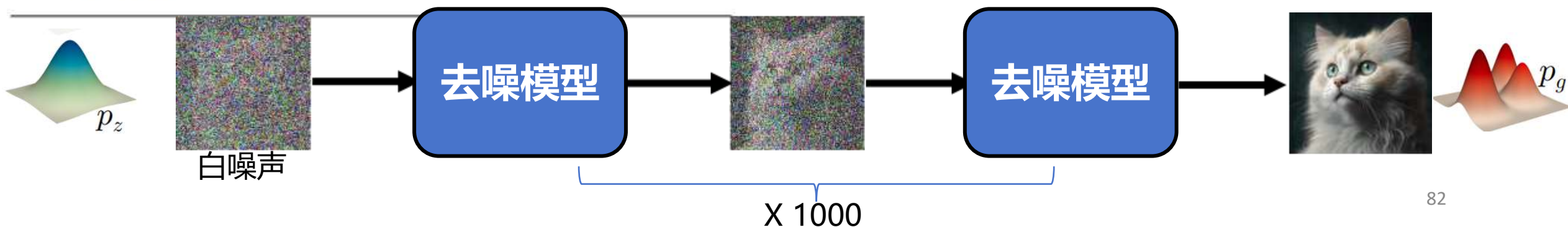
    for weight, ema_weight in zip(
        self.network.weights, self.ema_network.weights
    ):
        ema_weight.assign(0.999 * ema_weight + (1 - 0.999) * weight) ⑨

    return {m.name: m.result() for m in self.metrics}
```

扩散模型生成推理

Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 2: **for** $t = T, \dots, 1$ **do**
- 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
- 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
- 5: **end for**
- 6: **return** $\mathbf{x}_0$



扩散模型的代码实践

扩散模型生成推理

➤ 原始推理方法：需要1000步模型迭代，时间开销大

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

如何加速推理生成？

$$\mathbf{x}_0(\mathbf{x}_t, t) = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_{\theta}(\mathbf{x}_t, t)}{\sqrt{\bar{\alpha}_t}}$$

$$P_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0(\mathbf{x}_t, t))$$

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}\left(\frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right), \sigma_t \mathbf{I}\right)$$

1. 使用模型根据 $\mathbf{x}_t$ 预测噪声,并估算原图像 $\mathbf{x}_0$
2. 利用估算的 $\mathbf{x}_0$ 代入 $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$
3. 采样得到 $\mathbf{x}_{t-1}$

扩散模型的代码实践

扩散模型生成推理

- 原始推理方法：需要1000步模型迭代，时间开销大

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

$$x_0(x_t, t) = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_{\theta}(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

$$P_{\theta}(x_s | x_t) = q(x_s | x_t, x_0(x_t, t))$$

$$p_{\theta}(x_{t-1} | x_t) = \mathcal{N}\left(\frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right), \sigma_t I\right)$$

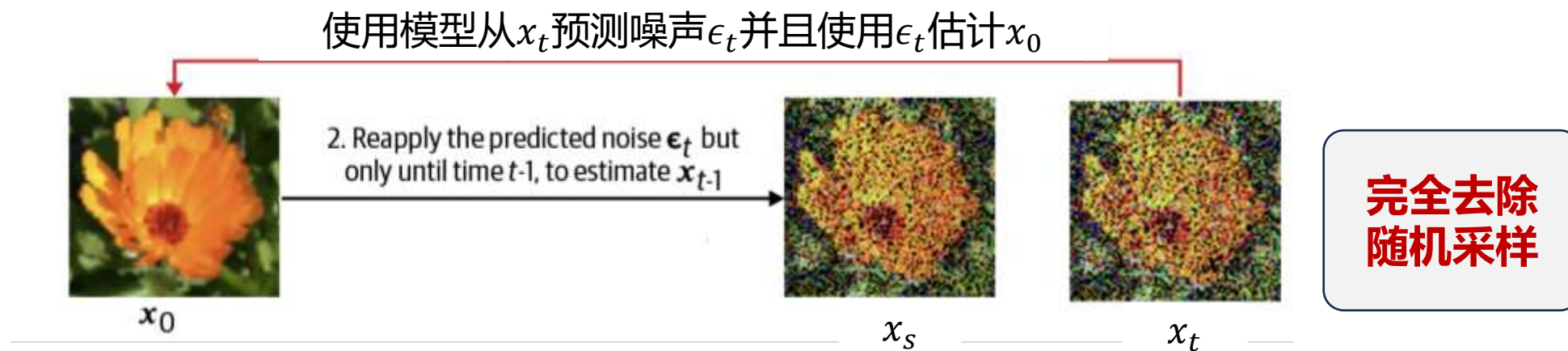
如何加速推理生成？

- 引入跳步推理？

1. 使用模型根据 x_t 预测噪声, 并估算原图像 x_0
2. 利用估算的 x_0 代入 $q(x_s | x_t, x_0)$, $t - s \gg 1$
3. 采样得到 x_s

扩散模型生成推理

- DDIM 算法：一个更直观更高效的采样方式
 - 使用模型根据 x_t 预测噪声, 并估算原图像 x_0
 - 重新应用预测的噪声, 估算 x_s



完全去除
随机采样

$$x_0(x_t, t) = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

$$x_s = \sqrt{\bar{\alpha}_s} x_0(x_t, t) + \sqrt{1 - \bar{\alpha}_s} \epsilon_\theta(x_t, t)$$

• 参数只和 t 相关

• 参数只和 s 相关

扩散模型生成推理

➤ DDIM 算法:

- Step_size 控制 t 与 s 之间的间隔
- 选出当前扩散步 t
- 选取对应的 $\bar{\alpha}_t, 1 - \bar{\alpha}_t$
- 模型预测噪声 $\epsilon_\theta(x_t, t)$ 与估计 $x_0(x_t, t)$
- 确定下一步扩散步 s
- 计算 x_s

```
class DiffusionModel(models.Model):  
    ...  
  
    def reverse_diffusion(self, initial_noise, diffusion_steps):  
        num_images = initial_noise.shape[0]  
        step_size = 1.0 / diffusion_steps  
        current_images = initial_noise  
        for step in range(diffusion_steps): ①  
            diffusion_times = tf.ones((num_images, 1, 1, 1)) - step * step_size ②  
            noise_rates, signal_rates = self.diffusion_schedule(diffusion_times) ③  
            pred_noises, pred_images = self.denoise(  
                current_images, noise_rates, signal_rates, training=False  
            ) ④  
            next_diffusion_times = diffusion_times - step_size ⑤  
            next_noise_rates, next_signal_rates = self.diffusion_schedule(  
                next_diffusion_times  
            ) ⑥  
            current_images = (  
                next_signal_rates * pred_images + next_noise_rates * pred_noises  
            ) ⑦  
        return pred_images ⑧
```

扩散模型的代码实践

扩散模型生成效果

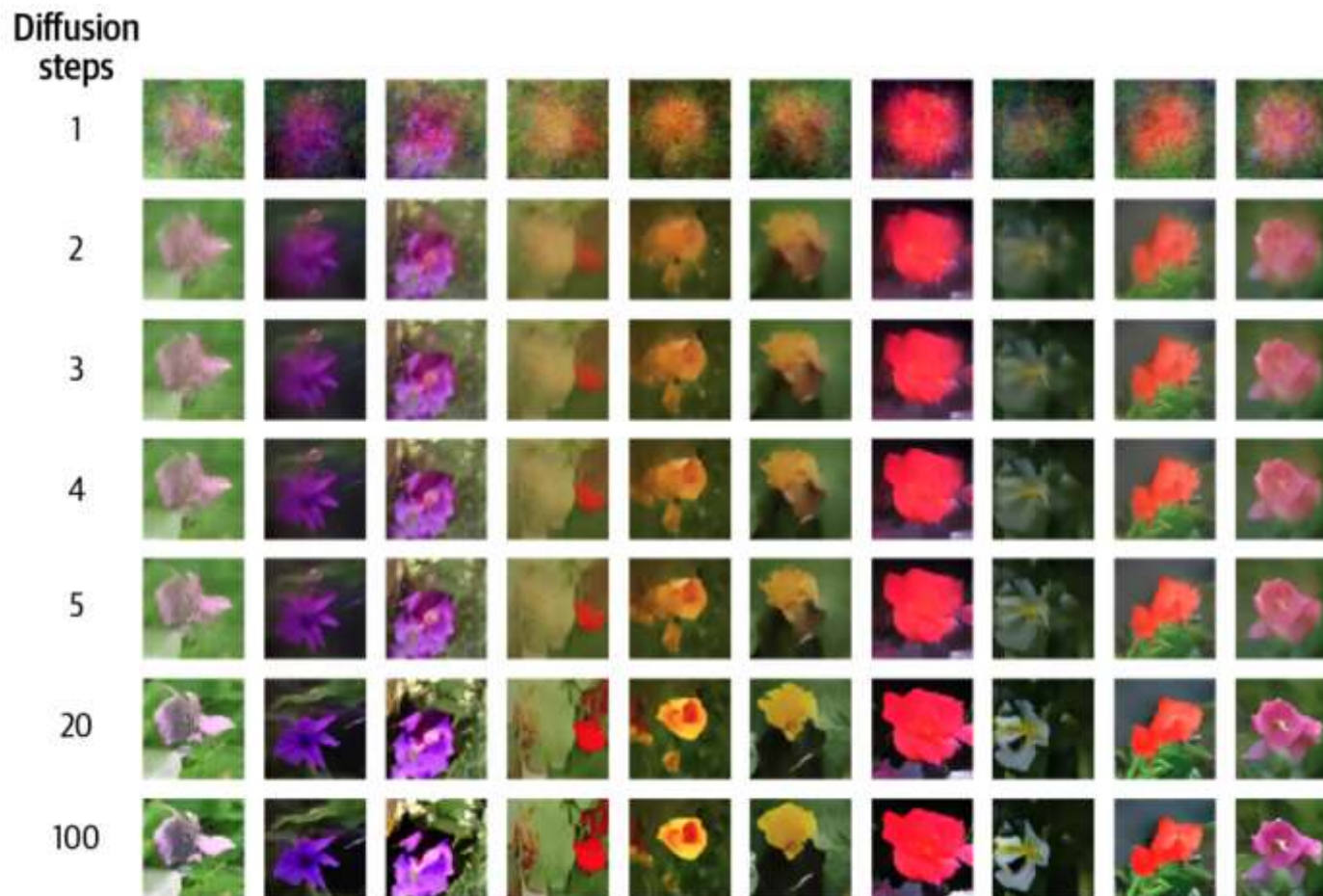


随着训练轮次的增加，
生成效果越来越好

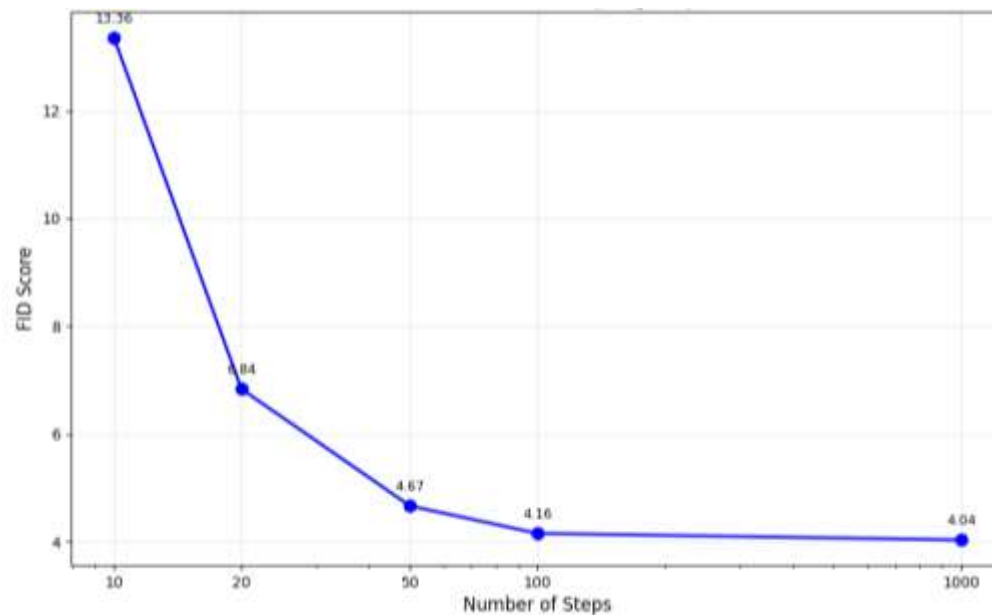


扩散模型生成效果

- 扩散步数越大生成效果越好
- DDIM 100步下效果已经很好



扩散模型生成效果



实际使用中，DDIM采样步数**设为50**
步，可兼顾效率与质量。

扩散模型的代码实践

作业

- 基于flowers数据集实现DDPM与DDIM推理
- 要求:
 - 基于pytorch实现代码
 - 提交完整运行的jupyter文件



网盘提交:

<https://pan.nuaa.edu.cn/collection/23d7b2b5381bd8af2d91107008d88f5d>

命名格式: **Homework4-姓名-学号**

截止时间: **下周日**

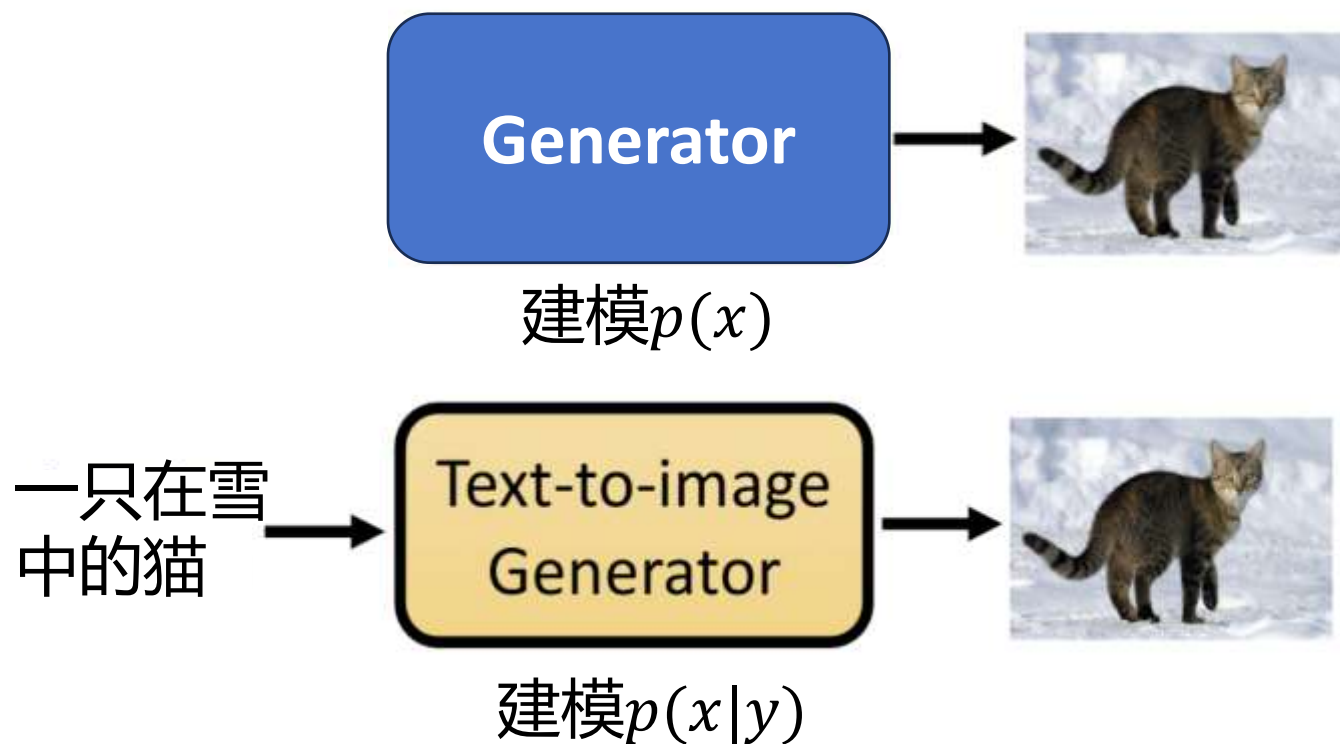
目录

- 第一节 扩散模型的基本原理
- 第二节 扩散模型的极大似然估计角度理解
- 第三节 扩散模型的代码实践
- 第四节 文生图扩散模型与潜在扩散模型
- 第五节 非视觉模态扩散模型

文生图扩散模型与潜在扩散模型

条件生成：文生图模型

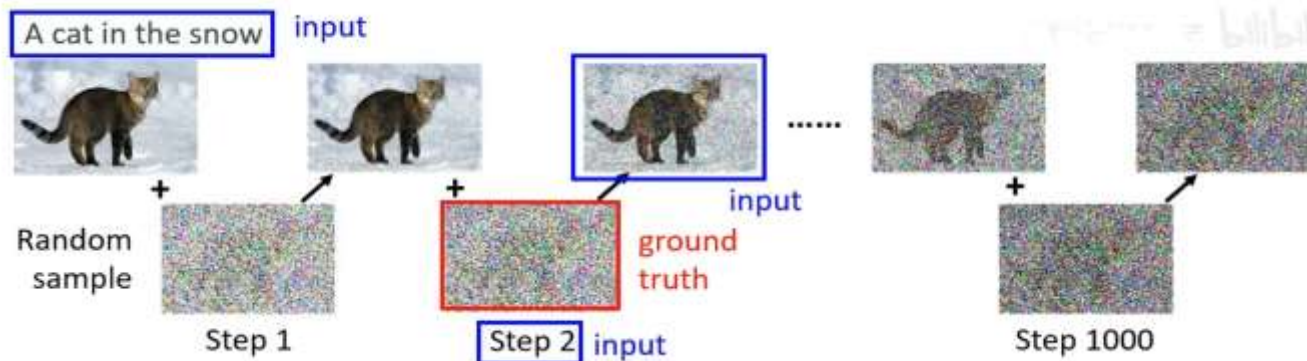
给定文本描述，生成符合描述的图片



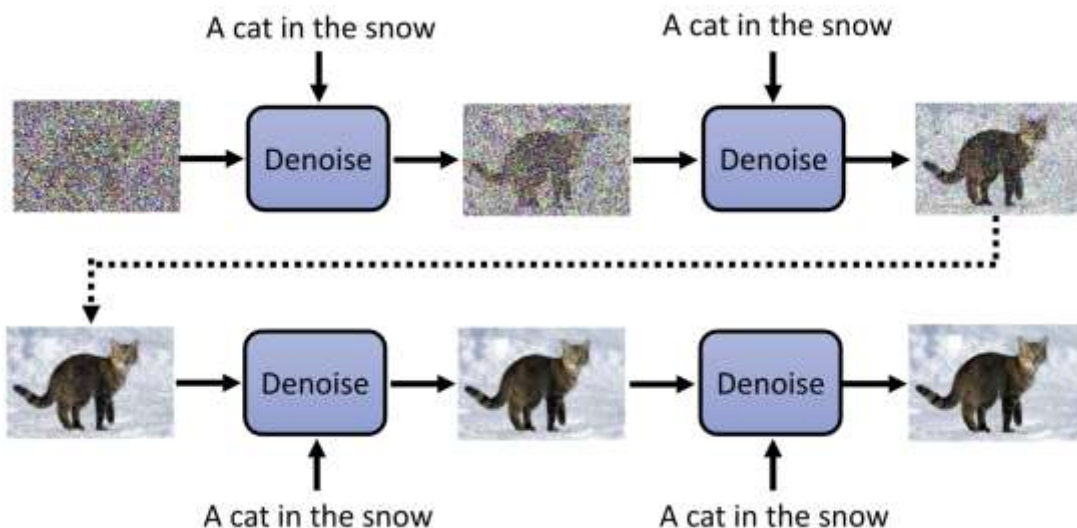
文生图扩散模型与潜在扩散模型

条件生成：文生图扩散模型

前向扩散



反向去噪



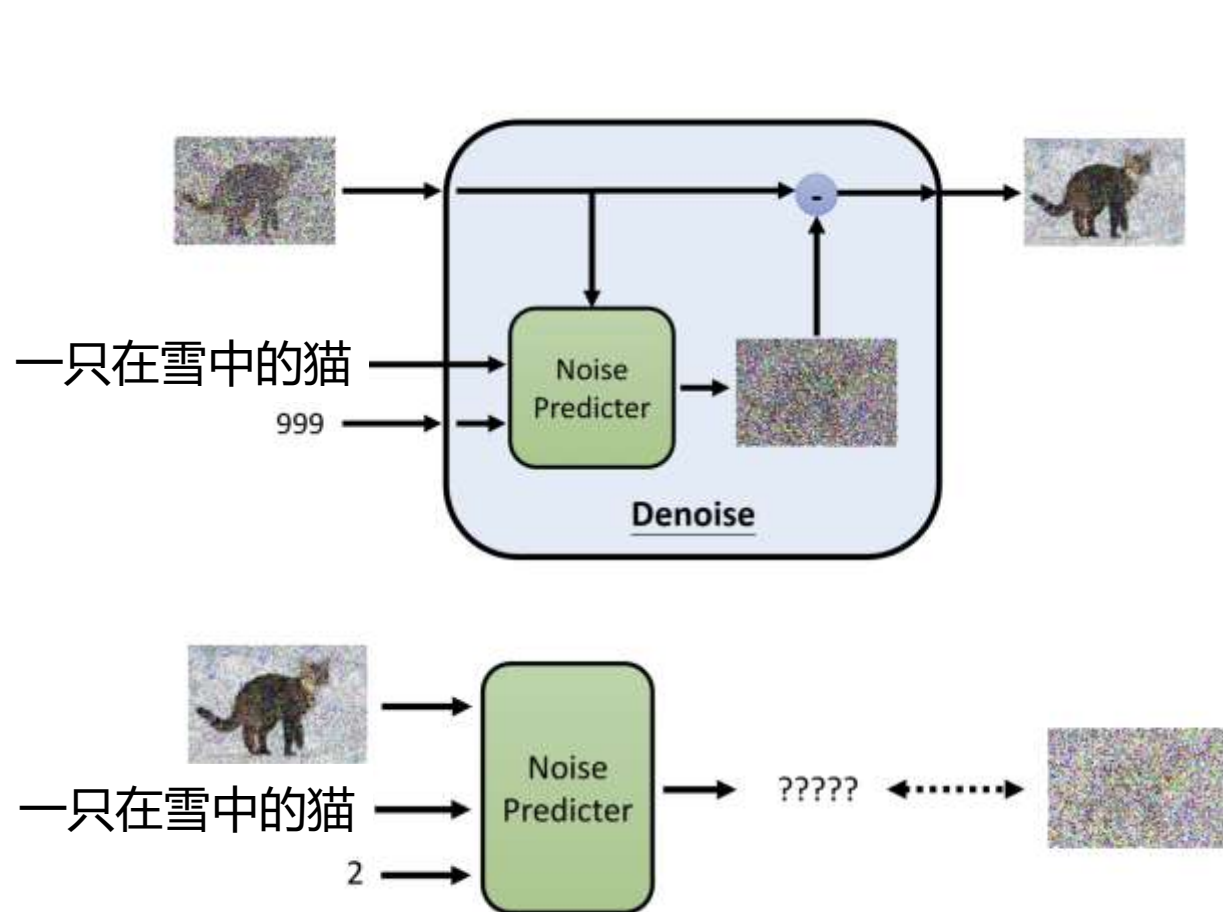
前向扩散过程：不变

反向去噪过程：为噪声预测器
引入文本输入。

文生图扩散模型与潜在扩散模型

条件生成：文生图扩散模型

噪声预测器引入文本输入



Algorithm 1 Training

- 1: repeat
- 2: $x_0, c \sim q(x_0, c)$ • 从图文数据对中采样
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 4: $\epsilon \sim N(0, I)$ • 噪声预测器引入文本输入
- 5: Take gradient descent step on
- 6: $\|\epsilon - \epsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon, c, t)\|$
- 7: until converged

文生图扩散模型与潜在扩散模型

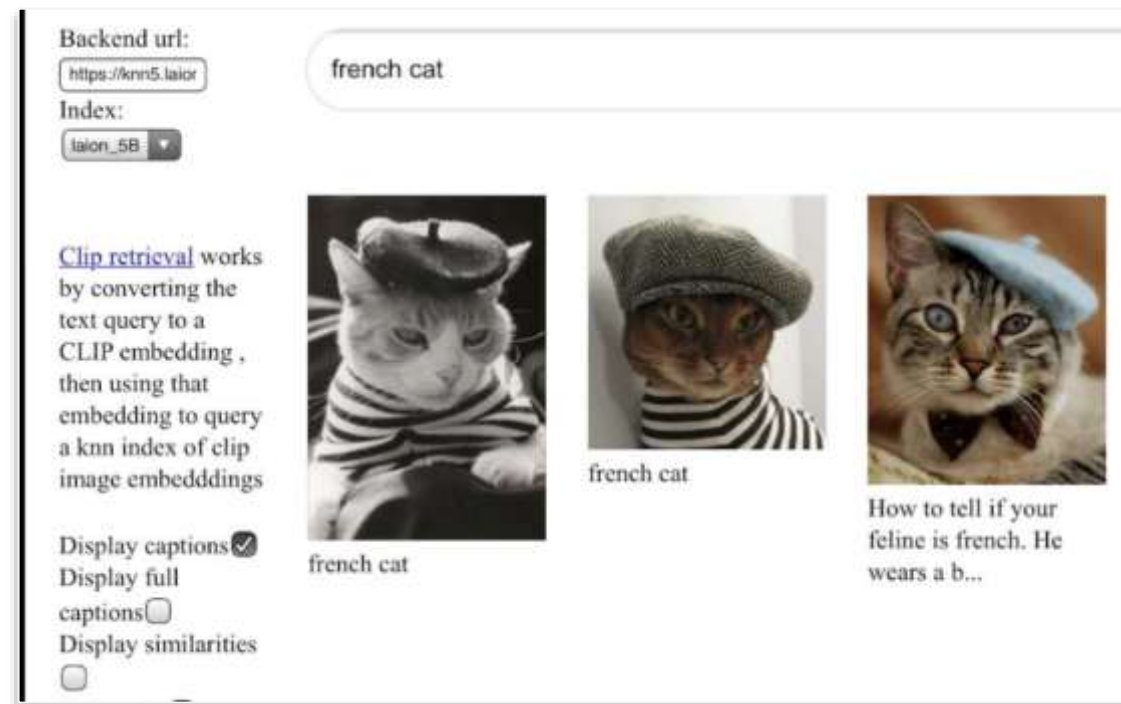
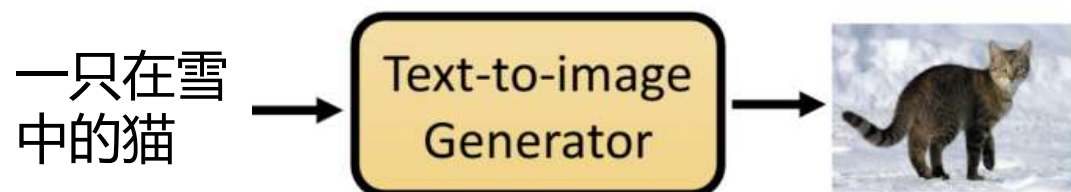
条件生成：文生图扩散模型

Flowers
8k

ImageNet
1M

LAION
5B

- 噪声预测器引入文本输入
- 需要大量的文本-图像数据对训练

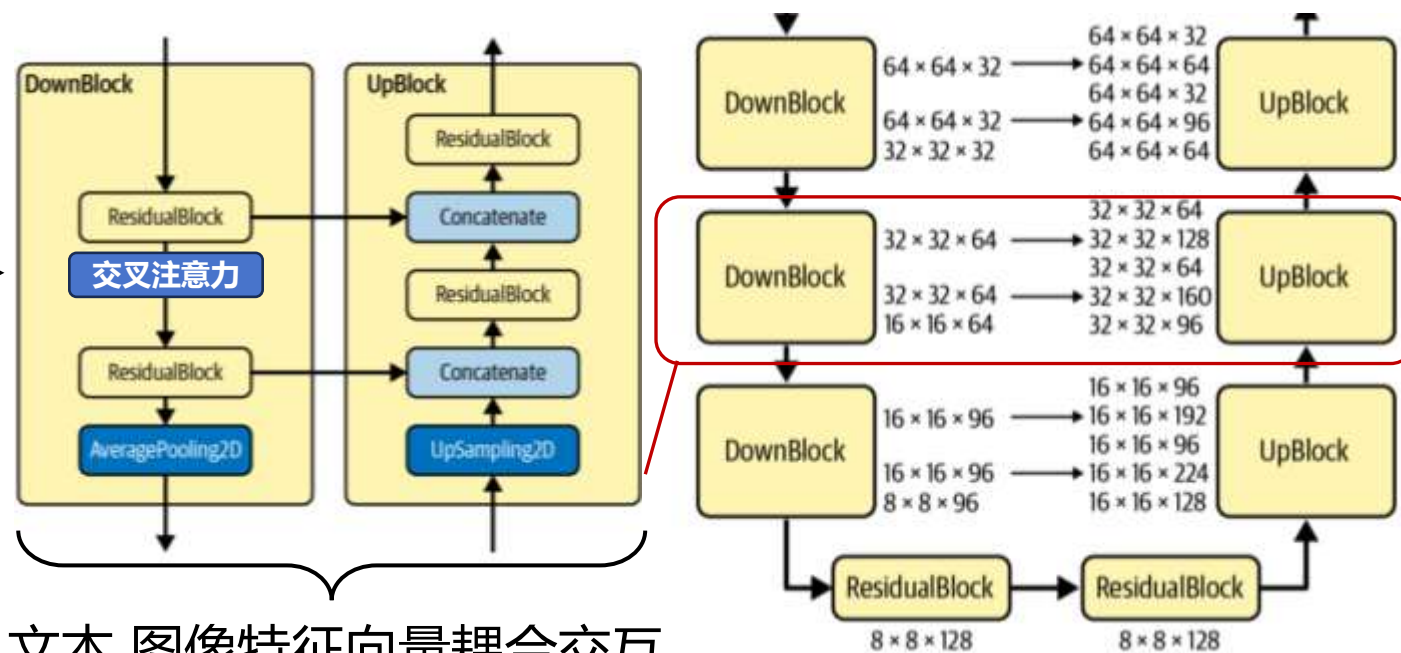
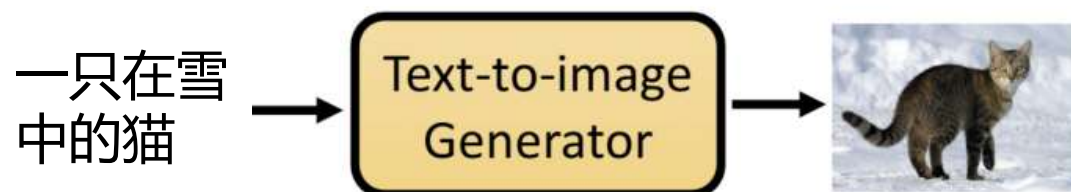
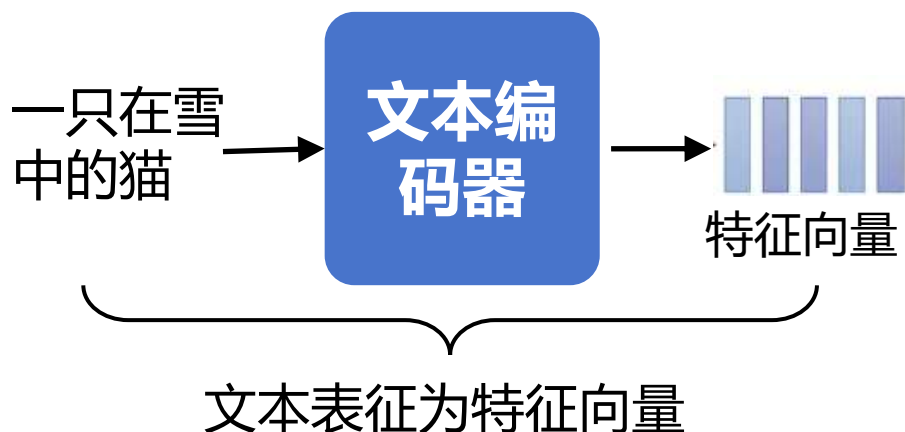


文生图扩散模型与潜在扩散模型

条件生成：文生图扩散模型

➤ 文本条件如何嵌入？

核心思想： 使用文本编码器，结合交叉注意力机制引入文本信息



文本-图像特征向量耦合交互

文生图扩散模型与潜在扩散模型

条件生成：文生图扩散模型

➤ 文生图模型如何推理呢？

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$$



$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, c, t) \right) + \sigma_t z$$

一只在雪
中的猫



骑马的宇航员



实践中发现，生成效果不佳

文生图扩散模型与潜在扩散模型

条件生成：文生图扩散模型

➤ Classifier-Free Guidance (CFG)技巧增强文

本条件表达

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$$

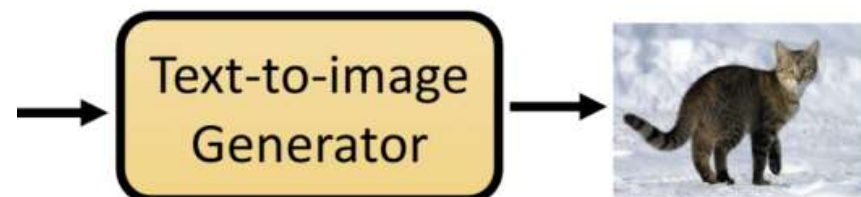


$$\epsilon'_{\theta}(x_t, c, t) = \epsilon_{\theta}(x_t, t) + \gamma(\epsilon_{\theta}(x_t, c, t) - \epsilon_{\theta}(x_t, t))$$



$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon'_{\theta}(x_t, c, t) \right) + \sigma_t z$$

一只在雪
中的猫



- 估计出更倾向于表达语义的向量方向
- $\gamma > 1$, 强化语义

文生图扩散模型与潜在扩散模型

条件生成：文生图扩散模型

- Classifier-Free Guidance (CFG)技巧增强文本条件表达

一只在雪中的猫

Text-to-image
Generator



$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$$



$$\epsilon'_{\theta}(x_t, c, t) = \epsilon_{\theta}(x_t, t) + \gamma (\epsilon_{\theta}(x_t, c, t) - \epsilon_{\theta}(x_t, t))$$



$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon'_{\theta}(x_t, c, t) \right) + \sigma_t z$$

- γ 越大语义表达越强
- γ 过大会影响图像质量

增大
 γ



文生图扩散模型与潜在扩散模型

条件生成：文生图扩散模型

- Classifier-Free Guidance (CFG)技巧增强文本条件表达

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$$

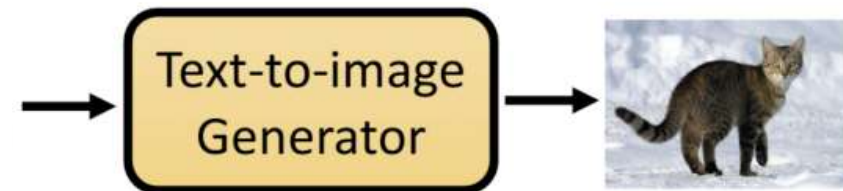


$$\epsilon'_{\theta}(x_t, c, t) = \epsilon_{\theta}(x_t, t) + \gamma(\epsilon_{\theta}(x_t, c, t) - \epsilon_{\theta}(x_t, t))$$



$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon'_{\theta}(x_t, c, t) \right) + \sigma_t z$$

一只在雪
中的猫



Algorithm 1 Training

- 1: repeat
- 2: $x_0, c \sim q(x_0, c)$ → 从图文数据对中采样
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 4: $\epsilon \sim N(0, I)$
- 5: Take gradient descent step on
- 6: $\|\epsilon - \epsilon_{\theta}(\sqrt{\alpha_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon[c], t)\|$ ↗ 所有模型输入都包含 c
- 7: until converged

训练过程没见过 $\epsilon_{\theta}(x_t, t)$ 怎么办?

文生图扩散模型与潜在扩散模型

条件生成：文生图扩散模型

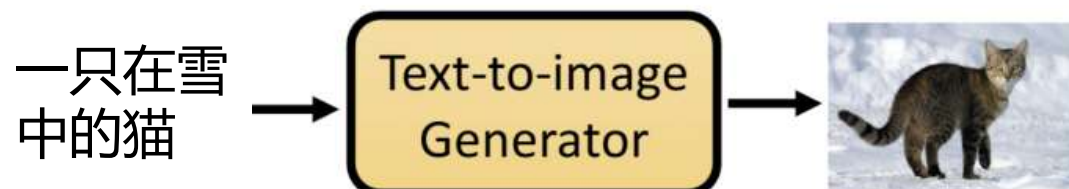
➤ 训练过程没见过 $\epsilon_{\theta}(x_t, t)$ 怎么办？

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$$

$$\epsilon'_{\theta}(x_t, c, t) = \epsilon_{\theta}(x_t, t) + \gamma(\epsilon_{\theta}(x_t, c, t) - \epsilon_{\theta}(x_t, t))$$

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon'_{\theta}(x_t, c, t) \right) + \sigma_t z$$

解决方案：训练过程中一定概率赋值文本条件为空集，使模型同时具备有/无条件生成的能力



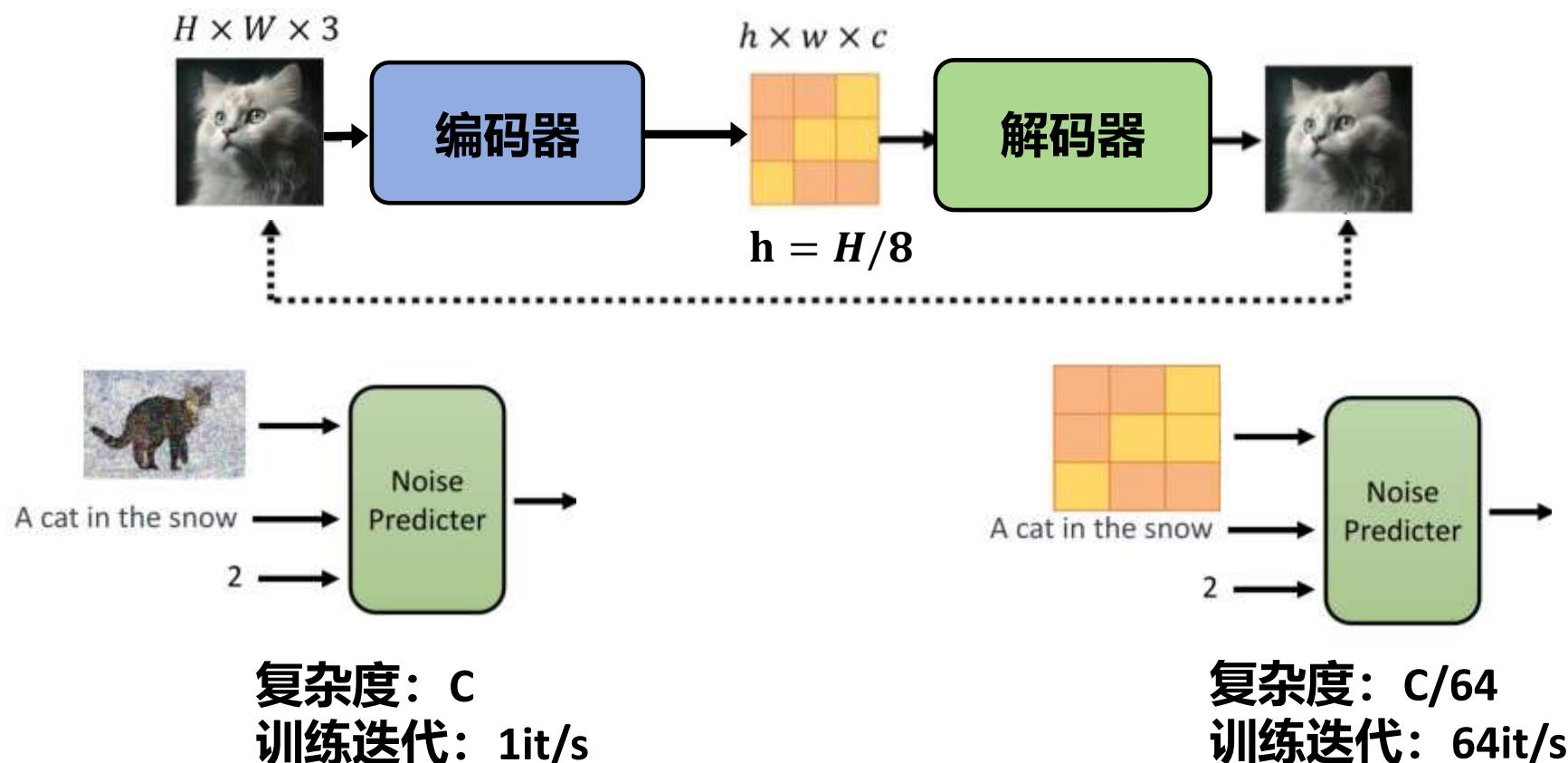
Algorithm 1 Training

- 1: repeat
- 2: $x_0, c \sim q(x_0, c)$
- 3: $c = \emptyset$ with possibility η → 以一定的概率将 c 置为空
- 4: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 5: $\epsilon \sim N(0, I)$
- 6: Take gradient descent step on
- 7: $\|\epsilon - \epsilon_{\theta}(\sqrt{\alpha_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, c, t)\|$
- 8: until converged

文生图扩散模型与潜在扩散模型

潜在扩散模型

通过在VAE低维隐层上构建扩散生成模型，平衡计算复杂度与生成质量

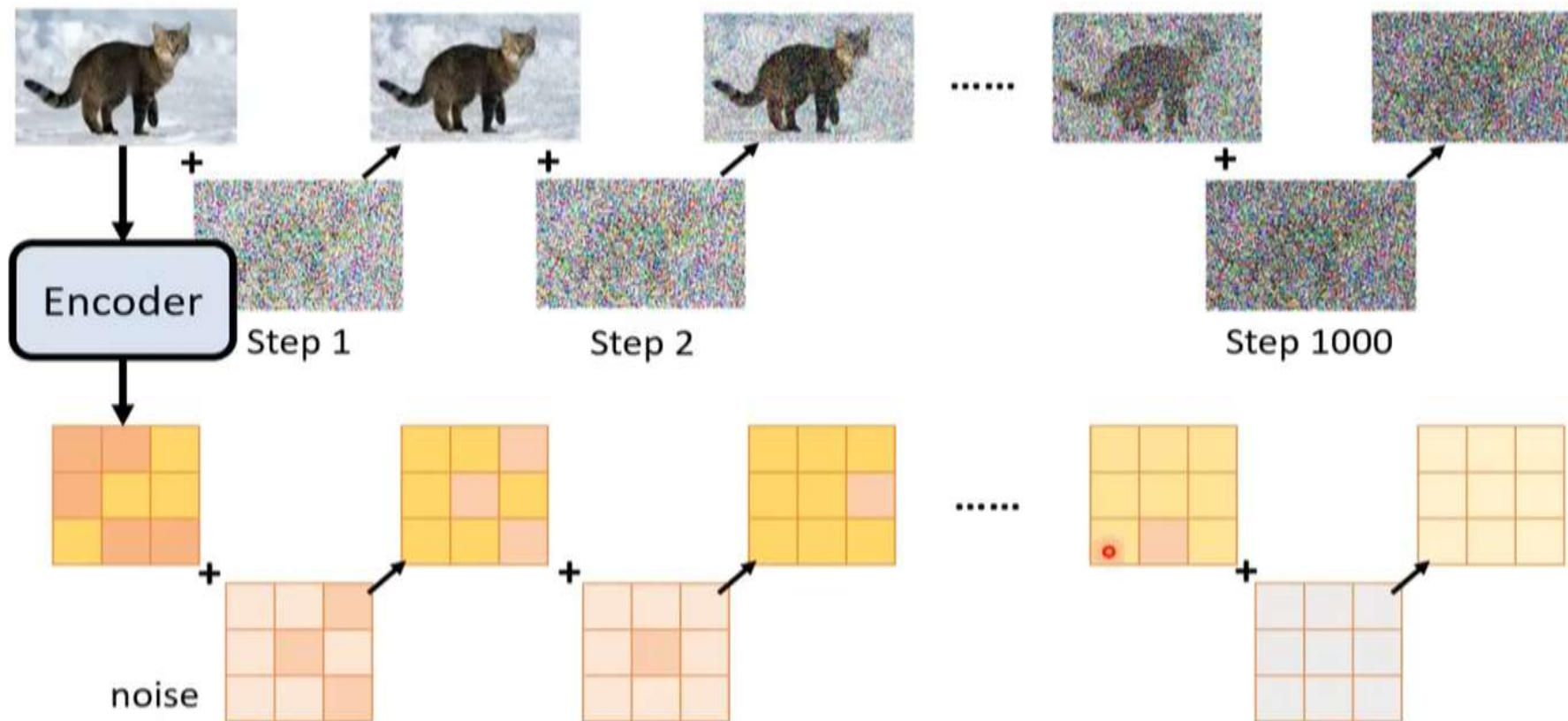


文生图扩散模型与潜在扩散模型

潜在扩散模型

前向扩散过程：在VAE隐层进行扩散加噪

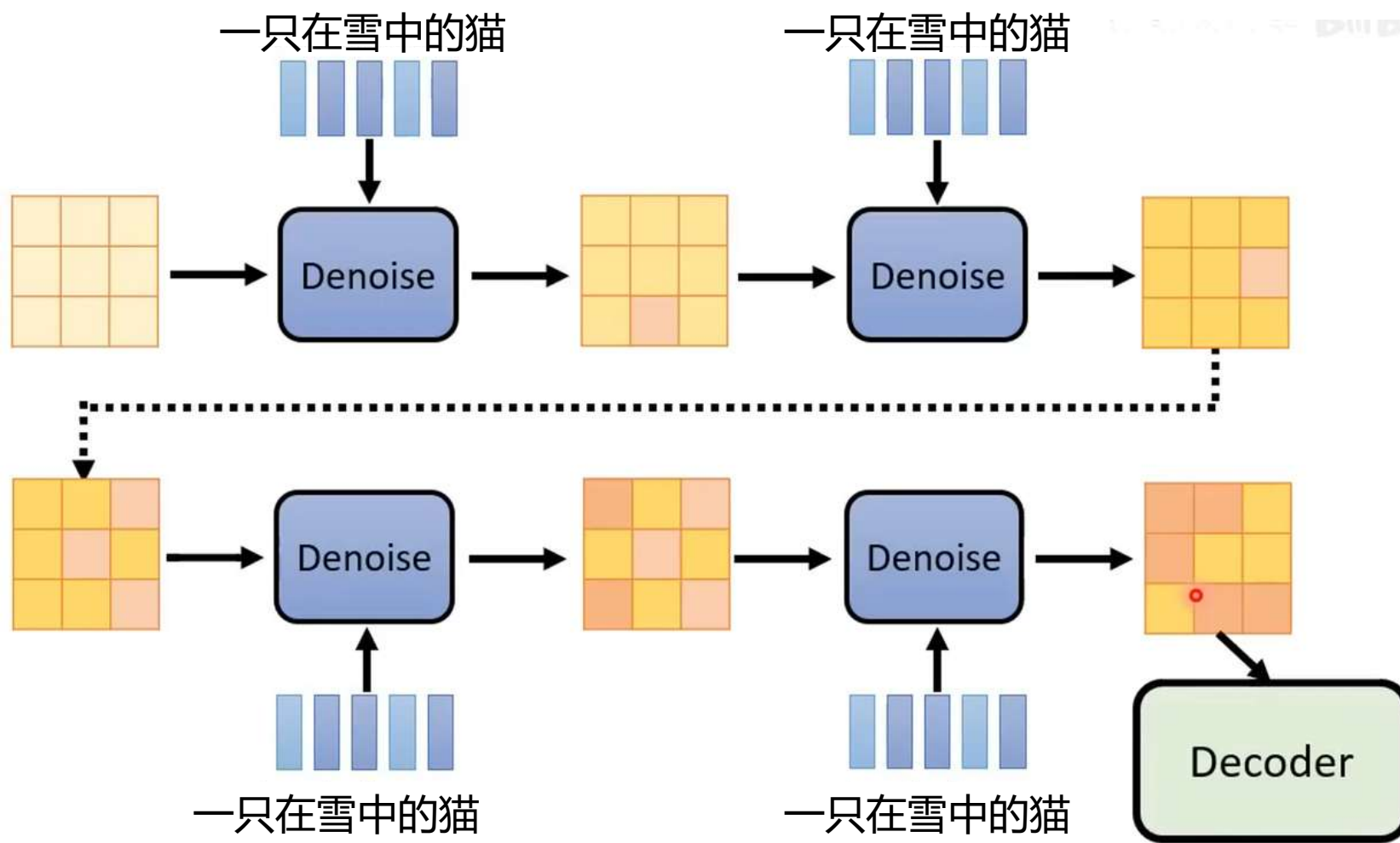
一只在雪中的猫



文生图扩散模型与潜在扩散模型

潜在扩散模型

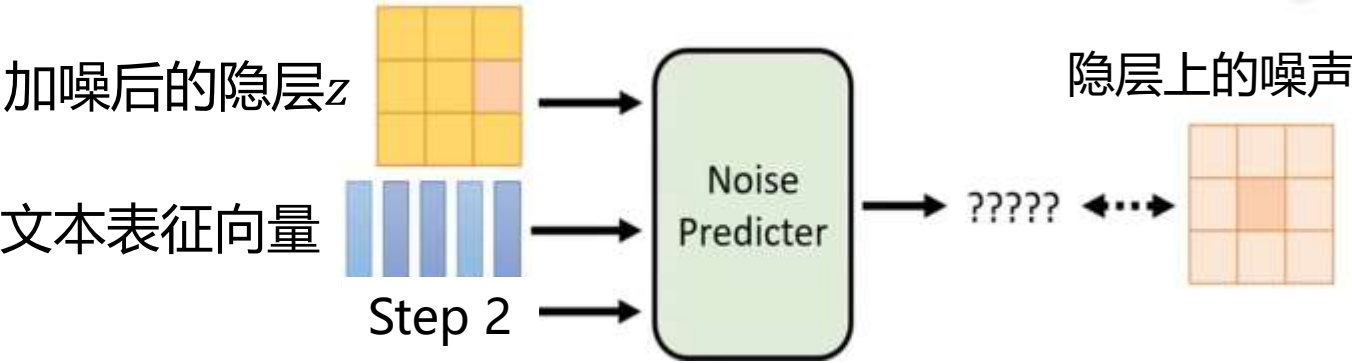
反向去噪过程：在隐层中完成去噪，最后通过VAE decoder解码出图像



文生图扩散模型与潜在扩散模型

潜在扩散模型

反向去噪过程：在隐层中完成去噪，最后通过VAE decoder解码出图像



Algorithm 1 Training

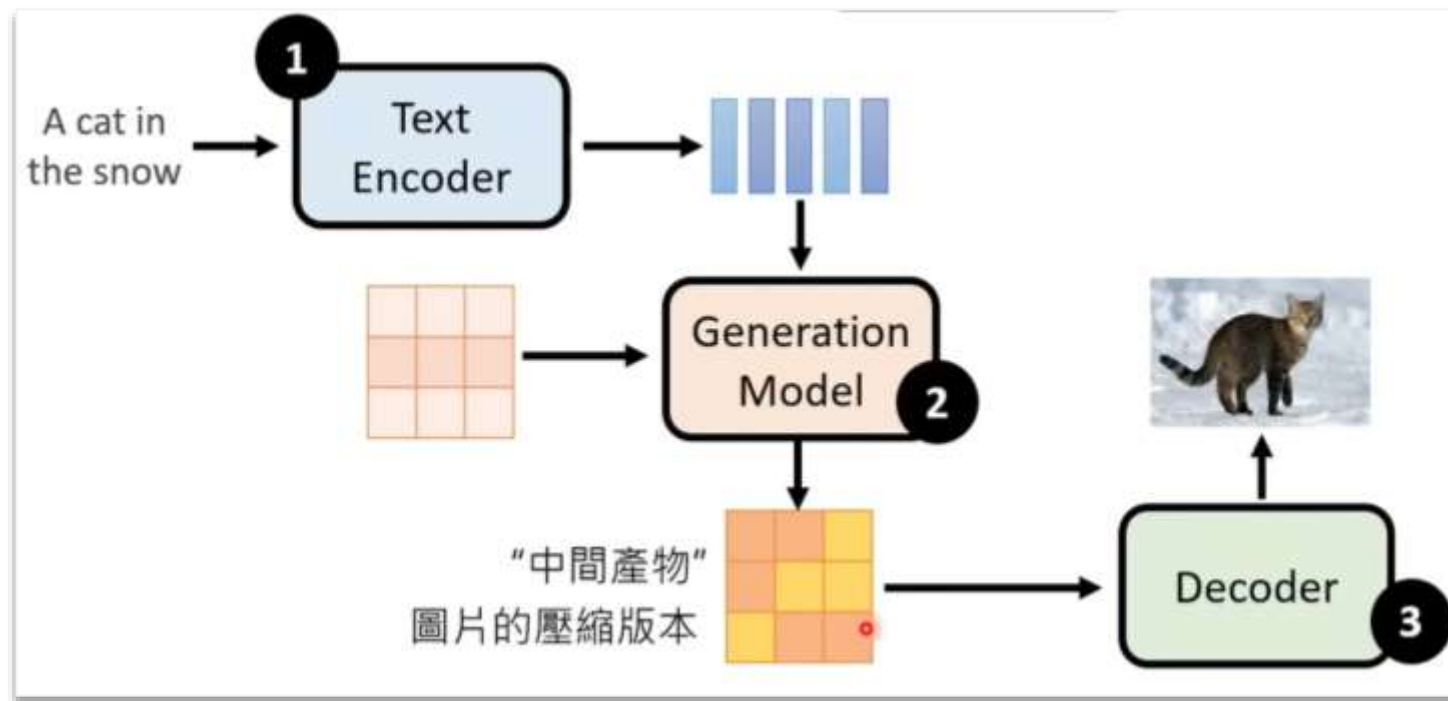
- 1: repeat
- 2: $x_0, c \sim q(x_0, c)$
- 3: $z_0 = \text{Encoder}(x_0)$
- 4: $c = \emptyset$ with possibility η
- 5: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 6: $\epsilon \sim N(0, I)$
- 7: Take gradient descent step on
- 8: $\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, c, t)\|$
- 9: until converged

文生图扩散模型与潜在扩散模型

潜在扩散模型

➤ 生成推理过程

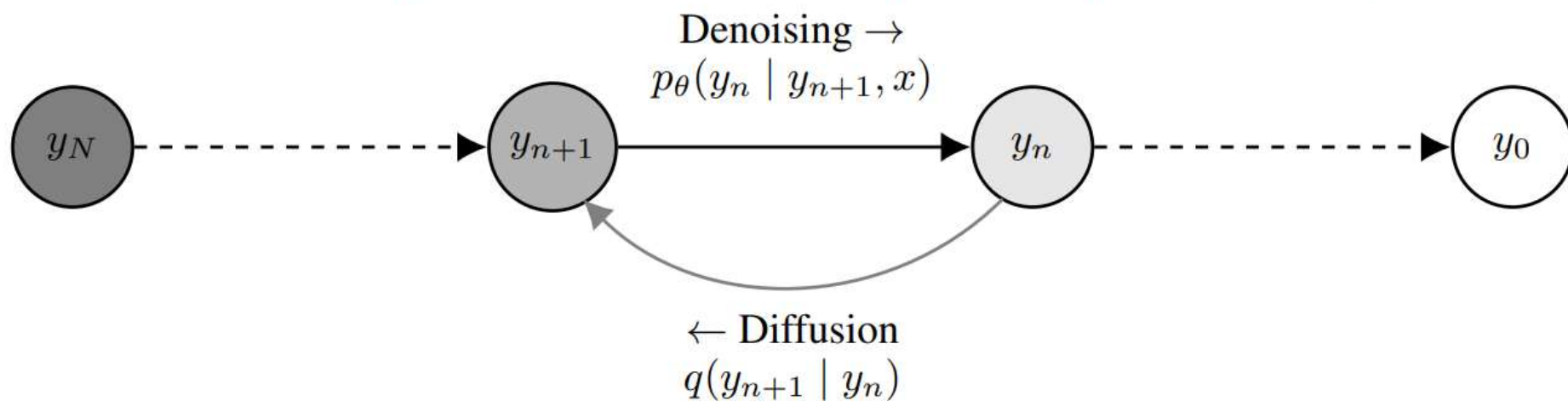
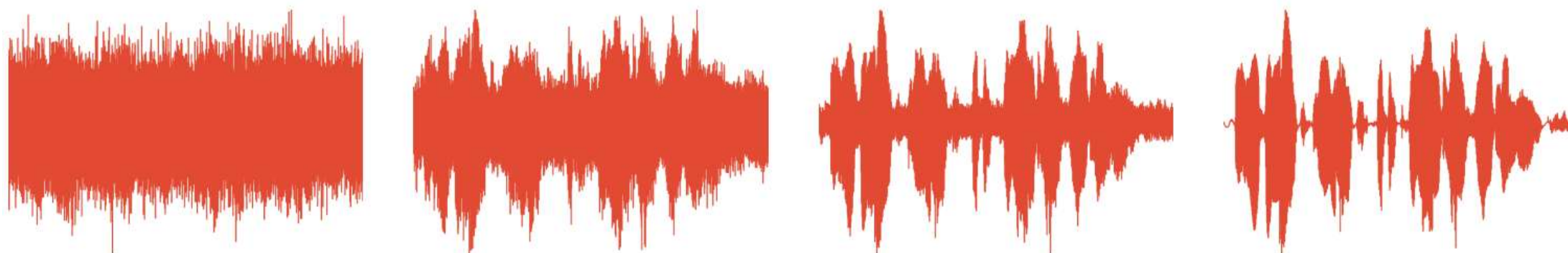
- 文本编码器：构建文本表征向量 c
- 扩散生成模型：输入文本条件 c , 迭代生成隐变量 z
- 解码器：输入隐变量 z , 输出图像 x



目录

- 第一节 扩散模型的基本原理
- 第二节 扩散模型的极大似然估计角度理解
- 第三节 扩散模型的代码实践
- 第四节 文生图扩散模型与潜在扩散模型
- 第五节 非视觉模态扩散模型

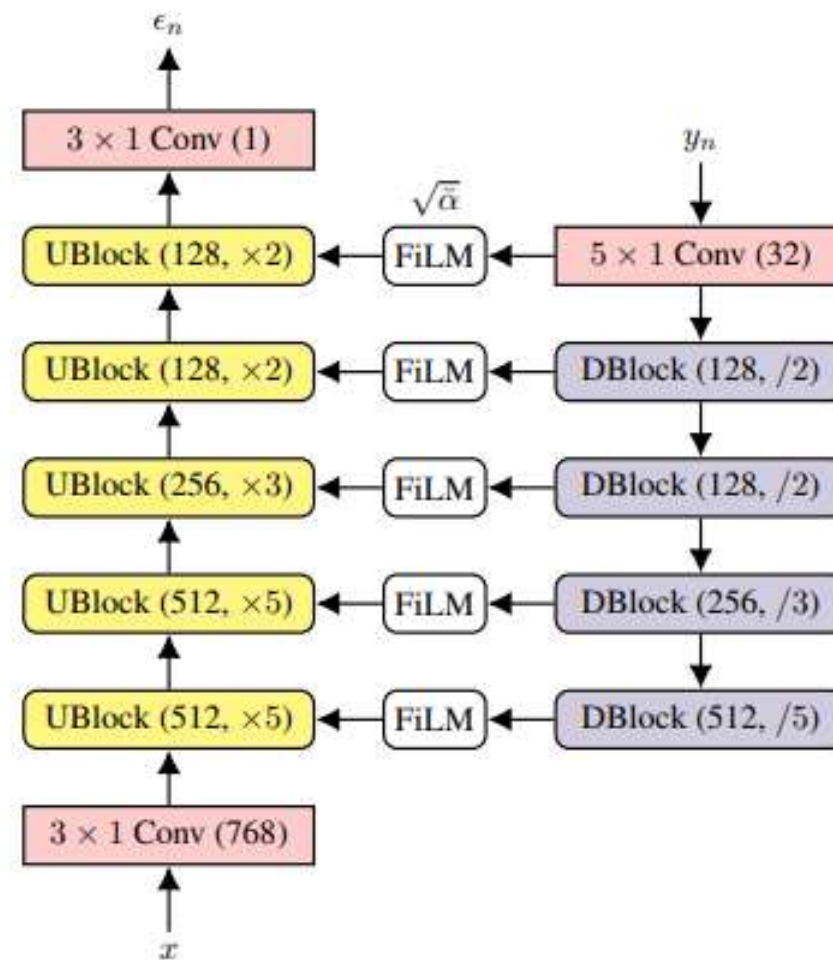
语音扩散模型——WaveGrad



第五章 扩散去噪生成模型

语音扩散模型——WaveGrad

- DBlock: 下采样噪声音频, 提取特征。
- UBlock: 上采样生成预测, 恢复分辨率。
- FiLM: 融合音频特征、mel谱、噪声水平, 调制网络输出。



第五章 扩散去噪生成模型

语音扩散模型——WaveGrad

Algorithm 1 Training. WaveGrad directly conditions on the continuous noise level $\sqrt{\bar{\alpha}}$. l is from a predefined noise schedule.

```
1: repeat  
2:    $y_0 \sim q(y_0)$   
3:    $s \sim \text{Uniform}(\{1, \dots, S\})$   
4:    $\sqrt{\bar{\alpha}} \sim \text{Uniform}(l_{s-1}, l_s)$   
5:    $\epsilon \sim \mathcal{N}(0, I)$   
6:   Take gradient descent step on  
      $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}} y_0 + \sqrt{1 - \bar{\alpha}} \epsilon, \boxed{x}, \sqrt{\bar{\alpha}})\|_1$   
7: until converged
```

- WaveGrad 是条件语音扩散模型，也以文字为条件吗？
- 文字和音频无法精确时序对齐。
 - 文字丢失韵律、音高、情感等声学细节。
 - 模型需同时学习语言学+声学+信号处理。

第五章 扩散去噪生成模型

语音扩散模型——WaveGrad

Algorithm 1 Training. WaveGrad directly conditions on the continuous noise level $\sqrt{\bar{\alpha}}$. l is from a predefined noise schedule.

```
1: repeat  
2:    $y_0 \sim q(y_0)$   
3:    $s \sim \text{Uniform}(\{1, \dots, S\})$   
4:    $\sqrt{\bar{\alpha}} \sim \text{Uniform}(l_{s-1}, l_s)$   
5:    $\epsilon \sim \mathcal{N}(0, I)$   
6:   Take gradient descent step on  
      $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}} y_0 + \sqrt{1 - \bar{\alpha}} \epsilon, \boxed{x}, \sqrt{\bar{\alpha}})\|_1$   
7: until converged
```

➤ 以 mel 谱为条件

- **定义：**Mel 谱是音频的“声学指纹”，记录每个瞬间不同频率的能量分布。

第五章 扩散去噪生成模型

语音扩散模型——WaveGrad

➤ 传统方法训练与推理的噪声调度能否不同？

- t 或 α 的任意不同都会导致模型遇到未见过的噪声水平，从而生成质量严重下降。
- 一个模型服务一种场景
- 调参成本极高

Algorithm 1 Training. WaveGrad directly conditions on the continuous noise level $\sqrt{\bar{\alpha}}$. l is from a predefined noise schedule.

```
1: repeat  
2:    $y_0 \sim q(y_0)$   
3:    $s \sim \text{Uniform}(\{1, \dots, S\})$   
4:    $\sqrt{\bar{\alpha}} \sim \text{Uniform}(l_{s-1}, l_s)$   
5:    $\epsilon \sim \mathcal{N}(0, I)$   
6:   Take gradient descent step on  
       $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}} y_0 + \sqrt{1 - \bar{\alpha}} \epsilon, x, \sqrt{\bar{\alpha}})\|_1$   
7: until converged
```

第五章 扩散去噪生成模型

语音扩散模型——WaveGrad

解决方案：连续噪声水平条件替代离散迭代索引

Algorithm 1 Training. WaveGrad directly conditions on the continuous noise level $\sqrt{\bar{\alpha}}$. l is from a predefined noise schedule.

```
1: repeat  
2:    $y_0 \sim q(y_0)$   
3:    $s \sim \text{Uniform}(\{1, \dots, S\})$   
4:    $\sqrt{\bar{\alpha}} \sim \text{Uniform}(l_{s-1}, l_s)$   
5:    $\epsilon \sim \mathcal{N}(0, I)$   
6:   Take gradient descent step on  
      $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}} y_0 + \sqrt{1 - \bar{\alpha}} \epsilon, x, \sqrt{\bar{\alpha}})\|_1$   
7: until converged
```

- 定义 S (相当于 t)个段
- 计算每个段对应的噪声水平 l_s (相当于传统的 t 时刻的 $\sqrt{\bar{\alpha}}$)
- 在 $[l_{s-1}, l_s]$ 采样 $\sqrt{\bar{\alpha}}$

直接将 $\sqrt{\bar{\alpha}}$ 作为模型输入

第五章 扩散去噪生成模型

语音扩散模型——WaveGrad

Algorithm 2 Sampling. WaveGrad generates samples following a gradient-based sampler similar to Langevin dynamics.

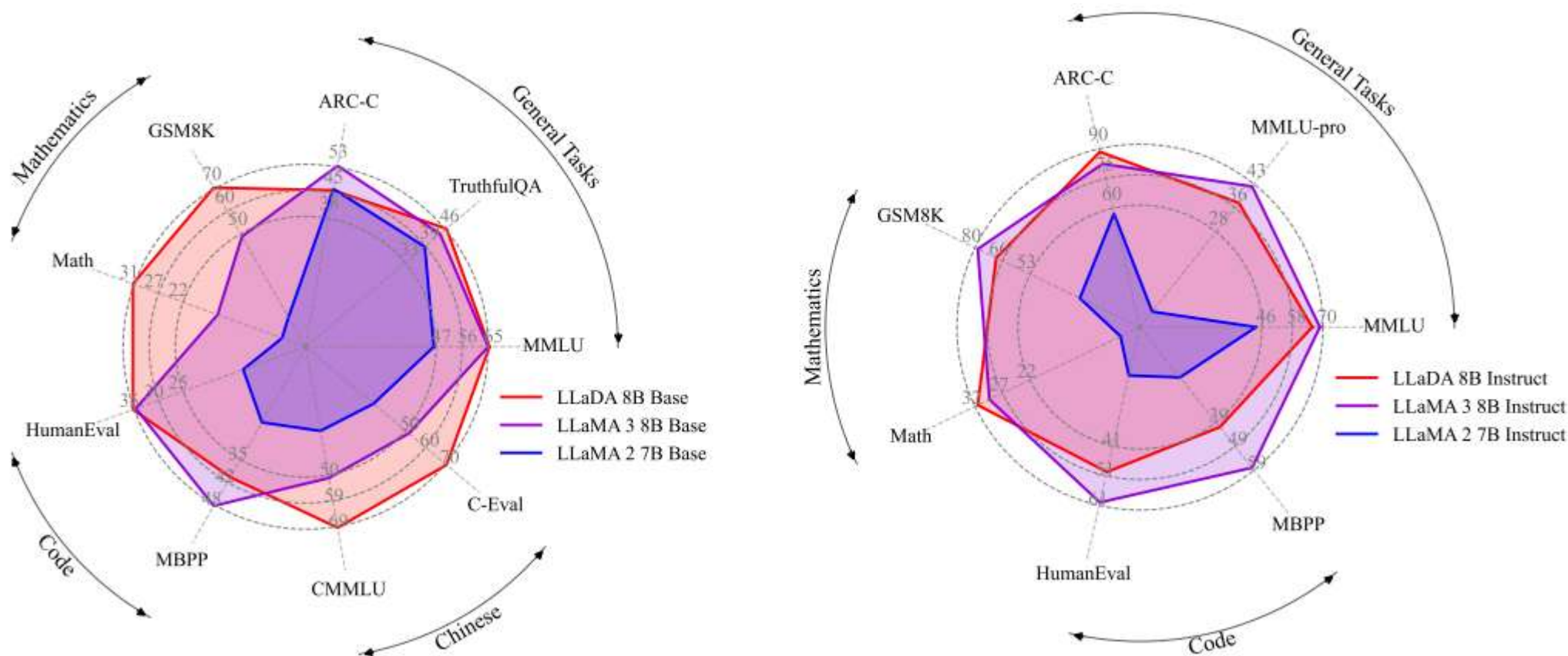
```
1:  $y_N \sim \mathcal{N}(0, I)$ 
2: for  $n = N, \dots, 1$  do
3:    $z \sim \mathcal{N}(0, I)$ 
4:    $y_{n-1} = \frac{\left( y_n - \frac{1-\alpha_n}{\sqrt{1-\alpha_n}} \epsilon_\theta(y_n, x, \sqrt{\bar{\alpha}_n}) \right)}{\sqrt{\alpha_n}}$ 
5:   if  $n > 1$ ,  $y_{n-1} = y_{n-1} + \sigma_n z$ 
6: end for
7: return  $y_0$ 
```

和传统方式相同的 $\sqrt{\bar{\alpha}_n}$

第五章 扩散去噪生成模型

文本扩散模型——LLaDA

LLaDA 已经具备了与目前最流行的自回归模型相竞争的能力



第五章 扩散去噪生成模型

文本扩散模型——LLaDA

用离散分布取代高斯分布

